Quantum Gates Decompositions

Table of Contents

Decomposition of Multi-controlled Special Unitary Single-Qubit Gates

Rafaella Vale, Thiago Melo D. Azevedo, Ismael C. S. Araújo, Israel F. Araujo, Adenilton J. da Silva

Abstract—Multi-controlled unitary gates have been a subject of interest in quantum computing since its inception, and are widely used in quantum algorithms. The current state-of-the-art approach to implementing n-qubit multi-controlled gates involves the use of a quadratic number of single-qubit and CNOT gates. However, linear solutions are possible for the case where the controlled gate is a special unitary SU(2). The most widely-used decomposition of an n-qubit multi-controlled SU(2) gate requires a circuit with a number of CNOT gates proportional to 28n. In this work, we present a new decomposition of n-qubit multi-controlled SU(2) gates that requires a circuit with a number of CNOT gates proportional to 20n, and proportional to 16n if the SU(2) gate has at least one real-valued diagonal. This new approach significantly improves the existing algorithm by reducing the number of CNOT gates and the overall circuit depth. As an application, we show the use of this decomposition for sparse quantum state preparation. Our results are further validated by demonstrating a proof of principle on a quantum device accessed through quantum cloud services.

Index Terms—Quantum Computing, Quantum Circuit Optimization, Quantum Gate Decomposition, Multi-controlled Quantum Gates.

1 Introduction

The prospect of quantum speedup for some computational tasks, such as prime number factoring [1] and unstructured search [2] motivated research on novel quantum computing applications. However, quantum devices in the current technology, referred to as Noisy Intermediate-Scale Quantum (NISQ) devices, are constrained by the number of qubits and amount of noisy operations [3]. For this reason, using quantum error correction techniques becomes nonviable for near-term hardware due to the overhead cost in additional required qubits and the increased gate count [4].

To move towards practical quantum advantage [5], we need to overcome these limitations in quantum devices by achieving, for instance, noise reduction in quantum operations. There are different approaches to this problem. One possible solution is to reduce the depth and gate count of quantum circuits through quantum circuit optimization techniques [6]. Strategies to reduce circuit depth include compilation processes [7], the use of auxiliary qubits [8], divide-and-conquer approaches [6], and approximated quantum circuits [9].

The decomposition of multi-controlled gates into a set of elementary gates is necessary for algorithm implementation in current quantum devices. Several works contributed to the decomposition of unitary matrices into quantum circuits. Some take into consideration general unitary gates [10], [11], while others tackle more specific operations, such as multi-controlled single-qubit gates [12], special unitary gates [10], [11], σ_x [8], [10], [11], [13], [14], [15], [16], R_x [17], and phase gates [18].

Ref. [12] shows how to decompose an n-controlled single-qubit gate U with $O(n^2)$ gates and linear circuit depth. However, this decomposition requires the calculation of $\sqrt[2^n]{U}$, which generates numerical errors and does not allow the decomposition of controlled gates with any number of qubits. An n-qubit multi-controlled $SU(2)$ gate requires $28n - 88$ CNOT gates for even n and $28n - 92$ CNOT gates for odd n [10], [11]. Here, we show how to decompose an n-qubit multi-controlled $SU(2)$ gate with at most $20n-38$ CNOTs. If the n-qubit multi-controlled $SU(2)$ has at least one real-valued diagonal, the proposed decomposition requires at most $16n - 40$ CNOTs. The proposed decomposition is based on [8] but does not require auxiliary qubits.

The proposed method has applications in several quantum algorithms [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], for instance, in quantum machine learning, unitary matrix compilation into quantum circuits and quantum state preparation. In this work, we show the impact of the proposed method in sparse quantum state initialization.

The rest of this paper has four sections. Section 2 describes the related works. Section 3 is the main section. We first show how to decompose a multi-controlled $SU(2)$ gate with one real diagonal with the number of CNOTs proportional to $16n$ and then use this result to decompose any multi-controlled $SU(2)$ with the number of CNOTs proportional to $20n$. Section 4 shows the application of the proposed decomposition to reduce the amount of CNOT gates to initialize a sparse quantum state. Section 5 presents the conclusion.

2 Related Work

An implementation of an n-controlled $R_x(\pi)$ gate was proposed in [13]. This construction consists in chaining controlled gates so that operations with different controls and target qubits are applied in parallel. That construction

- *R. Vale, T.M.D. Azevedo, I.C.S. Araújo and A.J. da Silva are with Centro de Informática, Universidade Federal de Pernambuco, Recife, Pernambuco, Brazil.*
- *I.F Araujo is with Department of Statistics and Data Science, Yonsei University, Seoul, Republic of Korea.*

Manuscript received April 19, 2005; revised August 26, 2015.

was further generalized to an n-controlled unitary with linear depth and $4n^2 - 12n + 10$ CNOTs [12]. However, to calculate the angles used on the operators those methods rely heavily on computing fractions of π such as $\pi/2^{n-1}$ for a number n of control qubits or computing the 2^n–th root of the desired operator as in $\sqrt[2^{n-1}]{U}$ in the case of [12], leading to numerical errors when calculating angles in systems with many qubits.

The authors in [10] present several gate decompositions for operations with single and multiple controls. The method for multi-controlled gates in $U(2)$ is subject to numerical errors as in [12] and uses $16n^2 - 60n + 42$ CNOTs. The gate decomposition for multi-controlled gates in $SU(2)$ has linear complexity in terms of basic operations and produces circuits with $32n - 74$ CNOTs. In [11] the authors lay out a construction that further optimizes the decomposition of $SU(2)$ gates in [10] requiring $28n - 88$ CNOT gates for even n and $28n - 92$ CNOT gates for odd n. This optimization is accomplished by utilizing an approximate version of the Toffoli gate with a phase change on state $|010\rangle$.

The construction shown in [8] is the basis for the decomposition scheme proposed in this paper, whereas the works of [10] and [11] are used for comparison. Thus, they will be summarized in Section 2.1 and Section 2.2.

2.1 Multi-controlled special unitary gates

In order to build an $(n-1)$-controlled version of a gate $U \in SU(2)$ as [10], we must find a gate decomposition as illustrated in Fig. 1. Then we can use the $(n-1)$-th qubit as an auxiliary for decomposing the two multi-controlled σ_x gates more efficiently. Such scheme is derived from the fact that the gate U can be decomposed into three gates A, B and C such that $ABC = I$ and $U = A\sigma_x B\sigma_x C$, where A, B and $C \in SU(2)$ if and only if $U \in SU(2)$ [10, Lemma 7.9].

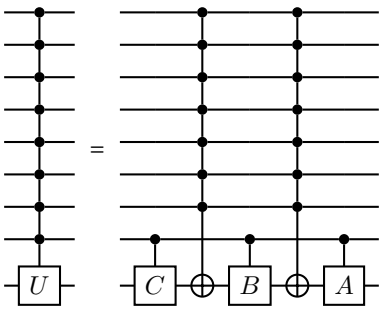


Fig. 1: Decomposition of an 8-controlled U gate, where $U, A, B, C \in SU(2)$.

Since we know that A, B and C are also special unitary gates, then their controlled versions can be decomposed according to a similar reasoning used for the $(n-1)$-controlled U (Fig. 2). But this construction would only need to take into account a single-control gate [10, Lemma 5.1]. With those constructions introduced, we can focus on the $(n-2)$-controlled σ_x gates with the extra $(n-1)$-th qubit as an auxiliary.

On an n-qubit system where $n \geq 5$ with one auxiliary qubit, an $(n-2)$-controlled σ_x can be decomposed into two

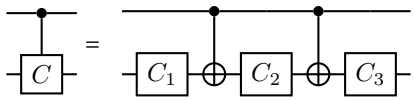


Fig. 2: Single-control C gate, where $C, C_1, C_2, C_3 \in SU(2)$.

pairs of m_1-controlled σ_x and m_2-controlled σ_x gates, where $m_2 = \lceil n/2 \rceil$ and $m_1 = n - m_2 - 1$ [10, Lemma 7.3]. Each gate is further decomposed into a chain of Toffoli gates, with the unused qubits as auxiliary qubits as illustrated in Fig. 3. The first step is dedicated to flipping the target qubit and the second to reverting the auxiliary qubits. The auxiliary qubit can be dirty since they are reset to their original values.

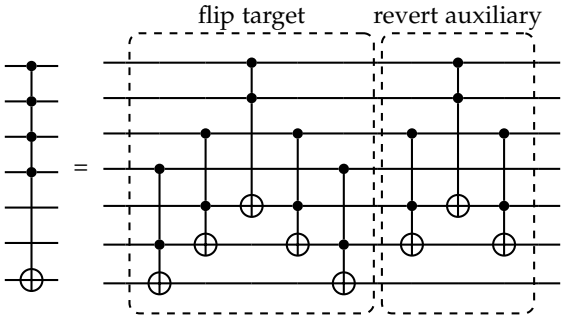


Fig. 3: Decomposition of a 4-controlled σ_x in a 7-qubit system with two dirty auxiliary qubits [10, Lemma 7.2]. In the optimizations proposed in [11, Lemma 8], all Toffoli gates that target the auxiliary qubits are approximated.

The construction of the chain of Toffoli gates can be further optimized [10], [11]. By choosing an appropriate rotation gate (for example, $R_y(\pi/4)$), an approximated Toffoli operation can be built with just three CNOTs. However, the local phase for some input states will differ from the action of an actual Toffoli gate, which makes this optimization equivalent to a Toffoli up to some diagonal gate. In the example of Fig. 4, this is similar to adding a diagonal gate Δ that performs the mapping $|010\rangle \mapsto -|010\rangle$ after (or before) the Toffoli gate. This is a useful optimization when the circuit can be designed to match gates of this type in a way that forces gate canceling to further reduce the depth and gate count of the circuit.

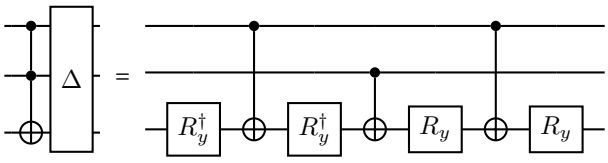


Fig. 4: Optimized version of a Toffoli gate up to a diagonal gate [10], [11]. In this example, the angles of all R_y operators are equal to $\pi/4$.

The approximate version of the Toffoli gate shown in Fig. 4 can be used to replace each auxiliary-targeting Toffoli gate on the chain illustrated in Fig. 3, where each gate pairs up with another acting on the same qubits. Doing this, we obtain a sequence of operations that eliminate the extra

phases and cancel nearly half of the gates in the circuit shown in Fig. 4. This optimization step is demonstrated in [11, Lemma 8]. This consequently enables us to construct an efficient version of the $(n-2)$-controlled σ_x gate with one auxiliary qubit.

As mentioned before, two pairs of m_1-controlled σ_x gates in the top m_1 control qubits and m_2-controlled σ_x gates in the bottom m_2 control qubits, such as the one shown in Fig. 3, implementing the optimizations discussed, are used to compose the $(n-2)$-controlled σ_x circuit. Fig. 5 illustrates how [11, Lemma 8] is used to construct this scheme stated in [11, Lemma 9]. One thing to note is that the gates controlled by the top qubits act only on the auxiliary qubits and not on the target of the main operation being decomposed. Due to this, all Toffoli gates in the top gates can be substituted by their approximate counterparts, since the auxiliary qubits are reset at the end of the operation. This particular step is also noted in [10, Corollary 7.4]. This set of optimizations for the circuit seen in Fig. 5 has a cost of $16n - 40$ CNOTs.

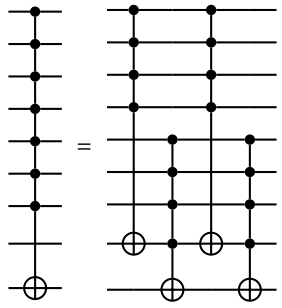


Fig. 5: Decomposition of a 7-controlled σ_x gate with one free qubit, which can be dirty, that can be used as an auxiliary. If the four controlled σ_x gates follow the optimized decomposition scheme of [11, Lemma 8] and use only approximate Toffoli gates when the bottom qubit is not targeted, the total number of CNOTs in the circuit is at most $16n - 40$.

So far, these adjustments enable the decomposition of an $(n-1)$-controlled $U \in SU(2)$ with $32n - 74$ CNOTs with two $(n-2)$-controlled σ_x and the three controlled $SU(2)$ gates. The final improvement proposed in [11, Theorem 5] uses the fact that the circuit in Fig. 5 can be reversed without affecting its action. The second occurrence of this gate in Fig. 1 can then mirror the first. This allows the canceling of the auxiliary-resetting part shown in Fig. 3 for the two bottom m_2-controlled σ_x gates surrounding the controlled B gate. The bottom m_2-controlled σ_x gate is the last gate represented in Fig. 5. With these optimizations taken into account, an $(n-1)$-controlled $U \in SU(2)$ can be built, with an upper bound of $28n - 88$ CNOTs when n is even, and $28n - 92$ CNOTs when n is odd.

The decomposition displayed in Fig. 1 can be further reduced for a special case of $SU(2)$ operators. Given an operator $U \in SU(2)$ defined as

$$U = R_z(\beta)R_y(\theta)R_z(\beta) \tag{1}$$

$$= \begin{pmatrix} e^{-i\beta}\cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & e^{i\beta}\cos\frac{\theta}{2} \end{pmatrix}, \tag{2}$$

its controlled version can be decomposed into two single-qubit gates $A, B \in SU(2)$ and two CNOTs, such that $AB = I$ and $U = A\sigma_x B\sigma_x$. This decomposition can be achieved by making $A = R_z(\beta)R_y(\theta/2)$ and $B = A^\dagger$ [10, Lemma 5.4]). Applying this special case to the decomposition of Fig. 1, one would just need to consider the controlled C gate as the identity.

2.2 Multi-controlled σ_x gates with one auxiliary qubit

In [8] the authors have proposed a gate decomposition of an n-controlled σ_x operator that uses a single dirty auxiliary qubit, illustrated in Fig. 6. In an $(n+2)$-qubit system, with $n \geq 3$ plus the auxiliary qubit, the procedure involves splitting the control qubits into two groups of k_1 and k_2 qubits, where $k_1 + k_2 = n$. Then, both control qubit groups are used to manipulate the phase of the auxiliary qubit by using the σ_x gate and the phase gate S, which can also be seen as flipping the phase of the target qubit.

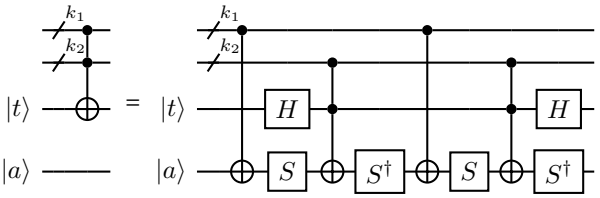


Fig. 6: Decomposition scheme of an n-controlled σ_x operator with a single auxiliary qubit $|a\rangle$ and target qubit $|t\rangle$.

The operation has no effect on the target qubit when all control qubits are 0. The action of the circuit satisfies the following equation on the auxiliary qubit when at least one of the k_1 controls is 0 and all k_2 controls are 1:

$$S^\dagger\sigma_x S S^\dagger\sigma_x S = I, \tag{3}$$

and the following equation when the k_1 controls are 1 and at least one of the k_2 controls is 0:

$$S S^\dagger\sigma_x S S^\dagger\sigma_x = I. \tag{4}$$

What remains to ensure the action of σ_x on the target qubit is to flip the phase of the auxiliary qubit with the sequence of transformations

$$S^\dagger\sigma_x S\sigma_x S^\dagger\sigma_x S\sigma_x = -I \tag{5}$$

when all controls are 1, corresponding to a controlled σ_z on the target, and to apply the Hadamard operators on both ends of the circuit, since $H\sigma_z H = \sigma_x$.

3 Decomposition of multi-controlled single-qubit $SU(2)$ gates

Different schemes can be used to decompose multi-controlled $SU(2)$ operators [10], [11]. The construction from [10, Lemma 7.9] results in linear depth and a linear number of gates. The previous optimal number of CNOTs is obtained using the optimizations of [11, Theorem 5], with the total number of CNOTs equal to $28n - 88$ and $28n - 92$ for even and odd numbers of qubits, respectively. In this section, we present a decomposition scheme and its variations for different types of $SU(2)$ operators.

3.1 Multi-controlled SU(2) gates with real-valued diagonal

Here, we modify the decomposition scheme of [8] for multi-controlled σ_x with one auxiliary qubit to multi-controlled $SU(2)$ gates with at least one real-valued diagonal. The quantum circuit shown in Fig. 7 is based on that scheme and provides the basic structure for our result. Unlike in [8], there is no auxiliary qubit, and all actions occur on the target qubit.

As in [8], the circuit requires every consecutive single-qubit gate pair to be inverse of each other to satisfy Equations (3) and (4), with the difference that the gate represented by A in Fig. 7 is not restricted to just one particular gate. If all control qubits are 1, then the action on the target is

$$A^\dagger \sigma_x A \sigma_x A^\dagger \sigma_x A \sigma_x = U. \tag{6}$$

For simplicity, we assume the A gates in the circuit are $SU(2)$ gates, so they have the form

$$A = \begin{pmatrix} \alpha & -\beta^* \\ \beta & \alpha^* \end{pmatrix}. \tag{7}$$

With that said, the quantum circuit still follows the same design as in [8], but with some gate $A \in SU(2)$ replacing the phase gate S and without an auxiliary qubit.

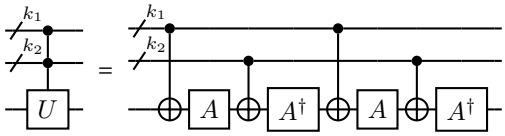


Fig. 7: Decomposition scheme based on [8] for multi-controlled $SU(2)$ gates, where $U, A \in SU(2)$, and U has at least one real-valued diagonal.

The first step of the decomposition of an n-qubit multi-controlled U with k controls consists in dividing the control register roughly in half by choosing $k_1 = \lceil k/2 \rceil$ and $k_2 = \lfloor k/2 \rfloor$ as the size of each new register. Since they both have available qubits from the other register to use as dirty auxiliary qubits, the k_1-controlled and k_2-controlled σ_x gates are implemented following [11, Lemma 8], which improves the gate count of [10, Lemma 7.2].

Lemma 1. *An operator given by the circuit in Fig. 7 whose action on the target qubit is given by Equation (6) and where A is of the form of Equation (7) generates an n-controlled $SU(2)$ gate with the restriction that its off-diagonal is real-valued.*

Proof. When all the control qubits are active, the circuit generates the matrix $A^\dagger \sigma_x A \sigma_x A^\dagger \sigma_x A \sigma_x = (A^\dagger \sigma_x A \sigma_x)^2$. It can be shown that

$$A^\dagger \sigma_x A \sigma_x = \begin{pmatrix} \omega_1^* & \omega_2 \\ -\omega_2 & \omega_1 \end{pmatrix}, \tag{8}$$

where $\omega_1 = \alpha^2 - \beta^2$ and $\omega_2 = 2\,\mathrm{Re}(\alpha^*\beta)$, with ω_1 being a complex number and ω_2 being a real number. Since the determinant of the product $A^\dagger \sigma_x A \sigma_x$ equals 1, we can see that this is an $SU(2)$ matrix with real elements in its off-diagonal. Then

$$(A^\dagger \sigma_x A \sigma_x)^2 = \begin{pmatrix} (\omega_1^*)^2 - \omega_2^2 & 2\,\mathrm{Re}(\omega_1)\omega_2 \\ -2\,\mathrm{Re}(\omega_1)\omega_2 & \omega_1^2 - \omega_2^2 \end{pmatrix} \tag{9}$$

$$= \begin{pmatrix} z^* & x \\ -x & z \end{pmatrix} = U, \tag{10}$$

with z being a complex number and x being a real number. This shows that this modification of the decomposition scheme of [8] gives us $SU(2)$ matrices with real elements in the off-diagonal. □

Theorem 1. *Every n-controlled $SU(2)$ gate whose matrix has real elements in its off-diagonal can be generated by the circuit of the operator described in Lemma 1.*

Proof. Using Lemma 1:

$$\begin{pmatrix} z^* & x \\ -x & z \end{pmatrix} = \begin{pmatrix} (\omega_1^*)^2 - \omega_2^2 & 2\,\mathrm{Re}(\omega_1)\omega_2 \\ -2\,\mathrm{Re}(\omega_1)\omega_2 & \omega_1^2 - \omega_2^2 \end{pmatrix} \tag{11}$$

Noticing that the matrices on both sides have the determinant equal to 1, we can choose the following positive solutions:

$$\omega_1 = \sqrt{\frac{\mathrm{Re}(z)+1}{2}} + i\frac{\mathrm{Im}(z)}{\sqrt{2(\mathrm{Re}(z)+1)}}$$
$$\omega_2 = \frac{x}{\sqrt{2(\mathrm{Re}(z)+1)}}$$

And, as we know from Lemma 1:

$$\omega_1 = \alpha^2 - \beta^2$$
$$\omega_2 = 2\,\mathrm{Re}(\alpha^*\beta)$$

We can make a choice for β to be a real number. And since $\|\alpha\|^2 + \|\beta\|^2 = 1$, we can find the solutions:

$$\alpha = \sqrt{\frac{\sqrt{\frac{\mathrm{Re}(z)+1}{2}}+1}{2}} + i\frac{\mathrm{Im}(z)}{2\sqrt{(\mathrm{Re}(z)+1)\left(\sqrt{\frac{\mathrm{Re}(z)+1}{2}}+1\right)}} \tag{12}$$

$$\beta = \frac{x}{2\sqrt{(\mathrm{Re}(z)+1)\left(\sqrt{\frac{\mathrm{Re}(z)+1}{2}}+1\right)}} \tag{13}$$

So, we have proved that every $SU(2)$ matrix in the form of Equation (11) can be generated by the proposed modified circuit. □

A more detailed description of the steps taken in Theorem 1 can be found in Appendix A.

With a small modification, the proposed circuit of Fig. 7 can also generate n-controlled $SU(2)$ gates with real-valued elements in their main diagonal, while the off-diagonal could be complex.

Lemma 2. *A modification of Lemma 1 can be made such that when all the control qubits are active, it generates an n-controlled $SU(2)$ gate with the restriction that the main diagonal contains real elements.*

Proof. By modifying the target qubit basis with a pair of Hadamard gates, the proposed circuit generates an $SU(2)$ matrix with a real main diagonal, as shown in Equation (14).

$$H \begin{pmatrix} z^* & x \\ -x & z \end{pmatrix} H = \begin{pmatrix} \mathrm{Re}(z) & -x - i\,\mathrm{Im}(z) \\ x - i\,\mathrm{Im}(z) & \mathrm{Re}(z) \end{pmatrix} = \begin{pmatrix} x' & -z' \\ z'^* & x' \end{pmatrix} \tag{14}$$

Since H is its own inverse, this procedure preserves the identities of Equations (3) and (4). □

Theorem 2. *Every n-controlled $SU(2)$ gate whose matrix has real elements in its main diagonal can be generated by the circuit of the operator described in Lemma 2.*

Proof. With the change of basis from Lemma 2, Theorem 1 can be adapted such that the real part of z encodes the real-valued main diagonal, and both x and the imaginary part of z encode the complex off-diagonal.

$$\begin{aligned} x &= \mathrm{Re}(z') \\ z &= x' + i\,\mathrm{Im}(z') \end{aligned} \tag{15}$$

The construction of gate A is modified using Equation (15) to change Equations (12) and (13), so that every $SU(2)$ matrix in the form of Equation (14) can be generated. □

The proposed circuit can also generate n-controlled $SO(2)$ gates, which leads to Corollary 1.

Corollary 1. *Every $SO(2)$ matrix can be generated by Theorem 1 and Theorem 2.*

3.1.1 Complexity

As noted before, in between the single-qubit gates, the circuit implements the decomposition for the multi-controlled σ_x gate with multiple auxiliary qubits present in [10, Lemma 7.2] and includes the optimizations described in [10, Corollary 7.4] and [11, Lemma 8]. Then, for each multi-controlled σ_x with at least five qubits, that is, with the number of controls $k \geq 3$ and at least $k-2$ auxiliary qubits, at most $8k-6$ CNOTs are needed. This means that after the subdivision into two control registers of sizes k_1 and k_2, the maximum total number of CNOTs is $2(8k_1 - 6) + 2(8k_2 - 6) = 16(k_1 + k_2) - 24$. Given that $k_1 + k_2 = n - 1$, where n is the number of qubits, we can state the following theorem.

Theorem 3. *The quantum circuit shown in Fig. 7 can be implemented as an n-qubit circuit, where $n \geq 3$, with at most $16n - 40$ CNOTs.*

3.1.2 Application to multi-controlled R_x, R_y and R_z gates

From what has been shown, the operators R_x, R_y and R_z can be decomposed using the proposed decomposition scheme, which Corollary 2 formally states.

Corollary 2. *The multi-controlled versions of the rotation operator gates R_y and R_z can be generated by the circuit of the operator described in Lemma 1, whereas the multi-controlled version of R_x can be generated by the circuit of the operator described in Lemma 2.*

Proof. The proof follows from the application of the procedure elaborated in Theorem 1 to generate R_y and R_z and Lemma 2 to generate R_x. Alternatively, we can demonstrate that R_x, R_y and R_z can be generated by replacing the gate A of Equation (7) with different gates, as follows:

$$\begin{aligned} R_x(\theta) &= H(R_z(-\theta/4)\sigma_x R_z(\theta/4)\sigma_x)^2 H, \\ R_y(\theta) &= (R_y(\theta/4)\sigma_x R_y(-\theta/4)\sigma_x)^2, \\ R_z(\theta) &= (R_z(\theta/4)\sigma_x R_z(-\theta/4)\sigma_x)^2 \end{aligned} \tag{16}$$

□

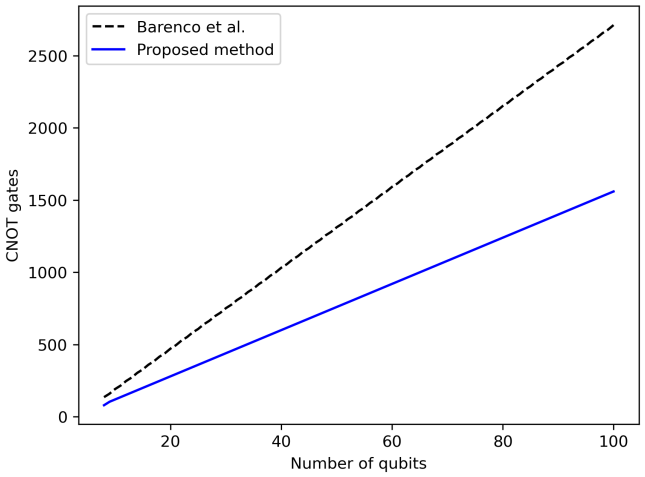


Fig. 8: Comparison of the number of CNOTs of the multi-controlled R_z gate using the decomposition scheme of [10] and [11] for special unitary gates (dashed black line) and the proposed method (solid blue line).

In Fig. (8), the impact of the proposed decomposition scheme on the number of CNOTs in multi-controlled R_z is shown in contrast with [10, Lemma 7.9] with the optimizations of [11, Theorem 5] as described in Section 2.1. The results were obtained from Qiskit's [32] transpilation routine with no additional optimizations and assume complete qubit connectivity. The basis gate set specified consisted of single qubit and CNOT gates.

3.2 Multi-controlled SU(2) gates

We can also use the eigendecomposition of U and the results of the previous sections to construct a multi-controlled version for any gate $U \in SU(2)$ with $20n - 38$ ($20n - 42$, n even) CNOTs.

Theorem 4. *Using Theorem 2 and eigendecomposition, it is possible to construct a circuit that generates any n-controlled $SU(2)$ gate.*

Proof. Given $U \in SU(2)$, it has an eigendecomposition QDQ^{-1}, in which D is a diagonal matrix and Q is formed from the eigenvectors of U. We can choose a suitable phase (if $\vec{v}$ is an eigenvector, then so is $e^{i\theta}\vec{v}$) so that the matrix Q only has real elements in its main diagonal. Then, an n-controlled version of U can be constructed using the decomposition QDQ^{-1} by applying an n-controlled version of each corresponding gate sequentially. Since D is a diagonal matrix, its off-diagonal elements are zeros, so it is possible to use the results from Theorem 1. Meanwhile, Q and Q^{-1}

have real elements in their main diagonal; consequently, we can use the results of Theorem 2. Therefore, it is possible to construct the n-controlled $SU(2)$ gate. □

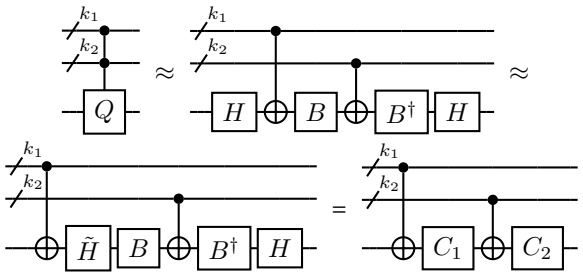


Fig. 9: Decomposition of the multi-controlled gate represented by the eigenvector matrix Q. The Hadamard gates can be combined with the adjacent B gates, reducing the total number of operators and exposing the k_1-controlled σ_x.

In particular, we can do further optimizations. The usual decomposition needs four operators to guarantee the cancellations that lead to the identity given any configuration of the control qubits, except all active (see Equation (3) and Equation (4)). But the first and last decomposition blocks of the circuit proposed here are inverses. Thus, we take advantage of this symmetry to guarantee cancellation. Therefore, for Q and Q^{-1} we only need half of the circuit depicted in Fig. 7, and the circuit for Q is shown in Fig. 9, where

$$
\begin{aligned}
B &= \begin{pmatrix} \alpha' & -\beta'^* \\ \beta' & \alpha'^* \end{pmatrix} \\
\alpha' &= \sqrt{\frac{\mathrm{Re}(z)+1}{2}} + i\frac{\mathrm{Im}(z)}{\sqrt{2(\mathrm{Re}(z)+1)}} \\
\beta' &= \frac{x}{\sqrt{2(\mathrm{Re}(z)+1)}}.
\end{aligned}
$$

The circuit for Q^{-1} is the same, but inverted. As for the diagonal gate D, we can use the circuit shown in Fig. 7 since its off-diagonal is real-valued. The three circuits just described are concatenated to form the final circuit, as illustrated in Fig. 10.

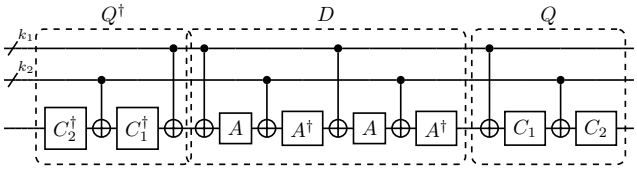


Fig. 10: Decomposition of a multi-controlled $SU(2)$ gate. Each block of the circuit represents one term of the operator's eigendecomposition.

For the Q operator decomposition, we can swap the position of the first Hadamard with the first k_1-controlled σ_x, as shown in Fig. 9, such that $\sigma_x H \equiv \tilde{H}\sigma_x$ for

$$
\tilde{H} = \frac{1}{\sqrt{2}}\begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix}. \tag{17}
$$

This procedure allows the cancellation of both the final k_1-controlled σ_x gate from the Q^{-1} circuit; and the first k_1-controlled σ_x gate from the D circuit; and the cancellation of the auxiliary qubit reversing parts (see Fig. 3) of the k_2-controlled σ_x gate from the Q^{-1} circuit; and the first k_2-controlled σ_x gate from the D circuit. It also allows the combination of the adjacent gates $B\tilde{H} = C_1$ and $HB^\dagger = C_2$. This last optimization also applies to Theorem 2. That way, the total number of operations for both Theorem 1 and Theorem 2 is the same.

3.2.1 Complexity

The multi-controlled σ_x gates that interact with the top k_1 control qubits each need $8k_1 - 6$ CNOTs [11, Lemma 8]. Since two of these gates are canceled, one in the Q^{-1} circuit and another in the D circuit, the contribution of the k_1-controlled σ_x gates is of $16k_1 - 12$ CNOTs. Due to gate canceling on the first two σ_x operators controlled by the bottom k_2 controls (one from the Q^{-1} circuit and the other from the D circuit), these gates only perform the target flipping part of their circuits (see Fig. 3). The target flipping part of one of these gates needs twelve CNOTs to apply two Toffoli gates and up to three CNOTs per application of each approximate Toffoli gate. There are $k_2 - 3$ pairs of approximate Toffoli gates where gate canceling occurs, so each pair contributes with four CNOTs. One approximate Toffoli remains, resulting in a total number of CNOTs equal to $12 + (k_2 - 3)4 + 3 = 4k_2 + 3$ for the two reduced k_2-controlled σ_x gates. Finally, there is no additional canceling of gates in the remaining two k_2-controlled σ_x operators. Therefore, the total cost of the circuit is at most

$$
\begin{aligned}
N_{\text{CNOT}} &= (16k_1 - 12) + 2(4k_2 + 3) + (16k_2 - 12) \\
&= 16(k_1 + k_2) + 8k_2 - 18 \\
&= 16(n-1) + 8\left\lfloor \frac{n-1}{2} \right\rfloor - 18 \\
&= 16n + 8\left\lfloor \frac{n-1}{2} \right\rfloor - 34
\end{aligned} \tag{18}
$$

With that, and given $\lfloor \frac{n-1}{2} \rfloor = \frac{n-1}{2}$ for odd n and $\lfloor \frac{n-1}{2} \rfloor = \frac{n}{2} - 1$ for even n, we have the following final CNOT count

$$
\begin{cases} 20n - 38, n \text{ odd} \\ 20n - 42, n \text{ even} \end{cases} \tag{19}
$$

The results obtained in Equation (19) can now be formalized in the following theorem.

Theorem 5. *The quantum circuit shown in Fig. 10 can be implemented as an n-qubit circuit, where $n \geq 3$, with at most $20n - 38$ CNOTs if n is odd or $20n - 42$ CNOTs if n is even.*

4 Experiments

As a use case, we apply the new multi-controlled gate to reduce the cost of circuits produced by the CVO-QRAM sparse state preparation algorithm [33].

The algorithm takes advantage of data storing in quantum random access memory to represent sparse data in the number of patterns stored. Additionally, the computational cost depends on the number of 1s in stored patterns, as opposed to the number of qubits. The circuits produced by the CVO-QRAM technique have an auxiliary qubit beside

the memory qubits and begin by initializing the complete register as $|u\rangle |m\rangle^{\otimes n-1} = |1\rangle |0\rangle^{\otimes n-1}$. For each input vector pattern p_k, the multi-controlled $U^{(x_k,\gamma_k)}$ gate, which is defined as

$$U^{(x_k,\gamma_k)} = \begin{pmatrix} \sqrt{\frac{\gamma_k - |x_k|^2}{\gamma_k}} & \frac{x_k}{\sqrt{\gamma_k}} \\ \frac{-x_k^*}{\sqrt{\gamma_k}} & \sqrt{\frac{\gamma_k - |x_k|^2}{\gamma_k}} \end{pmatrix}, \tag{20}$$

encodes the corresponding value x_k as a state amplitude (where $\gamma_k = \gamma_{k-1} - |x_{k-1}|^2$ and $\gamma_0 = 1$), plus two CNOT gates are applied before and after the multi-controlled gate, as depicted in Fig. 11.

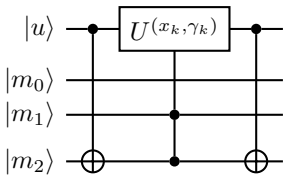


Fig. 11: Loading $x_k|011\rangle$ with CVO-QRAM.

Realizing that the multi-controlled operation is the main cause for the computational cost of the circuit and that the $U^{(x_k,\gamma_k)}$ operator belongs to the $SU(2)$ group with a real-valued main diagonal, the decomposition is readily replaced by the new linear version, reducing the number of CNOTs from $28n - 88$ ($28n - 92$ for odd n) [10], [11] to $16n - 40$. The advantage of this modification is demonstrated in two experiments.

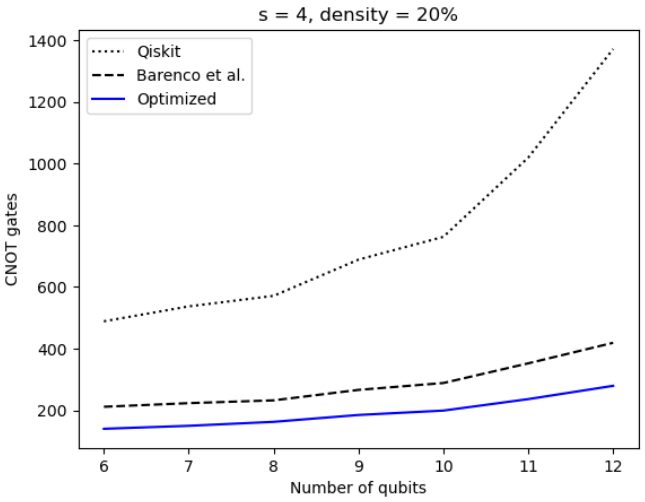
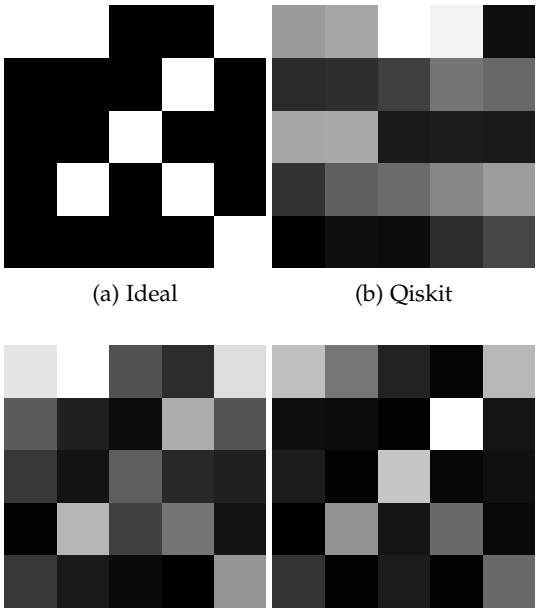


Fig. 12: Average number of CNOT gates produced by CVO-QRAM algorithm for a double sparse random state with n qubits and 2^s nonzero entries. The density is the average number of 1s in the binary strings.

The first experiment, shown in Fig. 12, compares the number of CNOTs on circuits produced by CVO-QRAM (using Qiskit's multi-controlled gate [32] and the method presented by Barenco et al. in [10] with the improvements from [11]) and by the optimized CVO-QRAM for double sparse states with the number of qubits ranging from $n = 6$ to $n = 12$, with 2^4 nonzero amplitudes, and a 20% average density of 1s present in the binary strings. Each point on the graph is an average of 30 different random state results. Fig. 12 shows that circuits produced by CVO-QRAM have significantly more CNOTs than the ones by its optimized version. This experiment does not target any device and does not use Qiskit's circuit optimization.



(a) Ideal (b) Qiskit

(c) Barenco et al. (d) Optimized

Fig. 13: Visual comparison of a 5-qubit double sparse quantum state preparation. (a) Visual representation of the ideal measurement probabilities. (b) State initialized by nonoptimized CVO-QRAM using Qiskit's multi-controlled gate. (c) State initialized by CVO-QRAM using the multi-controlled gate proposed in [10]. (d) State initialized by CVO-QRAM using the multi-controlled gate proposed in this work. These illustrations represent an estimate of the measurement probabilities on the ibm_oslo device.

The second experiment, depicted in Fig. 13 and summarized in Table 1, compares the performance of CVO-QRAM on IBM's ibm_oslo quantum device using a different implementation of the multi-controlled gate. This experiment prepares a 5-qubit double sparse state and estimates the measurement probabilities. The algorithm performance is evaluated by comparing the estimate against ideal values. Table 1 shows the mean absolute error (MAE) between the estimated and the ideal probabilities. The MAE produced with optimized CVO-QRAM is smaller than that of the nonoptimized ones. Fig. 13 is a visual representation of this result. This experiment uses Qiskit's circuit optimization level 3, and each figure is produced from one execution on the target device, with 8192 shots.

All experiments were performed using the *Qclib* library [34].

5 Conclusion

In this paper, we proposed a linear decomposition for n-qubit multi-controlled special unitary single-qubit gates without auxiliary qubits. Our method shows improved gate

	CNOTs	Depth	MAE
Qiskit	149	320	0.04541
Barenco et al.	82	190	0.03137
Optimized	39	113	0.01654

TABLE 1: Number of CNOTs and depth of the circuit produced by the CVO-QRAM algorithms to encode a 5-qubit double sparse state with eight nonzero amplitudes with an average of 10% of 1s in the binary strings using multi-controlled gates based on Qiskit [32], Barenco et al. [10], and the optimized method proposed in this work. The MAE column shows the mean absolute error between the ideal and the actual results.

counts and depth over the best schemes to decompose general multi-controlled $SU(2)$ gates [10], [11] known so far, with $20n - 38$ CNOTs ($20n - 42$ for even n) needed to use our construction compared with $28n - 88$ CNOTs ($28n - 92$ for odd n). We have also presented an additional scheme for $SU(2)$ gates with matrices containing at least one real-valued diagonal, which yields an improved CNOT count of $16n - 40$. [33] is suggested as a possible method in such cases, which, as we have shown, can have a total number of CNOT gates lower than originally estimated.

Some future considerations are the theoretical bounds when constructing the types of gates described in this paper and whether the proposed decomposition scheme achieves or approaches optimality. It is also worth noting that we have not developed similar methods with the inclusion of auxiliary qubits; thus, any potential enhancements that result from doing so are yet to be explored. We would also like to investigate the generalization of a decomposition method for any multi-controlled $U(2)$ gate that aims to maintain a lower CNOT cost and depth. In particular, a significant challenge to circumvent is the introduction of numerical error in the computation of gates generated in the decomposition of $U(2)$ gates by some known methods [10], [12]. As a result, for a decomposition scheme for these types of gates to be viable in practice, simply the reduction in depth and number of gates as a goal is not sufficient, pointing to the importance of different solutions to bypass this issue.

ACKNOWLEDGMENTS

This work is based upon research supported by CNPq (Grant No. 409506/2022-2, No. 409513/2022-9 and No. 162052/2021-9), CAPES – Finance Code 001, CAPES (Grant No. 25001019004P6), FACEPE (Grant No. APQ-1229-1.03/21), National Research Foundation of Korea (Grant No. 2022M3E4A1074591). We acknowledge the use of IBM Quantum services for this work. The views expressed are those of the authors, and do not reflect the official policy or position of IBM or the IBM Quantum team.

DATA AVAILABILITY

The sites https://github.com/qclib/qclib-papers and https://github.com/qclib/qclib contain all the data and the software generated during the current study.

APPENDIX A
THEOREM 1 DETAILS

In this section, we detail the mathematical steps taken to prove Theorem 1. First, we start with the equations for z

$$z = \omega_1^2 - \omega_2^2 \tag{21}$$

and x

$$x = 2\,\mathrm{Re}(\omega_1)\omega_2, \tag{22}$$

as well as the requirement of unitarity for both V from Equation (11) and the matrix defined in Equation (8):

$$\begin{cases} |z|^2 + x^2 = 1 \\ |\omega_1|^2 + \omega_2^2 = 1 \end{cases} \tag{23}$$

Expanding Equation (21), we obtain

$$\mathrm{Re}(z) + i\,\mathrm{Im}(z) = \mathrm{Re}(\omega_1)^2 - \mathrm{Im}(\omega_1)^2 - \omega_2^2 + 2i\,\mathrm{Re}(\omega_1)\,\mathrm{Im}(\omega_1). \tag{24}$$

Looking at the real elements and using the unitarity from Equation (23) we have

$$\mathrm{Re}(z) = 2\,\mathrm{Re}(\omega_1)^2 - 1, \tag{25}$$

which leads to

$$\mathrm{Re}(\omega_1) = \pm\sqrt{\frac{\mathrm{Re}(z)+1}{2}} \rightarrow \sqrt{\frac{\mathrm{Re}(z)+1}{2}}, \tag{26}$$

in which we have chosen the positive solution. Replacing Equation (26) into Equation (21) and Equation (22):

$$\mathrm{Im}(\omega_1) = \frac{\mathrm{Im}(z)}{\sqrt{2(\mathrm{Re}(z)+1)}} \tag{27}$$

$$\omega_2 = \frac{x}{\sqrt{2(\mathrm{Re}(z)+1)}} \tag{28}$$

Now we proceed to determine α and β. First, we can write $\alpha = a + bi$, $\beta = c + di$. So,

$$\omega_1 = (a^2 - b^2 - c^2 + d^2) + 2(ab + cd)i \tag{29}$$

$$\omega_2 = 2(ac + bd) \tag{30}$$

Now, ω_1 and ω_2 have three free variables, which can be reduced to two free variables due to unitarity. Meanwhile, α and β have three free variables after accounting for unitarity. With the extra free variable, we make a choice for $d = 0$, making β strictly real, which gives us

$$c = \frac{\omega_2}{2a} \tag{31}$$

and

$$b = \frac{\mathrm{Im}(\omega_1)}{2a}. \tag{32}$$

Plugging in these results in Equation (29) gives us a degree 4 equation:

$$a^4 - \mathrm{Re}(\omega_1)a^2 + \frac{\mathrm{Re}(\omega_1)^2 - 1}{4} = 1 \tag{33}$$

We choose a real and positive solution for a:

$$a = \sqrt{\frac{\mathrm{Re}(\omega_1)+}{2}} = \sqrt{\frac{\sqrt{\frac{\mathrm{Re}(z)+1}{2}}+1}{2}} \tag{34}$$

One can verify that $|a|^2+|b|^2+|c|^2 = 1$. Replacing a into Equation (31) and Equation (32) gives us

$$b = \frac{\mathrm{Im}(z)}{2\sqrt{(\mathrm{Re}(z)+1)\left(\sqrt{\frac{\mathrm{Re}(z)+1}{2}}+1\right)}} \tag{35}$$

and

$$c = \frac{x}{2\sqrt{(\mathrm{Re}(z)+1)\left(\sqrt{\frac{\mathrm{Re}(z)+1}{2}}+1\right)}}, \tag{36}$$

which is the result from Theorem 1.

REFERENCES

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," SIAM review, vol. 41, no. 2, pp. 303–332, 1999.

[2] L. K. Grover, "Quantum mechanics helps in searching for a needle in a haystack," Physical review letters, vol. 79, no. 2, p. 325, 1997.

[3] J. Preskill, "Quantum computing in the NISQ era and beyond," Quantum, vol. 2, p. 79, 2018.

[4] R. Takagi, S. Endo, S. Minagawa, and M. Gu, "Fundamental limits of quantum error mitigation," npj Quantum Information, vol. 8, no. 1, p. 114, 2022.

[5] S. Boixo, S. V. Isakov, V. N. Smelyanskiy, R. Babbush, N. Ding, Z. Jiang, M. J. Bremner, J. M. Martinis, and H. Neven, "Characterizing quantum supremacy in near-term devices," Nature Physics, vol. 14, no. 6, pp. 595–600, 2018.

[6] T. G. De Brugiere, M. Baboulin, B. Valiron, S. Martiel, and C. Allouche, "Reducing the depth of linear reversible quantum circuits," IEEE Transactions on Quantum Engineering, vol. 2, pp. 1–22, 2021.

[7] T. Nguyen and A. McCaskey, "Enabling pulse-level programming, compilation, and execution in XACC," IEEE Transactions on Computers, vol. 71, no. 3, pp. 547–558, 2021.

[8] Y. He, M.-X. Luo, E. Zhang, H.-K. Wang, and X.-F. Wang, "Decompositions of n-qubit Toffoli gates with linear circuit complexity," International Journal of Theoretical Physics, vol. 56, no. 7, pp. 2350–2361, 2017.

[9] I. F. Araujo, C. Blank, I. Cesar, and A. J. da Silva, "Approximated quantum-state preparation with entanglement dependent complexity," arXiv preprint arXiv:2111.03132, 2021.

[10] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, "Elementary gates for quantum computation," Physical Review A, vol. 52, pp. 3457–3467, 1995.

[11] R. Iten, R. Colbeck, I. Kukuljan, J. Home, and M. Christandl, "Quantum circuits for isometries," Physical Review A, vol. 93, no. 3, p. 032318, 2016.

[12] A. J. da Silva and D. K. Park, "Linear-depth quantum circuits for multiqubit controlled gates," Physical Review A, vol. 106, p. 042602, 2022.

[13] M. Saeedi and M. Pedram, "Linear-depth quantum circuits for n-qubit toffoli gates with no ancilla," Physical Review A, vol. 87, no. 6, p. 062318, 2013.

[14] P. Gokhale, J. M. Baker, C. Duckering, N. C. Brown, K. R. Brown, and F. T. Chong, "Asymptotic improvements to quantum circuits via qutrits," in Proceedings of the 46th International Symposium on Computer Architecture, 2019, pp. 554–566.

[15] L. Biswal, D. Bhattacharjee, A. Chattopadhyay, and H. Rahaman, "Techniques for fault-tolerant decomposition of a multicontrolled toffoli gate," Physical Review A, vol. 100, no. 6, p. 062326, 2019.

[16] S. Balauca and A. Arusoaie, "Efficient constructions for simulating multi controlled quantum gates," in International Conference on Computational Science. Springer, 2022, pp. 179–194.

[17] T. Tomesh, N. Allen, and Z. Saleem, "Quantum-classical tradeoffs and multi-controlled quantum gate decompositions in variational algorithms," arXiv preprint arXiv:2210.04378, 2022.

[18] T. Kim and B.-S. Choi, "Efficient decomposition methods for controlled-Rn using a single ancillary qubit," Scientific reports, vol. 8, no. 1, pp. 1–7, 2018.

[19] S. Lloyd, M. Mohseni, and P. Rebentrost, "Quantum principal component analysis," Nature Physics, vol. 10, no. 9, pp. 631–633, 2014.

[20] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," Physical review letters, vol. 103, no. 15, p. 150502, 2009.

[21] J. Li, F. Gao, S. Lin, M. Guo, Y. Li, H. Liu, S. Qin, and Q. Wen, "Quantum k-fold cross-validation for nearest neighbor classification algorithm," Physica A: Statistical Mechanics and its Applications, vol. 611, p. 128435, 2023.

[22] M. Guo, H. Liu, Y. Li, W. Li, F. Gao, S. Qin, and Q. Wen, "Quantum algorithms for anomaly detection using amplitude estimation," Physica A: Statistical Mechanics and its Applications, vol. 604, p. 127936, 2022.

[23] M. Schuld, I. Sinayskiy, and F. Petruccione, "Prediction by linear regression on a quantum computer," Physical Review A, vol. 94, no. 2, p. 022342, 2016.

[24] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," Nature, vol. 549, no. 7671, pp. 195–202, 2017.

[25] D. K. Park, F. Petruccione, and J.-K. K. Rhee, "Circuit-based quantum random access memory for classical data," Scientific reports, vol. 9, no. 1, p. 3949, 2019.

[26] N. Gleinig and T. Hoefler, "An efficient algorithm for sparse quantum state preparation," in 2021 58th ACM/IEEE Design Automation Conference (DAC). IEEE, 2021, pp. 433–438.

[27] G. H. Low, T. J. Yoder, and I. L. Chuang, "Quantum inference on bayesian networks," Physical Review A, vol. 89, no. 6, p. 062315, 2014.

[28] R. Orús, S. Mugel, and E. Lizaso, "Quantum computing for finance: Overview and prospects," Reviews in Physics, vol. 4, p. 100028, 2019.

[29] F. Mozafari, H. Riener, M. Soeken, and G. De Micheli, "Efficient boolean methods for preparing uniform quantum states," IEEE Transactions on Quantum Engineering, vol. 2, pp. 1–12, 2021.

[30] L. S. de Souza, J. H. de Carvalho, and T. A. Ferreira, "Classical artificial neural network training using quantum walks as a search procedure," IEEE Transactions on Computers, vol. 71, no. 2, pp. 378–389, 2021.

[31] T. M. De Veras, I. C. De Araujo, D. K. Park, and A. J. Da Silva, "Circuit-based quantum random access memory for classical data with continuous amplitudes," IEEE Transactions on Computers, vol. 70, no. 12, pp. 2125–2135, 2020.

[32] G. Aleksandrowicz and et al., "Qiskit: An open-source framework for quantum computing," 2021.

[33] T. M. de Veras, L. D. da Silva, and A. J. da Silva, "Double sparse quantum state preparation," Quantum Information Processing, vol. 21, no. 6, pp. 1–13, 2022.

[34] I. F. Araujo, I. C. S. Araújo, L. D. da Silva, C. Blank, and A. J. da Silva, "Quantum computing library," 7 2022. [Online]. Available: https://github.com/qclib/qclib

Efficient Z-Gates for Quantum Computing

David C. McKay,* Christopher J. Wood, Sarah Sheldon, Jerry M. Chow, and Jay M. Gambetta
IBM T.J. Watson Research Center, Yorktown Heights, NY 10598, USA
(Dated: June 29, 2017)

For superconducting qubits, microwave pulses drive rotations around the Bloch sphere. The phase of these drives can be used to generate zero-duration arbitrary "virtual" Z-gates which, combined with two $X_{\pi/2}$ gates, can generate any SU(2) gate. Here we show how to best utilize these virtual Z-gates to both improve algorithms and correct pulse errors. We perform randomized benchmarking using a Clifford set of Hadamard and Z-gates and show that the error per Clifford is reduced versus a set consisting of standard finite-duration X and Y gates. Z-gates can correct unitary rotation errors for weakly anharmonic qubits as an alternative to pulse shaping techniques such as DRAG. We investigate leakage and show that a combination of DRAG pulse shaping to minimize leakage and Z-gates to correct rotation errors (DRAGZ) realizes a 13.3 ns $X_{\pi/2}$ gate characterized by low error ($1.95[3] \times 10^{-4}$) and low leakage ($3.1[6] \times 10^{-6}$). Ultimately leakage is limited by the finite temperature of the qubit, but this limit is two orders-of-magnitude smaller than pulse errors due to decoherence.

Computers based on quantum bits (qubits) are predicted to outperform classical computers for certain critical problems, e.g., factoring. Unlike a classical bit, which is discretely in the state 0 or 1, a qubit can be in a superposition state $|\Psi\rangle = \cos(\theta/2)|0\rangle + e^{i\phi}\sin(\theta/2)|1\rangle$ where $|0\rangle$ and $|1\rangle$ are the quantum versions of the classical 0 and 1 states. This single-qubit superposition state can be geometrically represented as a point on the surface of a unit-sphere known as the *Bloch sphere*. Critical to implementing a quantum computer is the ability to control the state of the qubit, i.e., transform the qubit state arbitrarily between two points on the Bloch sphere. This is accomplished by unitary transformations (gates), which correspond to rotations of the state around different axes in the Bloch sphere representation. Physically, X and Y gates (rotations around the X and Y axes) are generated by modulating the coupling between the states $|0\rangle$ and $|1\rangle$ at the frequency difference between these states $\omega_{01} = (E_{|1\rangle} - E_{|0\rangle})/\hbar$. This modulation drive has the general form $\Omega(t)\cos(\omega_D t - \gamma)$ where $\Omega(t)$ is the drive strength of the rotation, ω_D is the drive frequency ($\omega_D = \omega_{01}$ on resonance) and γ is the drive phase. The duration of the gate is set by the desired rotation angle and the drive strength. On-resonance, when $\gamma = 0$, the qubit state rotates around the X axis and when $\gamma = \frac{\pi}{2}$ the rotation is around the Y axis. Therefore, the geometric X and Y axes in the Bloch sphere correspond to a real $\frac{\pi}{2}$ phase difference between drive fields.

Rotations around the remaining axis (Z axis), i.e., Z-gates, correspond to a change in the relative phase between the $|0\rangle$ and $|1\rangle$ states. A Z-gate can be implemented by either detuning the frequency of the qubit with respect to the drive field for some finite amount of time (e.g. see Ref. [1]) or by composite X and Y gates. The result is that the qubit state rotates with respect to the X and Y axes. However, it is equivalent to rotate the axes with respect to the qubit state – such a gate is known as a virtual Z-gate which corresponds to adding a phase offset to the drive field for all subsequent X and Y gates. This includes adding a phase offset to any two-qubit drives such as a drive used to implement a CNOT gate via cross-resonance [2]. In many qubit implementations this phase, γ, is defined by classical control hardware and software. A Z-gate implemented in this way is essentially perfect; the classical hardware is self-calibrated via a global frequency reference (e.g., an atomic clock) and the gate has zero duration. Virtual-Z (VZ) gates have long been used in quantum experiments such as in NMR [3], ions [4], and superconducting qubits [5]. Utilizing these gates can improve the overall fidelity of a quantum circuit if the circuit is optimized to maximize the number of single-qubit Z-gates. Additionally, any arbitrary rotation in the Bloch sphere can be generated by combining Z and $X_{\pi/2}$ gates. This greatly simplifies calibration procedures because only a single drive strength must be calibrated.

Furthermore, Z-gates can compensate for certain unitary errors which occur in physical qubit implementations. For example, when driving X and Y rotations in weakly anharmonic superconducting transmon qubits [6] there are unitary rotation errors (Stark shifts errors) and population leakage. Both of these errors can be corrected by implementing a full DRAG pulse [7], which involves pulse shaping and dynamic frequency tuning. However, only the pulse shaping component of DRAG is typically implemented [1, 8, 9], which cannot simultaneously correct both errors (we herein refer to DRAG by this definition). In most experiments DRAG is optimized to correct the more dominant unitary errors. This problem is solved by adding VZ-gates since VZ-gates can correct unitary phase errors while pulse shaping is then optimized to minimize leakage. Similar errors are common when driving two-qubit gates, such as the parametric iSWAP [10], cross-resonance [11] and adiabatic CZ [12], so VZ-gates plus pulse shaping is also applicable in multiqubit systems.

In this paper we explore how the VZ-gate can be used

* dcmckay@us.ibm.com

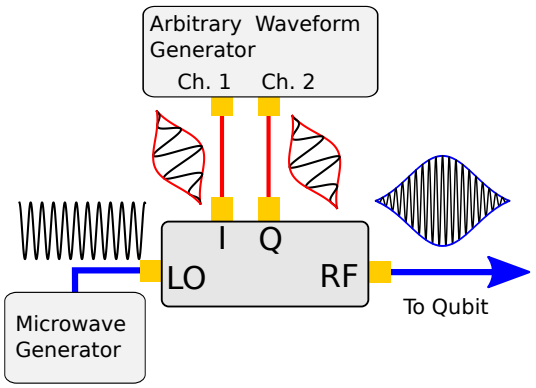


FIG. 1. (Color Online) Schematic of the typical experimental setup for generating shaped microwave pulses for driving superconducting qubits.

to minimize circuit error and, for superconducting transmon qubits, minimize pulse errors. First, we review the theory of the VZ-gate and show one specific formula for an arbitrary SU(2) gate. Next, we compare randomized benchmarking [13] of a qubit using Cliffords generated from two different sets of basis gates – one set using X and Y rotations and the other set using X and Z rotations. We show that the error per Clifford is lower for the XZ basis set, since the the number of finite-duration gates (i.e., X and Y rotations) required to represent each Clifford gate is reduced. We also perform interleaved benchmarking of the S gate ($Z_{\pi/2}$) and measure an error rate that is consistent with a perfect gate. Next, we demonstrate that VZ-gates can be used to compensate Stark-shift errors that arise when driving most X and Y rotations in weakly anharmonic systems. This technique, Gaussian plus Z (GZ), is a straightfoward alternative to the commonly used DRAG pulse [1, 7–9]. We show via randomized benchmarking that the pulse error for DRAG and GZ is essentially equivalent. Next, we measure population leakage to the $|2\rangle$ state during these RB sequences (see Ref. [14] for similar work). We show that GZ and DRAG (optimized for fidelity) have similar leakage rates. Combining DRAG and VZ-gates (DRAGZ) improves leakage without a loss of fidelity as does passive filtering plus VZ-gates (FILTZ). For pulses longer than 25 ns we find that the leakage rates for all methods are similar and limited by heating. Finally, we discuss some considerations when using these gates in a multi-qubit system.

I. THEORY

To elucidate the concept of the virtual Z-gate (VZ-gate), we first review basic single qubit gates as they are physically realized in many labs, i.e., with a local oscillator (LO) shaped by an arbitrary waveform generator (AWG) through an IQ mixer (setup shown in Fig. 1). The LO is a single tone microwave source that outputs a constant signal $\cos(\omega_{LO}t)$. The AWG outputs a programmable series of discrete voltage points at a specific sample rate, e.g., for the Tektronix 5014 used in these experiments that sample rate is 1.2GSa/s (0.833ns between points). These points are internally filtered so that the output is a smooth waveform. Including this filter (F), the AWG output is

$$V(t) = \int_0^t d\tau F(t-\tau) \sum_{n=0}^{\infty} V_n \sqcap (n - \tau/T) \tag{1}$$

where $\{V_n\}$ is the set of AWG voltages, T is the AWG period and $\sqcap(x)$ is a pulse function defined as $\sqcap(x) = \{1, 0\}$ conditioned on $|x| \leq 1/2$. The IQ mixer multiplies the I and Q channels with the LO such that,

$$\begin{aligned} V_{RF}(t) = & V_I(t)\cos(\omega_{LO}t) \\ & + V_Q(t)[1+\epsilon_Q]\sin(\omega_{LO}t + \epsilon_\phi) \\ & + \epsilon_{LO}\cos(\omega_{LO}t) \end{aligned} \tag{2}$$

where the ϵ terms are non-ideal errors common to IQ mixers.

The AWG voltages are used to shape the pulse and shift the pulse frequency to resonance via single sideband (SSB) modulation. If the desired pulse envelope is $\Omega(t)$ and the drive frequency is $\omega_D = \omega_{LO} + \omega_{SSB}$ then the output of an ideal AWG (ignoring the pixelation and filtering described by Eqn. 1) is,

$$V_I(t) = \Omega(t)\cos(\omega_{SSB}t - \gamma), \tag{3}$$

$$V_Q(t) = -\Omega(t)\sin(\omega_{SSB}t - \gamma). \tag{4}$$

These AWG signals, when applied to the inputs of an ideal mixer, output the desired drive pulse

$$V_{RF}(t) = \Omega(t)\cos(\omega_D t - \gamma). \tag{5}$$

A series of n shaped microwave pulses driving an anharmonic oscillator (a good description of a transmon qubit) is described by the Hamiltonian (in the lab frame),

$$\begin{aligned} H/\hbar = & \sum_n \Omega_n(t)\cos(\omega_D t + \gamma_n)(\hat{a} + \hat{a}^\dagger) + \omega_{01}\hat{n} \\ & + \frac{\alpha}{2}(\hat{n}-1)\hat{n}, \end{aligned} \tag{6}$$

where $\hat{a}(\hat{a}^\dagger)$ is the annihilation (creation) operator of the oscillator, $\hat{n} = \hat{a}^\dagger\hat{a}$ is the number operator and α is the anharmonicity. The $|0\rangle$ and $|1\rangle$ levels of this oscillator are the qubit levels. Leakage to higher levels and unitary errors due to mixing with these levels will be discussed in § III and IV. For simplicity we rewrite just the qubit Hamiltonian,

$$H/\hbar = \sum_n \Omega_n(t)\cos(\omega_D t - \gamma_n)\hat{\sigma}_X - \frac{\omega_{01}}{2}\hat{\sigma}_Z, \tag{7}$$

where $\hat{\sigma}_X, \hat{\sigma}_Z$ are the Pauli operators. For a resonant drive ($\omega_D = \omega_{01}$), the Hamiltonian in the qubit rotating frame is,

$$\tilde{H}/\hbar = \sum_n \frac{\Omega_n(t)}{2}\left[\cos(\gamma_n)\hat{\sigma}_X + \sin(\gamma_n)\hat{\sigma}_Y\right] \tag{8}$$

Assuming a constant amplitude pulse Ω_n for duration T, the unitary transformation (the gate) due to pulse n is,

$$U_n = e^{-i\frac{\Omega_n T}{2}[\cos(\gamma_n)\hat{\sigma}_X + \sin(\gamma_n)\hat{\sigma}_Y]}, \tag{9}$$

and so a $\gamma_n = 0(\frac{\pi}{2})$ pulse is a rotation of angle $\Omega_n T$ around the X (Y) axis of the Bloch sphere. Therefore, the AWG controls both the pulse rotation and the rotation axes. Since γ controls the rotation axes, intuitively we can perform a VZ-gate by adjusting γ. More formally, we can show how this works by consider two consecutive X pulses, $X_\theta = e^{-i\frac{\theta}{2}\hat{\sigma}_X}$, with the phase γ offset by ϕ between the pulses. The total unitary is,

$$e^{-i\frac{\theta}{2}(\cos(\phi)\hat{\sigma}_X + \sin(\phi)\hat{\sigma}_Y)} \cdot X_\theta, \tag{10}$$

which can be expanded to,

$$e^{i\frac{\phi}{2}\hat{\sigma}_Z} \cdot e^{-i\frac{\theta}{2}\hat{\sigma}_X} \cdot e^{-i\frac{\phi}{2}\hat{\sigma}_Z} \cdot X_\theta, \tag{11}$$

which equals

$$Z_{-\phi} \cdot X_\theta \cdot Z_\phi \cdot X_\theta. \tag{12}$$

Therefore, by simply adding a phase offset in software and redefining the rotation axes for subsequent X and Y gates, we can effectively implement an arbitrary Z-gate. The additional $Z_{-\phi}$ gate at the end is due to the fact that we are in the qubit frame of reference and so the phase offset ϕ must be carried through for all subsequent gates. For example, if we follow our original sequence with a Y_θ gate with the phase offset applied the gate sequence is

$$\begin{aligned} & e^{-i\frac{\theta}{2}\left(\cos\left[\frac{\pi}{2}+\phi\right]\hat{\sigma}_X + \sin\left[\frac{\pi}{2}+\phi\right]\hat{\sigma}_Y\right)} \cdot \ldots \\ & Z_{-\phi} \cdot X_\theta \cdot Z_\phi \cdot X_\theta, && (13) \\ &= Z_{-\phi} \cdot Y_\theta \cdot Z_\phi \cdot Z_{-\phi} \cdot X_\theta \cdot Z_\phi \cdot X_\theta, && (14) \\ &= Z_{-\phi} \cdot Y_\theta \cdot X_\theta \cdot Z_\phi \cdot X_\theta. && (15) \end{aligned}$$

The inverse Z-gate remains, but does not change the measurement outcomes which are measured along Z.

Given that we can easily create Z-gates, we now show that any arbitrary SU(2) gate can be constructed by combining Z-gates with two $X_{\pi/2}$ gates. In general, any SU(2) gate can be written in the form,

$$U(\theta,\phi,\lambda) = \begin{bmatrix} \cos(\theta/2) & -ie^{i\lambda}\sin(\theta/2) \\ -ie^{i\phi}\sin(\theta/2) & e^{i(\lambda+\phi)}\cos(\theta/2) \end{bmatrix} \tag{16}$$

which is conveniently represented (up to a global phase) as,

$$U(\theta,\phi,\lambda) = Z_\phi \cdot X_\theta \cdot Z_\lambda. \tag{17}$$

By using the identity,

$$X_\theta = Z_{-\pi/2} \cdot X_{\pi/2} \cdot Z_{\pi-\theta} \cdot X_{\pi/2} \cdot Z_{-\pi/2}, \tag{18}$$

we show that any SU(2) gate is

$$U(\theta,\phi,\lambda) = Z_{\phi-\pi/2} \cdot X_{\pi/2} \cdot Z_{\pi-\theta} \cdot X_{\pi/2} \cdot Z_{\lambda-\pi/2}. \tag{19}$$

We express some common gates in this notation in Table I. The ability to efficiently create arbitrary SU(2) gates using Z-gates is essential for performing universal quantum algorithms.

TABLE I. Common SU(2) gates expressed with Z gates.

Gate	θ	ϕ	λ
$\mathcal{I}$	0	0	0
X_π	π	0	0
Y_π	π	$\pi/2$	-$\pi/2$
Z_π	0	$\pi/2$	$\pi/2$
$X_{\pi/2}$	$\pi/2$	0	0
$Y_{\pi/2}$	$\pi/2$	$\pi/2$	-$\pi/2$
S	0	$\pi/4$	$\pi/4$
H	$\pi/2$	$\pi/2$	$\pi/2$
$X_{\pi/4}$	$\pi/4$	0	0
T	0	$\pi/8$	$\pi/8$

II. RANDOMIZED BENCHMARKING OF THE Z-GATE

To demonstrate how the VZ-gate can improve algorithms we perform randomized benchmarking [13] (RB) using a fixed-frequency superconducting transmon qubit of frequency $\omega/2\pi = 5.0353$ GHz, anharmonicity $\alpha/2\pi = -235.5$ MHz, and typical coherences $T_1 = 54(1)$ μs, $T_\phi = 135(4)$ μs. This qubit is part of a two-qubit device detailed in Ref. [10]; for the work here we consider it as a single independent qubit (the other qubit frequency is 5.924 GHz). A RB circuit consists of m random Clifford gates with a final inverting gate so that the full circuit implements the identity operator. These Clifford gates are constructed from single qubit gate primitives which, at minimum, are $\frac{\pi}{2}$ pulses along two independent axes. Here we compare two sets of basis pulses — the $XY_{\frac{\pi}{2}}$ and HZ sets. The $XY_{\frac{\pi}{2}}$ set consists of the finite-duration gates $\{X_{\pi/2}, X_{-\pi/2}, Y_{\pi/2}, Y_{-\pi/2}\}$ whereas the HZ set consists of one finite duration gate combined with VZ-gates $\{H, I = Z_0, S = Z_{\frac{\pi}{2}}, S^\dagger = Z_{-\frac{\pi}{2}}, Z_\pi\}$ where $H = Z_{\frac{\pi}{2}} \cdot X_{\frac{\pi}{2}} \cdot Z_{\frac{\pi}{2}}$ is the Hadamard gate. On average, 2.25 gates from the $XY_{\frac{\pi}{2}}$ set and 2.4583 gates from the HZ set are required to construct a Clifford. However, for the HZ set only one of those gates per Clifford is the finite-duration Hadamard gate. Since we expect the VZ-gates to be near perfect, the HZ set should have lower error per Clifford. For this experiment each of these finite duration gates is implemented as a DRAG pulse — a pulse with a Gaussian envelope along the main rotation axis and a pulse with a derivative Gaussian envelope along the orthogonal rotation axis. The Gaussian pulse is defined as,

$$\Omega_G(t) = \begin{cases} \Omega_0 \frac{e^{-t^2/2\sigma^2} - e^{-T^2/2\sigma^2}}{1 - e^{-T^2/2\sigma^2}} & , \ t \le T \\ 0 & , \ \text{else} \end{cases} \tag{20}$$

where T is the pulse length which is set to $T = 4\sigma$. For these experiments $T = 13.33$ ns and $\sigma = 3.33$ ns with a 6.7 ns buffer between pulses ($\omega_{SSB}/2\pi = -120$ MHz).

The total DRAG pulse is then

$$\Omega(t) = \begin{cases} \Omega_G(t) & , \ \gamma = 0 \\ \beta\dot{\Omega}_G(t) & , \ \gamma = \frac{\pi}{2} \end{cases}, \tag{21}$$

where the DRAG parameter β is calibrated to optimize pulse fidelity by cancelling Stark shift errors due to off-resonance driving of higher transmon levels. In theory, the value of β which optimizes fidelity is $1/2\alpha$ [9], however, in practice the experimentally optimized value of β is different since the DRAG pulse also compensates phase errors from other sources. DRAG pulses can also minimize population leakage to these higher levels, but the value of β for which the pulse minimizes leakage is not generically the value for which the pulse maximizes fidelity. This point will be discussed in more detail in § IV and is also addressed in Ref. [14].

Sample RB data is shown in Fig. 2. For the $XY_{\frac{\pi}{2}}$ set, averaging over 5 runs of 20 seeds each, we get an error per Clifford (EPC) of $5.6(1)\times10^{-4}$ and an error per gate in the set (EPG) of $2.48(5)\times10^{-4}$. For the HZ set, averaging over 10 runs, the EPC is $3.0(1)\times10^{-4}$ and the EPG is $1.22(4)\times10^{-4}$. The lower EPC of the HZ set is evident from the data in Fig. 2, and confirms our expectation that the VZ-gates are significantly better than finite-length X and Y gates. To quantify the error of the VZ-gates we perform interleaved RB [15] of the S gate; sample data is shown in Fig. 2. Averaging over 5 runs we get an error of -1.7(1.0)$\times10^{-5}$ with systematic errors bounds of [0,6$\times10^{-4}$]. This is consistent with the VZ-gate having zero error and, therefore, being a perfect gate.

The RB data demonstrate the advantage of utilizing the VZ-gates in quantum algorithms. In essence, each of the curves is implementing the same RB algorithm, i.e., generate a sequence of m random Cliffords that constructs the identity operator. By utilizing VZ-gates we are able to implement the algorithm with higher fidelity. Therefore, VZ-gates can lower error rates in many algorithms by optimizing the circuit to mazimize the number of Z-gates.

III. CORRECTING ERRORS WITH VZ-GATES

Beyond their direct use in quantum circuits, VZ-gates can correct for certain pulse errors that occur during physical one and two-qubit gates without introducing new errors. For example, a VZ-gate can correct a phase error, i.e. an unwanted Z-gate, by applying the inverse Z-gate. VZ-gates can also correct most off-resonance-rotation (ORR) errors. The unitary operator due to an ORR along the X axis is,

$$U_1 = e^{-it\left[\frac{\Omega}{2}\hat{\sigma}_X + \Delta\hat{\sigma}_Z\right]}, \tag{22}$$

$$U_1 = e^{-i\frac{\Omega_R t}{2}\left[\cos(\lambda)\hat{\sigma}_X + \sin(\lambda)\hat{\sigma}_Z\right]}, \tag{23}$$

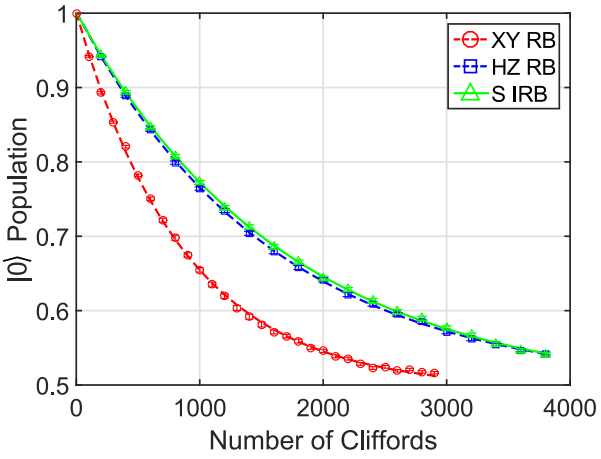


FIG. 2. (Color Online) RB curves for the two basis sets discussed in the main text: $XY_{\frac{\pi}{2}}$ (red circles) and HZ (blue squares). Interleaved RB for the S gate using the HZ set (green triangles). Each point is the average of 20 random seeds run in 5 separate experiments and fit to the standard RB exponential decay curve $Ar^m + B$ where m is the number of Clifford gates and the average error per Clifford is $\frac{1}{2}(1-r)$. The average error per gate is $\frac{1}{2}(1-r^{1/N_g})$ where N_g is the number of gates in the basis set required to implement a Clifford [13].

where $\tan(\lambda) = \frac{\Delta}{\Omega}$ and $\Omega_R = \sqrt{\Omega^2 + \Delta^2}$. If the goal is to implement the gate X_θ, then the question is whether there is a Z correction that can be applied to Eqn. 23, i.e., is there a ξ and Ω_R such that

$$X_\theta = Z_\xi \cdot U_1(\Omega_R t, \lambda) \cdot Z_\xi? \tag{24}$$

If we expand Eqn. (24) then we get the following relations,

$$\sin\left(\frac{\Omega_R t}{2}\right) = \frac{\sin\left(\frac{\theta}{2}\right)}{\cos(\lambda)}, \tag{25}$$

$$\tan(\xi) = \sin(\lambda)\tan\left(\frac{\Omega_R t}{2}\right), \tag{26}$$

and so there is a valid correction for the ORR error when $\frac{\sin\left(\frac{\theta}{2}\right)}{\cos(\lambda)} \leq 1$. For example, if $\lambda = 0.1$, and the desired gate is $X_{\frac{\pi}{2}}$ then $\xi \approx 0.1$ and $\Omega_R t = \pi/2 + 0.1^2$. Graphically, an exagerrated ORR for $X_{\pi/2}$ is illustrated on the Bloch sphere in Fig. 3. Starting from $|0\rangle$ the rotation is off-axis and so the final state is not along the Y axis. However, a rotation angle exists so that the state still crosses the XY plane and then a final VZ-gate corrects the angle error. Physically there is no solution when the rotation is sufficiently off-resonance such that it cannot pass through the plane defined by the desired final state. For example, a detuned π pulse cannot be compensated since a qubit starting in state $|0\rangle$ does not complete the rotation to $|1\rangle$.

Correcting ORR errors is of practical importance for weakly anharmonic transmon qubits. When resonantly driving the $|0\rangle$ to $|1\rangle$ transition, the drive frequency is

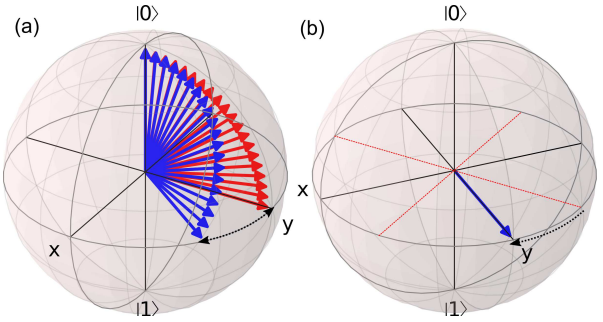


FIG. 3. (Color Online) (a) Bloch-sphere representation of an attempted $\frac{\pi}{2}$ rotation around the X axis starting in the state $|0\rangle$ on-resonance (red/light gray) and with a detuned drive (blue/dark gray). For a suitably compensated rotation angle the detuned drive ends up in the XY place, but with a finite Z-rotation error. (b) Correcting the error using a VZ-gate, i.e., axes rotation.

only slightly detuned from the higher level transitions such as $|1\rangle$ to $|2\rangle$, and so there is a strong Stark effect which shifts the frequency of the $|1\rangle$ state during the drive. The strength of the Stark shift is inversely proportional to the detuning, and thus ORR errors increase for short gates because of Fourier broadening of the drive frequency. The standard approach to correct these errors is to utilize DRAG pulse shaping, Eqn. (21), as we did for the RB data in § II. Here we show similar performance using Gaussian pulses with VZ-gates used to correct ORR errors in the form given by Eqn. (24); we refer to the combined pulse as Gaussian plus Z (GZ). In Fig. 4 we plot the error per gate (EPG) from $XY_{\frac{\pi}{2}}$ RB for DRAG, GZ and Gaussian pulses versus the sideband frequency of the pulse. Interestingly, the EPG for the Gaussian pulse is a strong function of the sideband frequency. This can be understood to be the result of the internal filtering of the AWG. In the rotating frame of the qubit the effect of this filter on the pulse shape is similar to the DRAG pulse shape. The value and sign of β is a function of the sideband frequency and so for certain frequencies the pulse shape exacerbates the ORR error. When we apply the Z gate correction the calibrated phase compensates for any passive DRAG shaping. From this data we conclude that GZ pulses are a viable alternative to DRAG when optimizing pulse fidelity. GZ pulses are not sensitive to the exact shape of the pulse and, as we will discuss next, permit the utilization of pulse shaping to address different errors such as leakage.

IV. LEAKAGE

In addition to unitary ORR errors, there is also population leakage to higher levels when driving transmon qubits. This leakage is mainly caused by frequency components in the drive at the transition frequency between the $|1\rangle$ and $|2\rangle$ states, $\omega_{12} = \omega_{01} + \alpha$. Pulse shaping,

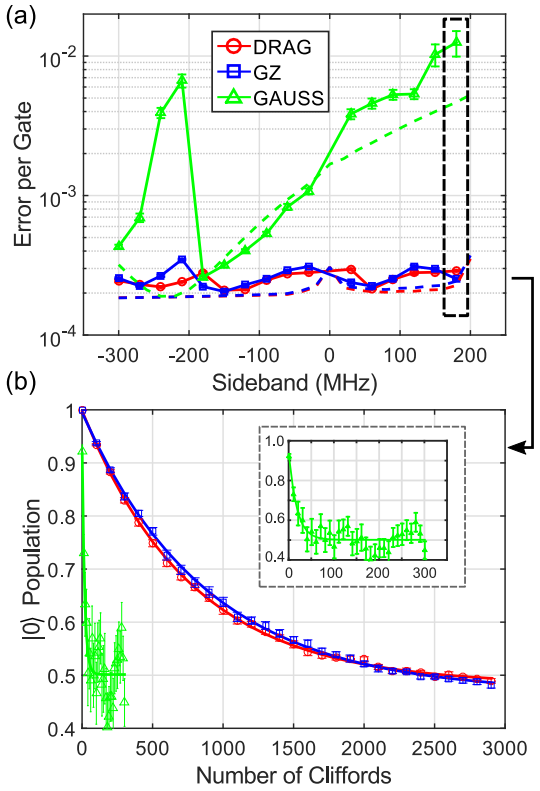


FIG. 4. (Color Online) (a) EPG for the $XY_{\frac{\pi}{2}}$ set (see main text) for three different pulse implementations: Gaussian, DRAG and Gaussian plus VZ-gate (GZ) as a function of the sideband frequency. Dashed lines are theory fits assuming the coherence numbers listed in the main text, LO leakage of -65 dBm and an internal AWG filter approximately as a Gaussian filter with a bandwidth of 300 MHz[16]. (b) Sample RB curves for the different pulses for a sideband frequency of 180 MHZ. The DRAG and GZ pulses are coherence limited, but the Gaussian pulse is completely dominated by unitary errors.

i.e., DRAG, can effectively mitigate leakage when VZ gates can be utilized to correct the unitary ORR errors (see Ref [14] for similar work using DRAG and frequency chirped pulses). Here we analyze leakage versus pulse width for different pulse types: DRAG optimized for fidelity, GZ, DRAG optimized for leakage with VZ-gates to correct ORR errors (DRAGZ) and GZ with the AWG outputs externally low-pass filtered (FILTZ). For these leakage experiments we operate with a sideband frequency of -120MHz, which has two advantages. For one, leakage components at $\omega_{\rm LO}$ and $\omega_{\rm LO} - \omega_{\rm SSB}$ due to non-idealities in the mixer are detuned by at least $|\alpha + \omega_{\rm SSB}|$ from ω_{12}. Second, by selecting a negative sideband we can passively filter the AWG for signals at $|\alpha + \omega_{\rm SSB}|$ which could mix with $\omega_{\rm LO}$ to produce ω_{12}. For the qubit in these experiments $|\alpha + \omega_{\rm SSB}|/2\pi = 355.5$ MHz, so a LP filter between 120 MHz and 355.5 MHz can effectively filter leakage components and leave enough bandwidth to drive short pulses. Specifically, we filtering using Mini-

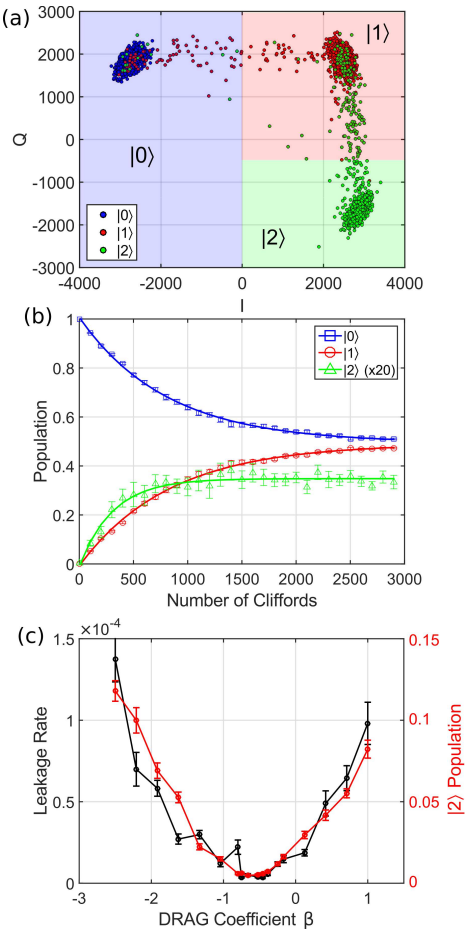


FIG. 5. (Color Online) (a)Single-shot measurements of the $|0\rangle$ (blue, left),$|1\rangle$ (red, top-right) and $|2\rangle$ (green, bottom-right) states in the IQ plane (1000 shots). The colored regions demonstrate the hard thresholding barriers used to bin measurements. For the RB data we use the results from this calibration to construct a POVM and invert the hard-threshold results to compensate for assignment errors. (b) Typical RB measurement for leakage. The $|2\rangle$ state population is multiplied by 20 times for scale. (c) Leakage rate and $|2\rangle$ population after 2901 Clifford gates (averaged over 20 seeds) versus the DRAG parameter β (black). This $|2\rangle$ population is used to calibrate β for the DRAGZ pulses (red/light gray).

Circuits VLF-180 (3dB frequency of 270 MHz). By filtering the AWG output we are effectively implementing a passive form of pulse shaping.

To measure leakage we perform standard RB sequences and measure the $|0\rangle$, $|1\rangle$ and $|2\rangle$ populations simultaneously by hard thresholding single-shot readout signals as shown in Fig. 5 (a). A sample RB curve for the 3 states is shown in Fig. 5 (b). The EPG is measured in the standard way by fitting the $|0\rangle$ state as described in the caption to Fig. 2. To measure the leakage rate per gate (LPG) we fit the $|2\rangle$ state RB data to the same type of RB curve $Ar^m + B$ and the LPG is given by the expression $p = (1-r)B/N_g$ [17]. In general there is a correction to the RB fit of the $|0\rangle$ data due to leakage, however, when EPG$\gg$LPG this correction is small [17]. A typical calibration curve for the DRAG parameter of the DRAGZ pulses is shown in Fig. 5 (c). An efficient proxy for the LPG is the averaged $|2\rangle$ population for a long Clifford sequence.

The LPG and EPG versus pulse width for the four pulse types is illustrated in Fig. 6. DRAG and GZ give similar results with a general trend of lower LPG for longer pulse widths as expected due to Fourier broadening. The exception to this trend is a pronounced minima at 10 ns which is an artifact of the Gaussian truncation such that there is a zero in the pulse spectrum at ω_{12}; this effect is captured accurately in the numerics. For pulses shorter than 20ns the DRAGZ and FILTZ pulses demonstrate nearly an order-of-magnitude lower LPG. For a pulse length of 13.3 ns we measure the LPG ($\times 10^{-6}$) for the various pulse types: DRAGZ 3.1(6), FILTZ 1.4(4), DRAG 13(1) and GZ 25(2). Overall, each of the pulses obtains similar EPG ($\times 10^{-4}$): DRAGZ 1.95(3), FILTZ 2.80(6), DRAG 2.24(3), GZ 2.75(6). These are all close to the coherence limit of 1.8×10^{-4}. While the LPG sets a lower bound on the EPG, significant gains in coherence will be required to reach that bound. For pulses greater than 20 ns the LPG rises; this is observed in theory calculations which include thermal relaxation to an effective system temperature of $T = 46$ mK. When the system is at a finite temperature there is incoherent population transfer between levels m and n such that in equilibrium the ratio of populations is $e^{-(E_n - E_m)/k_B T}$. Therefore, finite-temperature heating sets a lower bound on the leakage, which is also the conclusion of Ref. [14]. Understanding why the qubit is higher temperature than the cryostat ($10 - 15$ mK) and how to reduce that temperature is an active area of investigation.

Overall, replacing DRAG with GZ pulses does not affect the EPG and instead it frees up DRAG pulse shaping to specifically minimize leakage using DRAGZ pulses. The lowest LPG is obtained with FILTZ, albeit with similar performance to DRAGZ and with the caveat that it only works for specific sideband frequencies. Ultimately, leakage is not a limiting factor for single qubit gate performance, however, leakage can have detrimental effects on error correction protocols [18, 19].

V. THE VZ-GATE IN MULTIQUBIT SYSTEMS

Employing VZ-gates in multiqubit systems depends on the specific implementation of the two-qubit gate interaction. For example, consider a two-qubit Hamiltonian with the interaction term $\hat{\sigma}_Z^{(1)} \otimes \hat{\sigma}_X^{(2)}$,

$$H/\hbar = -\frac{\omega_1}{2}\hat{\sigma}_Z^{(1)} - \frac{\omega_2}{2}\hat{\sigma}_Z^{(2)} + g\hat{\sigma}_Z^{(1)} \otimes \hat{\sigma}_X^{(2)}. \quad (27)$$

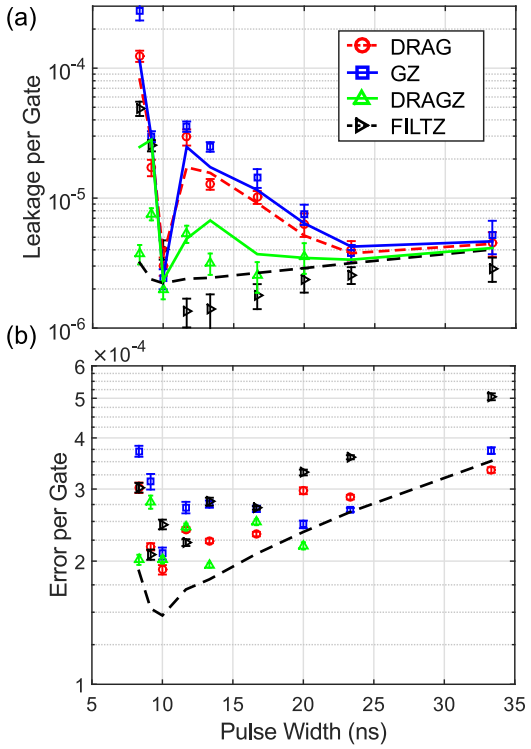


FIG. 6. (Color Online) (a) Leakage per gate versus pulse length for DRAG, GZ, DRAGZ and FILTZ. Lines are theory curves assuming the coherences mentioned in the main text, a temperature of 46 mK, LO leakage of -65 dBm and an AWG filter bandwidth of 300 MHz (100 MHz for FILTZ). DRAGZ points were only taken to a pulse length of 20 ns; beyond this point calibration of the DRAG parameter (to minimize leakage) was not reliable because the leakage signal was below the noise floor. (b) EPG from the same measurement. The black dashed line is a theory curve for a perfect DRAG pulse (i.e. no mixer or AWG effects) and is representative of the coherence limited error.

In the rotating frame of the single-qubit drives, $U_{\rm rot} = e^{-i\left(\frac{\omega_1 t+\phi_1}{2}\hat{\sigma}_Z^{(1)}+\frac{\omega_2 t+\phi_2}{2}\hat{\sigma}_Z^{(2)}\right)}$,

$$\tilde{H}/\hbar = g\hat{\sigma}_Z^{(1)} \otimes \Big(\cos(\omega_2 t+\phi_2)\hat{\sigma}_X^{(2)} + \sin(\omega_2 t+\phi_2)\hat{\sigma}_Y^{(2)}\Big), \quad (28)$$

and so the single-qubit drive phase is imprinted on the two-qubit interaction if the interaction term is $\hat{\sigma}_X$ or $\hat{\sigma}_Y$. When we apply a VZ-gate, the phase update to the single-qubit drive will affect subsequent two-qubit interactions; how to manage this issue is dependent on the specific implementation of that interaction.

For microwave-activated gates the phase update is straightforward. Here we will give two examples for the cross-resonance gate and the parametric iSWAP gate. The cross-resonance (CR) gate (see e.g. Ref [11]) interaction is the example considered in Eqn. (27). To turn on the CR interaction the ZX term is modulated at ω_2 with a separate drive. The CR rotation angle is defined with respect to the the phase ϕ_2 as shown in Eqn. (28). When a VZ-gate is applied to qubit 2 the phase of the CR drive must be updated accordingly. For the iSWAP gate (see e.g. Ref. [10]), an $XX+YY$ term is activated by modulating at the difference frequency $\omega_1-\omega_2$. The phase of this iSWAP drive is matched to the difference of the single-qubit drive phases $\phi_1-\phi_2$. Therefore, the VZ-gate phase is applied to the iSWAP drive with a different sign for qubits 1 and 2.

For flux-tunable qubits, i.e., where the qubit frequencies are dynamically tuned to go to an interaction resonance, compatibility with the VZ-gate is more difficult. In these systems there are time dependent single-qubit σ_Z terms in Eqn. (28) which do not necessarily commute with the interaction. Therefore VZ-gates before the interaction necessitate also updating the σ_Z dynamics. However, if the interaction is ZZ (see e.g. Refs [12, 20]), then these single-qubit Z terms commute through and can be compensated by a subsequent VZ-gate.

VI. CONCLUSIONS

In conclusion, we investigate a method to implement a near-perfect Z-gate by controlling the phase of the microwave drive used for X and Y rotations – the virtual Z-gate (VZ-gate). This gate can improve the fidelity of circuits with a large number of single-qubit gates, can be used to efficiently correct typical gate errors and be used to implement arbitrary SU(2) gates given a calibrated $X_{\pi/2}$ gate. In this work we used VZ-gates to correct single-qubit rotation errors, but the gate should have wide applicability for improving two-qubit gates. In particular, as the number of qubits increases, crosstalk Z-errors will be ubiquitous. The VZ-gate is a low-overhead method for correcting these type of errors. By using pulse shaping techniques to minimize leakage and VZ gates to correct rotation errors we demonstrated a hybrid pulse with leakage limited by the qubit temperature and gate fidelity limited by coherence. Further improvements in leakage are limited by the qubit temperature.

ACKNOWLEDGMENTS

We acknowledge Firat Solgun, George Keefe and Markus Brink for the simulation, layout and fabrication of devices. We acknowledge useful discussions with Lev Bishop. This work was supported by the Army Research Office under contract W911NF-14-1-0124.

Meta Large Language Model Compiler: Foundation Models of Compiler Optimization

Chris Cummins[†], **Volker Seeker**[†], **Dejan Grubisic, Baptiste Rozière, Jonas Gehring, Gabriel Synnaeve, Hugh Leather**[†]

Meta AI

Abstract

Large Language Models (LLMs) have demonstrated remarkable capabilities across a variety of software engineering and coding tasks. However, their application in the domain of code and compiler optimization remains underexplored. Training LLMs is resource-intensive, requiring substantial GPU hours and extensive data collection, which can be prohibitive. To address this gap, we introduce Meta Large Language Model Compiler (LLM Compiler), a suite of robust, openly available, pre-trained models specifically designed for code optimization tasks. Built on the foundation of Code Llama, LLM Compiler enhances the understanding of compiler intermediate representations (IRs), assembly language, and optimization techniques. The model has been trained on a vast corpus of 546 billion tokens of LLVM-IR and assembly code and has undergone instruction fine-tuning to interpret compiler behavior. LLM Compiler is released under a bespoke commercial license to allow wide reuse and is available in two sizes: 7 billion and 13 billion parameters. We also present fine-tuned versions of the model, demonstrating its enhanced capabilities in optimizing code size and disassembling from x86_64 and ARM assembly back into LLVM-IR. These achieve 77% of the optimising potential of an autotuning search, and 45% disassembly round trip (14% exact match). This release aims to provide a scalable, cost-effective foundation for further research and development in compiler optimization by both academic researchers and industry practitioners.

1 Introduction

There is increasing interest in large language models (LLMs) for software engineering tasks including code generation, code translation, and code testing. Models such as StarCoder (Lozhkov et al., 2024), Code Llama (Rozière et al., 2023), and GPT-4 (OpenAI, 2023) have a good statistical understanding of code and can suggest likely completions for unfinished code, making them useful for editing and creating software. However, there is little emphasis on training specifically to optimize code. Publicly available LLMs can be prompted to make minor tweaks to a program such as tagging variables to be stored as registers, and will even attempt more substantial optimizations like vectorization, though they easily become confused and make mistakes, frequently resulting in incorrect code.

Prior works on machine learning-guided code optimization have used a range of representations from hand-built features (Wang & O'Boyle, 2018) to graph neural networks (GNNs) (Liang et al., 2023). However, in all cases, the way the input program is represented to the machine learning algorithm is incomplete, losing some information along the way. For example, Trofin et al. (2021) use numeric features to provide hints for function inlining, but cannot faithfully reproduce the call graph or control flow. Cummins et al. (2021) form graphs of the program to pass to a GNN, but exclude the values of constants and some type information which prevents reproducing instructions with fidelity.

In contrast, LLMs can accept source programs, as is, with a complete, lossless representation. Using text as the input and output representation for a machine learning optimizer has desirable properties: text is a

†: Core contributors.

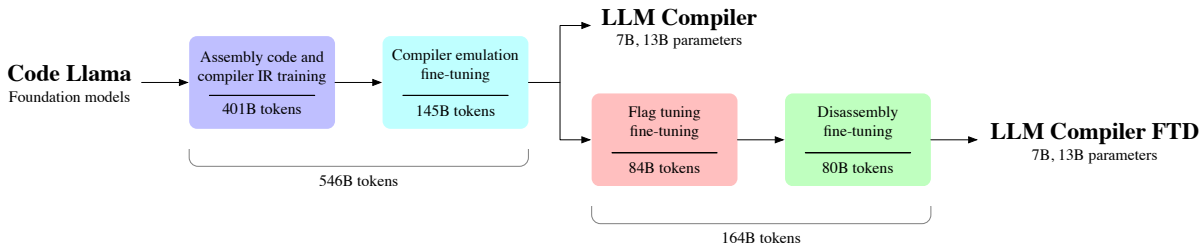


Figure 1: LLM Compiler models are specialized from Code Llama by training on 546 billion tokens of compiler-centric data in two stages. In the first stage the models are trained predominantly on unlabelled compiler IRs and assembly code. In the next stage the models are instruction fine-tuned to predict the output and effect of optimizations. LLM Compiler FTD models are then further fine-tuned on 164 billion tokens of downstream flag tuning and disassembly task datasets, for a total of 710 billion training tokens. During each of the four stages of training, 15% of data from the previous tasks is retained.

universal, portable, and accessible interface, and unlike prior approaches is not specialized to any particular task.

However, training LLMs incurs high cost in both compute and data. For example, training Code Llama's models consumed 1.4M A100 GPU hours to train, and curating the vast amounts of training data (hundreds of billions of tokens) can be challenging. These costs are often prohibitive to researchers in the field and this blocks advances that might otherwise be possible.

To address this issue, we are releasing LLM Compiler, a family of foundation models that have already been trained to understand the semantics of compiler IRs and assemblies and to emulate the compiler, allowing for easy fine-tuning with minimal data for specific downstream compiler optimization tasks. Building upon Code Llama, we extend its capabilities to encompass compiler optimization and reasoning.

The training pipeline for LLM Compiler is illustrated in Figure 1. We extend Code Llama with additional pretraining on a vast corpus of assembly codes and compiler IRs, and then instruction fine-tune on a bespoke *compiler emulation* dataset to better reason about code optimization. Our intention with releasing these models is to provide a foundation for researchers and industry practitioners to further develop code optimization models. We then adapt the models for two downstream compilation tasks: tuning compiler flags to optimize for code size, and disassembling x86_64 and ARM assembly to LLVM-IR. We also release these LLM Compiler FTD models to the community under the same bespoke commercial license. Compared to the autotuning technique on which it was trained, LLM Compiler FTD achieves 77% of the optimizing potential without the need for any additional compilations. When disassembling, LLM Compiler FTD creates correct disassembly 14% of the time. On both tasks LLM Compiler FTD models significantly outperform comparable LLMs Code Llama and GPT-4 Turbo.

Our work aims to establish a scalable, cost-effective foundation for further research and development in compiler optimization, catering to both academic researchers and industry practitioners. By providing access to pre-trained models in two sizes (7 billion and 13 billion parameters) and demonstrating their effectiveness through fine-tuned versions, LLM Compiler paves the way for exploring the untapped potential of LLMs in the realm of code and compiler optimization.

1.1 Overview

Figure 1 shows an overview of our approach. LLM Compiler models target compiler optimization. They are available in two model sizes: 7B and 13B parameters. The LLM Compiler models are initialized with Code Llama model weights of the corresponding size and trained on an additional 546B tokens of data comprising mostly compiler intermediate representations and assembly code. We then further train LLM Compiler FTD models using an additional 164B tokens of data for two downstream compilation tasks: flag tuning and disassembly. At all stages of training a small amount of code and natural language data from previous stages is used to help retain the capabilities of the base Code Llama model.

Dataset	Sampling prop.	Epochs	Disk size
IR and assembly pretraining (401 billion tokens)			
Code	85.00%	1.000	872 GB
Natural language related to code	14.00%	0.019	942 GB
Natural language	1.00%	0.001	938 GB
Compiler emulation (additional 145 billion tokens)			
Compiler emulation	85.00%	1.702	175 GB
Code	13.00%	0.055	872 GB
Natural language related to code	1.80%	0.001	942 GB
Natural language	0.20%	6.9e−5	938 GB
Flag tuning fine-tuning (additional 84 billion tokens)			
Flag tuning	85.00%	1.700	103 GB
Compiler emulation	11.73%	0.136	175 GB
Code	2.84%	0.007	872 GB
Natural language related to code	0.40%	1.1e−4	942 GB
Natural language	0.03%	8.8e−6	938 GB
Disassembly fine-tuning (additional 80 billion tokens)			
Disassembly	85.00%	1.707	88 GB
Flag tuning	4.68%	0.089	103 GB
Compiler emulation	8.07%	0.089	175 GB
Code	1.96%	0.004	872 GB
Natural language related to code	0.27%	7.5e−5	942 GB
Natural language	0.03%	5.7e−6	938 GB

Table 1: Training datasets used.

	Items	Tokens	Disk size
LLVM-IR	10.7 M	185 B	432 GB
Assembly	10.1 M	216 B	440 GB
Total	20.8 M	401 B	872 GB

(a) Language

	Items	Tokens	Disk size
x86_64-unknown-linux-gnu	17.3 M	340.3 B	738 GB
aarch64-unknown-linux-gnu	3.5 M	60.5 B	133 GB
nvptx64-nvidia-cuda	9.2 k	146 M	286 MB
Total	20.8 M	401 B	872 GB

(b) Target

Table 2: Composition of data used for initial IR and assembly pretraining. LLM Compiler is trained on a near-even split of IR and assembly code, predominantly targeting x86-64 architecture, with some 64-bit ARM, and a small amount of CUDA.

2 LLM Compiler: Specializing Code Llama for compiler optimization

2.1 Pretraining on assembly code and compiler IRs

The data used to train coding LLMs are typically composed largely of high level source languages like Python. Assembly code contributes a negligible proportion of these datasets, and compiler IRs even less. To build an LLM with a good understanding of these languages we initialize LLM Compiler models with the weights of Code Llama and then train for 401 billion tokens on a compiler-centric dataset composed mostly of assembly code and compiler IRs, shown in Table 1.

Dataset LLM Compiler is trained predominantly on compiler intermediate representations and assembly code generated by LLVM (Lattner & Adve, 2004) version 17.0.6. These are derived from the same dataset of publicly available code used to train Code Llama. We summarize this dataset in Table 2. As in Code Llama, we also source a small proportion of training batches from natural language datasets.

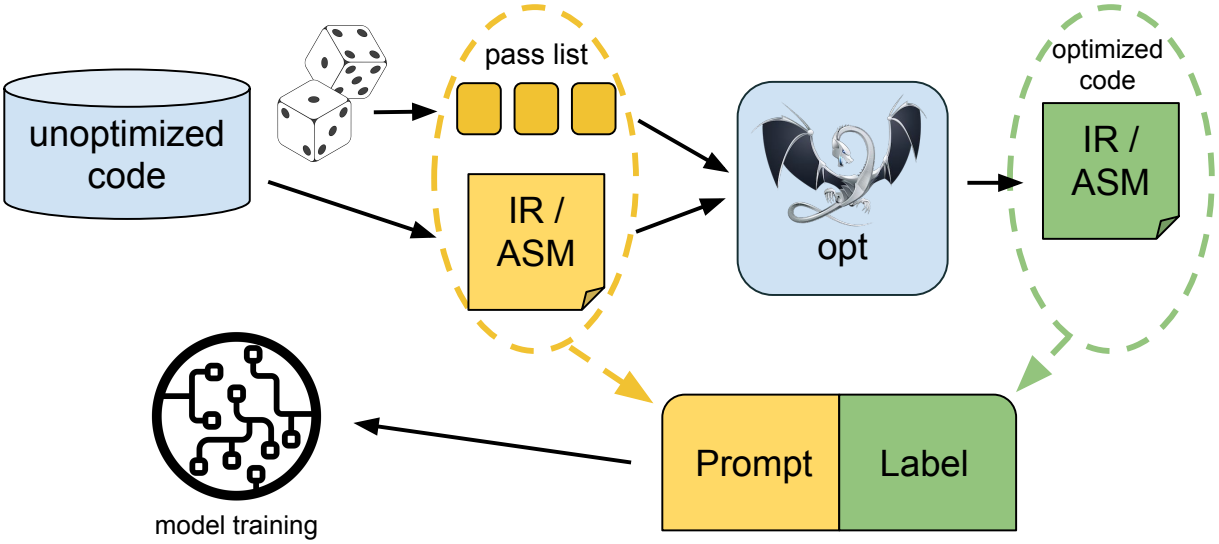


Figure 2: To give the model an understanding of how compiler optimizations work, we use *compiler emulation*. Unoptimized code samples and random pass lists are given to `opt` to generate optimized code (IR or assembly). Pass list and input code are taken together as prompt while the generated output code is used as label.

```
opt input.bc -o output.bc -p 'module(default<Oz>),module(iroutliner)'
clang output.bc -o output.o
size output.o
```

Listing 1: Commands used to apply an optimization pipeline comprising -Oz passes followed by IR outlining to an unoptimized IR `input.bc`. Binary size is the sum of `.TEXT` and `.DATA` section sizes of the lowered object file as reported by `size`.

2.2 Instruction fine-tuning for compiler emulation

To understand the mechanism of code optimization we instruction fine-tune LLM Compiler models to emulate compiler optimizations, illustrated in Figure 2. The idea is to generate from a finite set of unoptimized seed programs a large number of examples by applying randomly generated sequences of compiler optimizations to these programs. We then train the model to predict the code generated by the optimizations. We also train the model to predict the code size after the optimizations have been applied.

Task specification. Given unoptimized LLVM-IR (as emitted by the *clang* frontend), a list of optimization passes, and a starting code size, generate the resulting code after those optimizations have been applied and the resulting code size.

There are two flavors of this task: in the first the model is expected to output compiler IR, in the second the model is expected to output assembly code. The input IR, optimization passes, and code size are the same for both flavors. The prompt dictates the required output format. Examples of each prompt are provided in Appendices Listings 2 and 3.

Code size. We use two metrics for code size: the number of IR instructions, and *binary size*. Binary size is computed by summing the size of the `.TEXT` and `.DATA` sections of the IR or assembly after lowering to an object file; we exclude `.BSS` section from our binary size metric since it does not affect on-disk size.

Optimization passes. In this work we target LLVM 17.0.6 and use the New Pass Manager (PM, 2021) which classifies passes for different levels such as *module*, *function*, *loop*, etc. as well as transformation and analysis passes. Transformation passes change given input IR while analysis passes generate information that influence subsequent transformations.

Of the 346 possible pass arguments for *opt*, we select 167 to use. This includes each of the default optimization pipelines (e.g. `module(default<Oz>)`), individual optimization transform passes (e.g. `module(constmerge)`), but

excludes non-optimization utility passes (e.g. `module(dot-callgraph)`) and transformations passes that are not semantics preserving (e.g. `module(internalize)`). We exclude analysis passes since they have no side effects and we rely on the pass manager to inject dependent analysis passes as needed. For passes that accept parameter arguments we use the default values (e.g. `module(licm<allowspeculation>)`). Table 9 contains a list of all passes used. We used LLVM's *opt* tool to apply pass lists and *clang* to lower the resulting IR to object file. Listing 1 shows the commands used.

Dataset. We generated the compiler emulation dataset by applying random lists of between 1 and 50 optimization passes to unoptimized programs summarized in Table 2. The length of each pass list was selected uniformly at random. Pass lists were generated by uniformly sampling from the set of 167 passes described above. Pass lists which resulted in compiler crashes or timed out after 120 seconds were excluded.

3 LLM Compiler FTD: Extending for downstream compiler tasks

3.1 Instruction fine-tuning for optimization flag tuning

Manipulating compiler flags is well known to have a considerable impact on both runtime performance and code size (Fursin et al., 2005). We train LLM Compiler FTD models on the downstream task of selecting flags for LLVM's IR optimization tool *opt* to produce the smallest code size. Machine learning approaches to flag tuning have shown good results previously, but struggle with generalizing across different programs (Cummins et al., 2022). Previous works usually need to compile new programs tens or hundreds of times to try out different configurations and find out the best-performing option. We train and evaluate LLM Compiler FTD models on the zero-shot version of this task by predicting flags to minimize code size of unseen programs. Our approach is agnostic to the chosen compiler and optimization metric, and we intend to target runtime performance in the future. For now, optimizing for code size simplifies the collection of training data.

Task specification. We present the LLM Compiler FTD models with an unoptimized LLVM-IR (as emitted by the *clang* frontend) and ask it to produce a list of *opt* flags that should be applied to it, the binary size before and after these optimizations are applied, and the output code. If no improvement can be made over the input code, a short output message is generated that contains only the unoptimized binary size. Listings 4 and 5 provide the prompt and output templates for this task.

We used the same constrained set of optimization passes as in the compiler emulation task, and compute binary size in the same manner.

Figure 3 illustrates the process used to generate training data (described below) and how the model is used for inference. Only the generated pass list is needed at evaluation time. We extract the pass list from the model output and run *opt* using the given arguments. We can then evaluate the accuracy of the model predicted binary sizes and optimized output code, but those are auxiliary learning tasks not required for use.

Correctness. LLVM's optimizer is not free from bugs and running optimization passes in unexpected or untested orders may expose subtle correctness errors that undermine the utility of the model. To mitigate this risk we developed *PassListEval*, a tool to help in automatically identifying pass lists that break program semantics or cause compiler crashes. An overview of the tool is shown in Figure 4. PassListEval accepts as input a candidate pass list and evaluates it over a suite of 164 self-testing C++ programs, taken from HumanEval-X (Zheng et al., 2023). Each program contains a reference solution for a programming challenge, e.g. *"Check if in given vector of numbers, are any two numbers closer to each other than given threshold"*, and a suite of unit tests that validate correctness. We apply the candidate pass lists to the reference solution, and then link them against the test suites to produce a binary. When executed, the binary will crash if any of the tests fail. If any binary crashes, or if any of the compiler invocations fail, we reject the candidate pass list.

Dataset. We trained LLM Compiler FTD models on a dataset of flag tuning examples derived from 4.5M of the unoptimized IRs used for pretraining. To generate the example optimal pass list for each program we ran an extensive iterative compilation process depicted in Figure 3 and outlined below:

1. We used large-scale *random search* to generate an initial candidate best pass list for the programs. For each program we independently generated random lists of up to 50 passes by uniformly sampling from the set of 167 searchable passes described previously. Every time we evaluated a pass list on a program we recorded

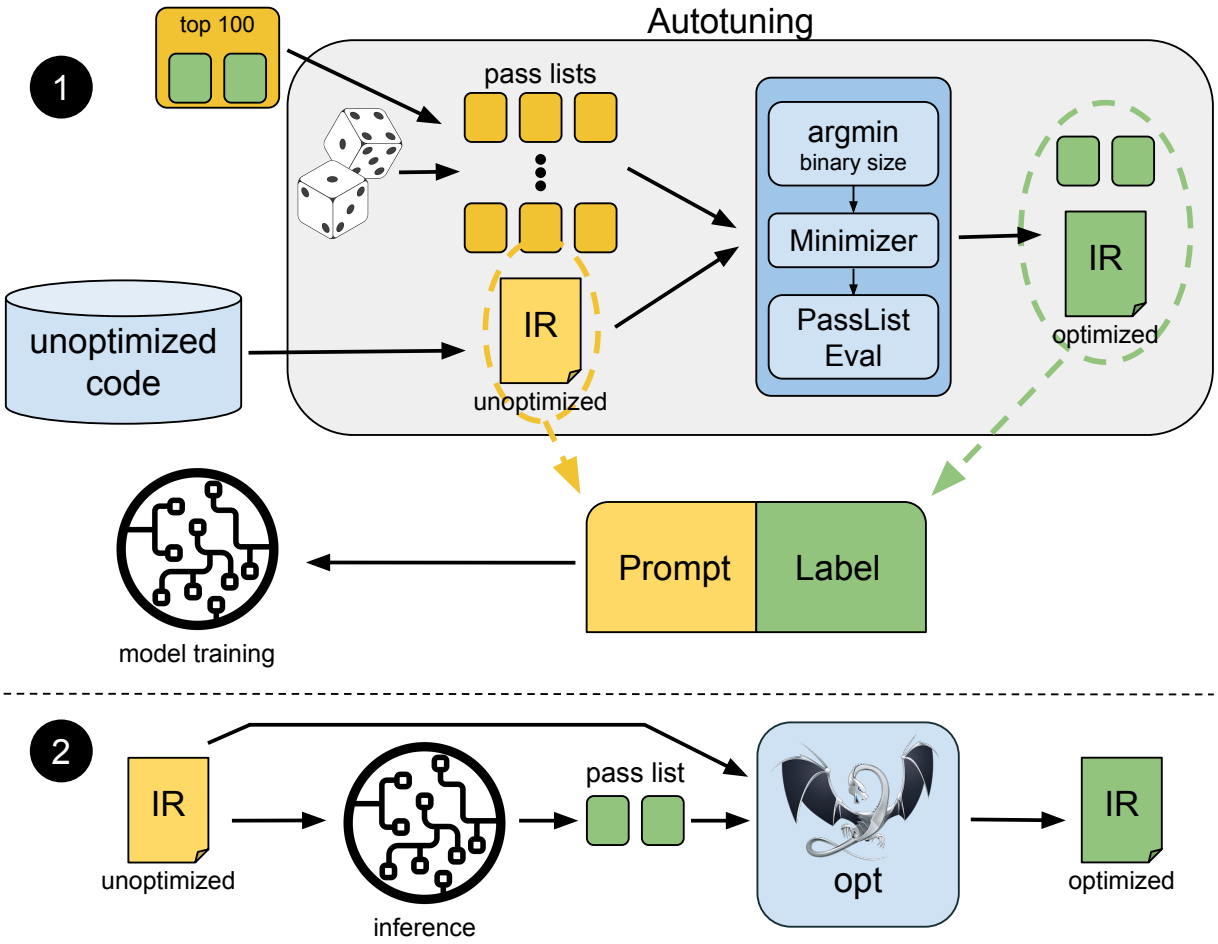


Figure 3: Overview of our approach, showing the model input (Prompt) and output (Label) during training ① and inference ②. The prompt contains unoptimized code. The label contains an optimization pass list, binary size, and the optimized code. To generate the label for the training prompt, the unoptimized code is compiled against multiple random pass lists. The pass list achieving the minimum binary size is selected, minimized and checked for correctness with PassListEval. The final pass list together with its corresponding optimized IR are used as label during training. In a last step, the top 100 most often selected pass lists are broadcast among all programs. For deployment we generate only the optimization pass list which we feed into the compiler, ensuring that the optimized code is correct.

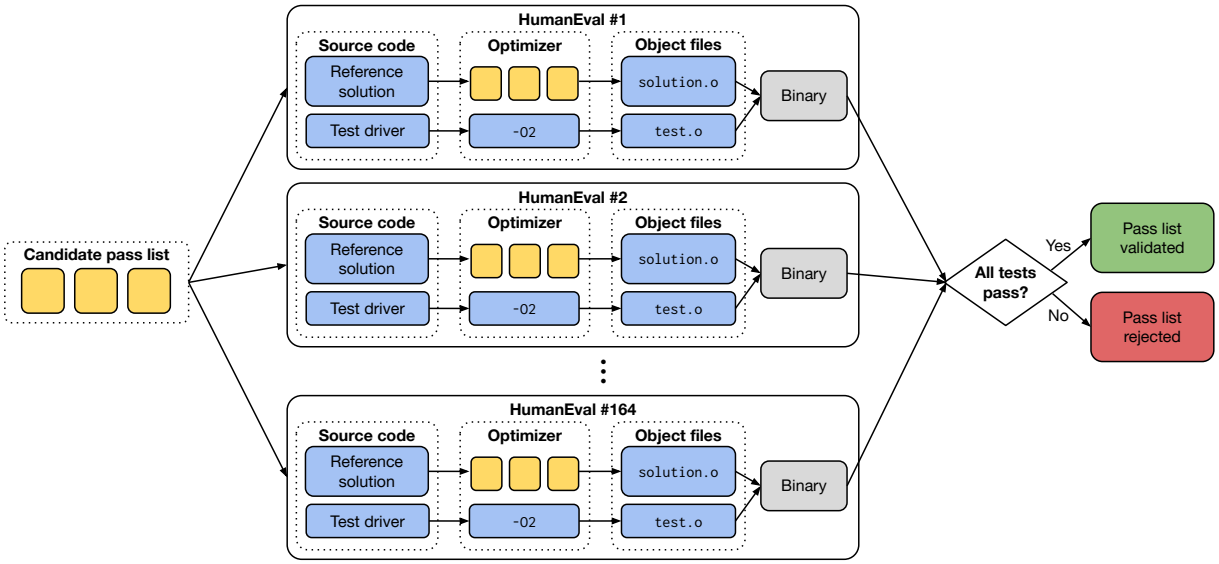


Figure 4: Validating a candidate list of optimization passes using PassListEval. The candidate pass list is applied to the reference solutions for all 164 programs in HumanEval-X. The unit tests for these reference solutions are optimized using a conservative -O2 pass pipeline to ensure correctness, and then linked against the reference solutions. The resulting binaries are executed and if any of the binaries crash during execution, or if any of the compiler invocations fail, the pass list is rejected.

the resulting binary size. We then pick the per-program pass lists that produced the lowest binary size. We ran 22 billion unique compilations for an average 4,877 per program.

2. The pass lists generated by random search may contain redundant passes that have no effect on the final outcome. Further, some pass orderings are commutative such that reordering then does not affect the final outcome. Since these would introduce noise in our training data, we developed a *minimization* process which we applied to each pass list. Minimization comprises three steps: redundant pass elimination, bubble sort, and insertion search. In redundant pass elimination we minimize the best pass list by iteratively removing individual passes to see if they contribute to the binary size. If not, they are discarded. This is repeated until no further passes can be discarded. Bubble sort then attempts to provide a uniform ordering for pass subsequences by sorting passes based on a key. Finally, insertion sort performs a local search by iterating over each pass in the pass list and attempting to insert each of the 167 search passes before it. If doing so improves the binary size, this new pass list is kept. The entire minimization pipeline loops until a fixed point is reached. The distribution of minimized pass list lengths is shown in Figure 9. The average pass list length is 3.84.
3. We apply *PassListEval*, described previously, to the candidate best pass lists. Through this we identified 167,971 of 1,704,443 unique pass lists (9.85%) as causing compile time or runtime errors.
4. We broadcast the *top 100* most frequently optimal pass lists across all programs, updating the per-program best pass lists if improvements are found. After this the total number of unique best pass lists decreases from 1,536,472 to 581,076.

The autotuning pipeline outlined above produced a geometric mean 7.1% reduction in binary size over -Oz. Figure 10 shows the frequency of individual passes. For our purposes, this autotuning serves as a gold standard for the optimization of each program. While the binary size savings discovered are significant, this required 28 billion additional compilations at a computational cost of over 21,000 CPU days. The goal of instruction fine-tuning LLM Compiler FTD to perform the flag tuning task is to achieve some fraction of the performance of the autotuner without requiring running the compiler thousands of times.

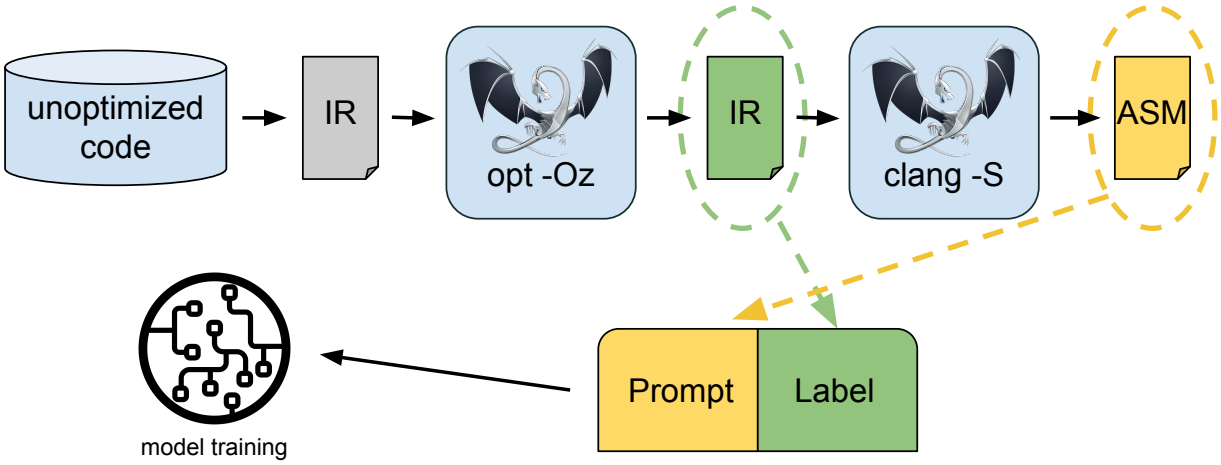


Figure 5: We train the model to understand the relationship between assembly and IR by training it to disassemble a given code sample to its corresponding IR. The IR used to label this training task was generated by optimizing an IR with the -Oz flag.

3.2 Instruction fine-tuning for disassembly

The ability to lift code from assembly back into higher level structures enables running additional optimizations on library code directly integrated with application code or porting of legacy code to new architectures. The field of decompilation has seen advancements in applying machine learning techniques to generate readable and accurate code from binary executables. Several studies explore the use of machine learning for decompilation tasks, such as lifting binaries into intermediate representations for evaluation against synthetic C programs (Cao et al., 2022), utilizing evolutionary approaches like genetic algorithms for program analysis (Schulte et al., 2018), and proposing methods like XLIR for matching binary code across different programming languages (Gui et al., 2022). Armengol-Estapé et al. (2024) have trained a language model to decompile x86 assembly into high level C code. In this study, we demonstrate how LLM Compiler FTD can learn the relationship between assembly code and compiler IR by fine-tuning it for disassembly. The task is to learn the inverse translation of `clang -xir - -o - -S`, shown in Figure 5.

Round tripping. Using an LLM for disassembly causes problems of correctness. The lifted code must be verified by an equivalence checker which is not always feasible or manually verified for correctness or subjected to sufficient test cases to give confidence. However, a lower bound on correctness can be found by round-tripping. That is to say by compiling the lifted IR back into assembly, if the assembly is identical then the IR is correct. This gives an easy route to using the results of the LLM and an easy way to measure the utility of a disassembly model.

Task specification. We provide the model with assembly code and train it to emit the corresponding disassembled IR. Listing 7 shows the prompt format. The context length for this task is set to 8k tokens for the input assembly code and 8k tokens for the output IR.

Dataset. We derive the assembly codes and IR pairs from the same dataset used in previous tasks. Our fine-tuning dataset consists in 4.7M samples. The input IR has been optimized with -Oz before being lowered to x86 assembly.

4 Training parameters

Data is tokenized via byte pair encoding (Gage, 1994), employing the same tokenizer as Code Llama, Llama (Touvron et al., 2023a), and Llama 2 (Touvron et al., 2023b).

We use the same training parameters for all four stages of training. Most of the training parameters we used are the same as for the Code Llama base model. We use the AdamW (Loshchilov & Hutter, 2017) optimizer with β_1 and β_2 values of 0.9 and 0.95. We use a cosine schedule with 1000 warm-up steps, and set the final learning rate to be 1/30th of the peak learning rate. Compared to the Code Llama base model, we increased the context length of individual sequences from 4,096 to 16,384, but kept the batch size constant at 4M tokens. To account for the longer context, we set our learning rate to $2e^{-5}$ and modified the parameters of the RoPE positional embeddings (Su et al., 2024) where

we reset frequencies with a base value of $\theta = 10^6$. These settings are in accordance with the long context training done for the Code Llama base model.

5 Evaluation

In this section we evaluate the performance of LLM Compiler models on the tasks of flag tuning and disassembly, compiler emulation, next-token prediction, and finally software engineering tasks.

5.1 Flag tuning task

Methodology. We evaluate LLM Compiler FTD on the task of optimization flag tuning for unseen programs and compare to GPT-4 Turbo and Code Llama - Instruct. We run inference on each model and extract from the model output the optimization pass list. We then use this pass list to optimize the particular program and record the binary size. The baseline is the binary size of the program when optimized using -Oz.

For GPT-4 Turbo and Code Llama - Instruct we append a suffix to the prompt with additional context to further describe the problem and expected output format. After some experimentation we found that the prompt suffix shown in Listing 6 provides the best performance.

All model-generated pass lists are validated using *PassListEval*, and -Oz is used as substitute if validation fails. To further validate correctness of model-generated pass lists we link the final program binaries and differential test their outputs against the outputs of the benchmark when optimized using a conservative -O2 optimization pipeline.

Dataset. We evaluate on 2,398 test prompts extracted from the MiBench benchmark suite (Guthaus et al., 2001). To generate these prompts we take all of the 713 translation units that make up the 24 MiBench benchmarks and generate unoptimized IRs from each. We then format them as prompts as per Listing 4. If the resulting prompt exceeds 15k tokens we split the LLVM module representing that translation unit into smaller modules, one for each function, using *llvm-extract*. This results in 1,985 prompts which fit within the 15k token context window, leaving 443 translation units which do not fit. We use -Oz when for the 443 excluded translation units when computing performance scores. Table 10 summarizes the benchmarks.

Results. Table 3 shows zero-shot performance of all models on the flag tuning task. Only LLM Compiler FTD models provide an improvement over -Oz, with the 13B parameter model marginally outperforming the smaller model, generating smaller object files than -Oz in 61% of cases.

In some cases the model-generated pass list causes a larger object file size than -Oz. For example, LLM Compiler FTD 13B regresses in 12% of cases. These regressions can be avoided by simply compiling the program twice: once using the model-generated pass list, once using -Oz, and selecting the pass list which produces the best result. By eliminating regressions wrt -Oz, these *-Oz backup* scores raise the overall improvement over -Oz to 5.26% for LLM Compiler FTD 13B, and enable modest improvements over -Oz for Code Llama - Instruct and GPT-4 Turbo. Figure 6 shows the performance of each model broken down by individual benchmark.

Binary size accuracy. While the model-generated binary size predictions have no effect on actual compilation, we can evaluate the performance of the models at predicting binary sizes before and after optimization to give an indication of each model's understanding of optimization. Figure 7 shows the results. LLM Compiler FTD binary size predictions correlate well with ground truth, with the 7B parameter model achieving MAPE values of 0.083 and 0.225 for unoptimized and optimized binary sizes respectively. The 13B parameter model improved has similar MAPE values of 0.082 and 0.225. Code Llama - Instruct and GPT-4 Turbo binary size predictions show little correlation with ground truth. We note that the LLM Compiler FTD errors are slightly higher for optimized code than unoptimized code. In particular, there is an occasional tendency for LLM Compiler FTD to overestimate the effectiveness of optimization, resulting in a lower predicted binary size than actual.

Ablation studies. Table 4 ablates the performance of models on a small holdout validation set of 500 prompts taken from the same distribution as our training data (though not used during training). We trained for flag tuning at each stage of the training pipeline from Figure 1 to compare performance. As shown, disassembly training causes a slight regression in performance from average 5.15% to 5.12% improvement over -Oz. We also show performance of the autotuner used for generating the training data described in Section 2. LLM Compiler FTD achieves 77% of the performance of the autotuner.

Table 3: Comparison of model performance when flag tuning 2,398 object files from MiBench. *Overall improvement* scores include 443 object files which do not fit in the context window of LLM Compiler FTD. For GPT-4 and the Code Llama models we appended a suffix to the prompt to provide additional context (see Listing 6 in the Appendix).

	Size	Improved	Regressed	Overall improvement over -Oz zero-shot	Overall improvement over -Oz -Oz backup
LLM Compiler FTD	7B	1,465	302	4.77%	5.24%
	13B	**1,466**	**299**	**4.88%**	**5.26%**
Code Llama - Instruct	7B	379	892	-0.49%	0.23%
	13B	319	764	-0.42%	0.18%
	34B	230	493	-0.27%	0.15%
GPT-4 Turbo (2024-04-09)	-	13	24	-0.01%	0.03%

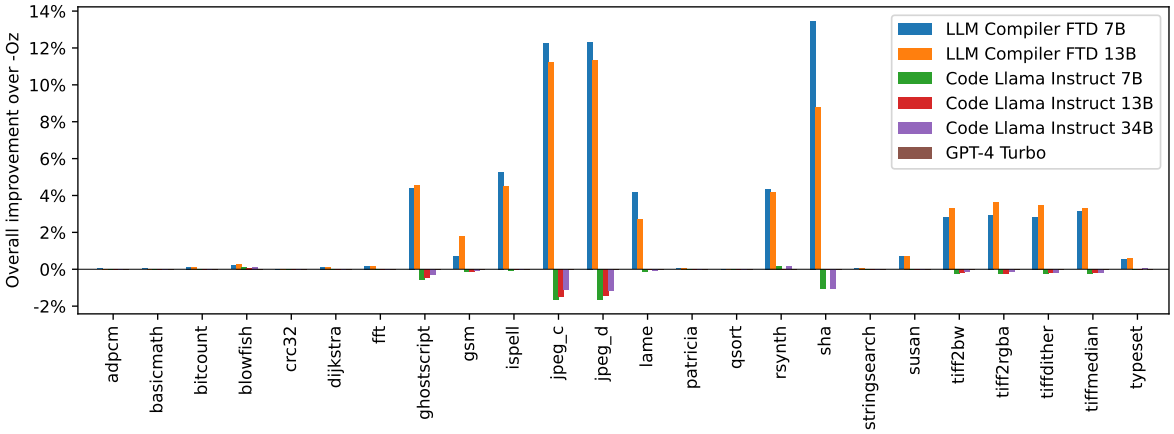
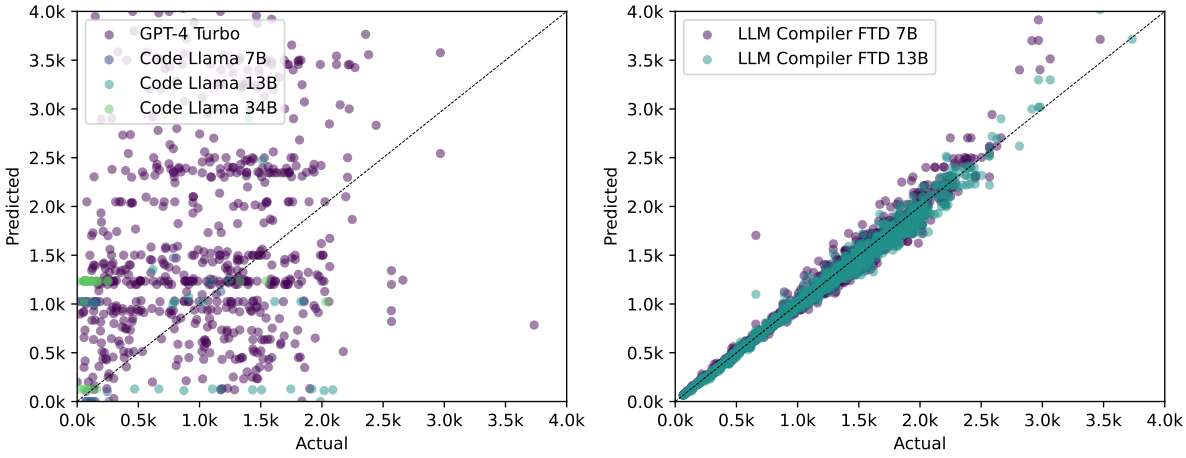


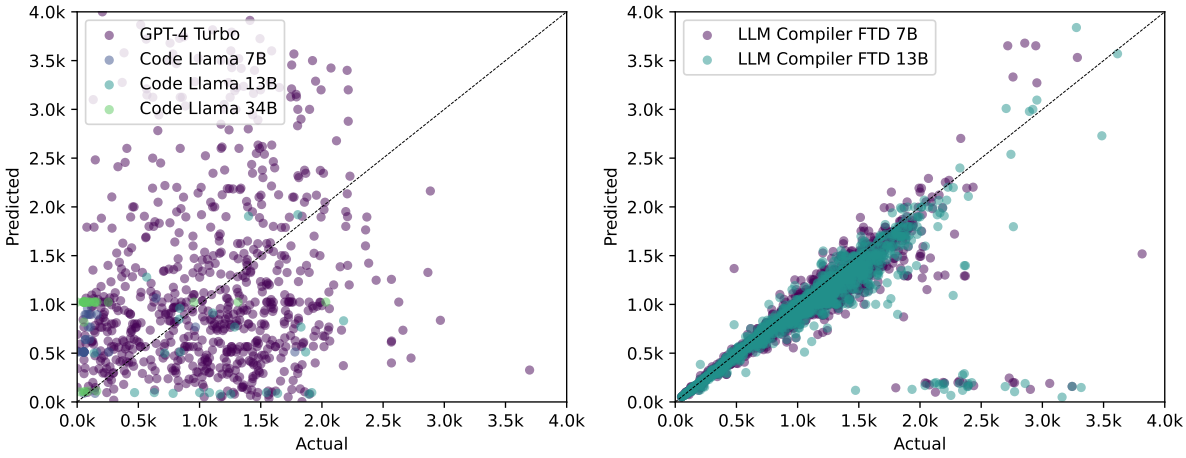
Figure 6: Improvement over -Oz for each of the benchmarks in MiBench.

Table 4: Ablating the LLM Compiler FTD training regime on the flag tuning task. All results are for 7B parameter models, evaluated on the same holdout validation set of 500 programs. The first row is the LLM Compiler FTD release. All other rows strip out successful components of the training regime.

Code Llama	IR & asm pretraining	Compiler emulation	Flag tuning	Disassembly	Mean improvement over -Oz	Mean improvement wrt. Autotuner
✓	✓	✓	✓	✓	5.12%	77%
✓	✓	✓	✓		**5.15%**	**78%**
✓	✓		✓		5.07%	76%
✓			✓		4.94%	75%
			✓		4.79%	72%
Autotuner					6.63%	100%



(a) Unoptimized binary size



(b) Optimized binary size

Figure 7: Accuracy of models at predicting code size before and after optimization. LLM Compiler FTD is most accurate at predicting code size before optimization than after optimization. Code Llama and GPT-4 Turbo, shown left, display little correlation between predicted and actual values.

Table 5: Model performance at disassembling 2,015 assembly codes taken from MiBench. We use *Round trips* to evaluate the capabilities of models, by taking the IR generated by the models and attempting to lower it back to assembly. *Round trips* shows the number of disassembled IRs that can be lowered back, *Round trip BLEU* compares the round-tripped assemblies against the originals, and *Round trip exact match* is the proportion of round-tripped assemblies that are exact character-for-character matches with the input, indicating lossless round-trip from assembly up to IR and back down again.

	Size	Round trips	Round trip BLEU	Round trip exact match
LLM Compiler FTD	7B	**936**	0.951	12.7%
	13B	905	**0.960**	**13.8%**
Code Llama - Instruct	7B	30	0.477	0.0%
	13B	53	0.615	0.0%
	34B	12	0.458	0.0%
GPT-4 Turbo (2024-04-09)	-	127	0.429	0.0%

Table 6: Ablating the LLM Compiler FTD training regime on code disassembly. All results are for 7B parameter model sizes, evaluated on a holdout validation set of 500 programs. Values in parentheses show relative performance to the first row (i.e. the LLM Compiler FTD release).

Code Llama	IR & asm pretraining	Compiler emulation	Flag tuning training	Disassembly	Round trips	Round trip BLEU
✓	✓	✓	✓	✓	**49.4% (-)**	0.951 (-)
✓	✓	✓		✓	45.2% (-8.5%)	0.955 (+0.4%)
✓	✓			✓	44.2% (-10.5%)	0.957 (+0.7%)
✓				✓	39.0% (-21.1%)	**0.965 (+1.5%)**
				✓	8.8% (-82.8%)	0.908 (-4.5%)

5.2 Disassembly task

Methodology. We evaluate the functional correctness of LLM-generated code when disassembling assembly code to LLVM-IR. As in Section 5.1 we evaluate LLM Compiler FTD and compare to Code Llama - Instruct and GPT-4 Turbo, and find that an additional prompt suffix, shown in Listing 8, is required to extract the best performance from these models. The suffix provides additional context about the task and the expected output format. To evaluate the performance of models we *round-trip* the model-generated disassembled IR back down to assembly. This enables us to evaluate accuracy of the disassembly by comparing the BLEU score (Papineni et al., 2002) of the original assembly against the round-trip result. A lossless and perfect disassembly from assembly to IR will have a round-trip BLEU score of 1.0 (*exact match*).

Dataset. We evaluate on 2,015 test prompts extracted from the MiBench benchmark suite. We took the 2,398 translation units used for the flag tuning evaluation above and generated disassembly prompts. We then filtered the prompts on a maximum 8k token length, allowing 8k tokens for the model output, leaving 2,015. Table 11 summarizes the benchmarks.

Results. Table 5 shows performance of the models on the disassembly task. LLM Compiler FTD 7B has a slightly higher round-trip success rate than LLM Compiler FTD 13B, but LLM Compiler FTD 13B has the highest accuracy of round-tripped assembly (*round trip BLEU*) and most frequently produces a perfect disassembly (*round trip exact match*). Code Llama - Instruct and GPT-4 Turbo struggle with generating syntactically correct LLVM-IR. Figure 8 shows the distribution of round-trip BLEU scores for all models.

Ablation studies. Table 6 ablates the performance of models on a small holdout validation set of 500 prompts taken from the MiBench dataset used previously. We trained for disassembly at each stage of the training pipeline from Figure 1 to compare performance. Round trip rate is highest when going through the whole stack of training data and drops consistently with every training stage, though round trip BLEU varies little with each stage.

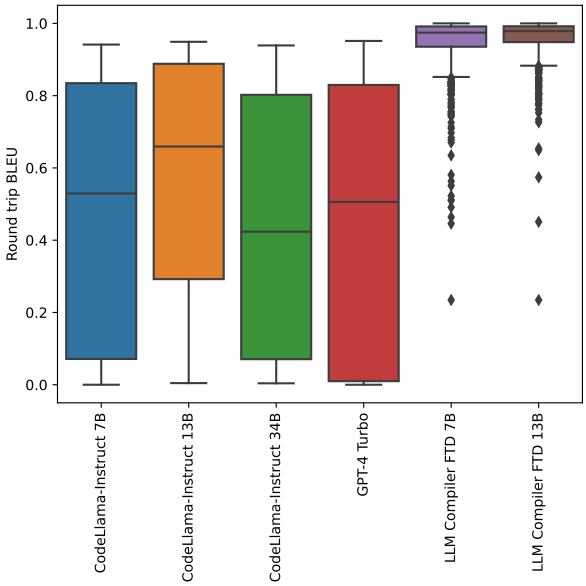


Figure 8: Distribution of round trip BLEU scores on the disassembly task.

5.3 Foundation model tasks

Methodology We ablate LLM Compiler models on the two foundation model tasks of next-token prediction and compiler emulation. We perform this evaluation at each stage of the training pipeline to see how training for each successive task affects performance. For next-token prediction we compute perplexity on a small sample of LLVM-IR and assembly code from all optimization levels. We evaluate compiler emulation using two metrics: whether the generated IR or assembly code compiles, and whether the generated IR or assembly code is an exact match for what the compiler would produce.

Dataset. For next-token prediction we use a small holdout set of validation data that is drawn from the same distribution as our training data but has not been used for training. We use a mixture of optimization levels including unoptimized code, code optimized with -Oz, and randomly generated pass lists. For compiler emulatino we evaluate using 500 prompts generated from MiBench using randomly pass lists generated in the manner described in Section 2.2.

Results Table 7 shows performance of LLM Compiler FTD across all training stages on the two foundation model training tasks of next-token prediction and compiler emulation. Next-token prediction performance jumps sharply after Code Llama, which has seen very little IR and assembly, and declines slightly with each subsequent stage of fine-tuning.

For compiler emulation, the Code Llama base model and the pre-trained models perform poorly since they have not been trained on this task. The highest performance is achieved directly after compiler emulation training where 95.6% of IR and assembly generated by LLM Compiler FTD 13B compiles, and 20% of it matches the compiler exactly. Performance declines after fine-tuning for flag tuning and disassembly.

5.4 Software engineering tasks

Methodology. While the purpose of LLM Compiler FTD is to provide foundation models for code optimization, it builds upon base Code Llama models which were trained for software engineering tasks. To evaluate how the additional training of LLM Compiler FTD has affected the performance of code generation we use the same benchmark suites as in Code Llama that evaluate the ability of LLMs to generate Python code from natural language prompts, such as *"Write a function to find the longest chain which can be formed from the given set of pairs."*.

Table 7: Performance at next-token prediction and compiler emulation tasks. For *Perplexity*, lower is better. For *Compiles* and *Exact match*, higher is better.

Code Llama	IR & asm pretraining	Compiler emulation	Flag tuning	Disassembly	Size	Perplexity		Compiler emulation	
						IR	Asm	Compiles	Exact match
✓					7B	1.456	1.423	5.4%	1.2%
					13B	1.429	1.404	4.8%	0.8%
✓	✓				7B	1.050	1.041	0.8%	0.0%
					13B	**1.045**	**1.038**	35.8%	2.8%
✓	✓	✓			7B	1.052	1.046	87.0%	16.0%
					13B	1.047	1.043	**95.6%**	**20.0%**
✓	✓	✓	✓		7B	1.058	1.051	55.0%	1.2%
					13B	1.052	1.048	58.6%	4.2%
✓	✓	✓	✓	✓	7B	1.057	1.053	71.0%	4.6%
					13B	1.054	1.052	61.4%	5.4%

Table 8: Performance on Python programming tasks. pass@1 are computed with greedy decoding. The pass@10 and pass@100 scores are computed with nucleus sampling with p=0.95 and temperature=0.6.

Code Llama	IR & asm pretraining	Compiler emulation	Flag tuning	Disassembly	Size	HumanEval			MBPP		
						pass@1	pass@10	pass@100	pass@1	pass@10	pass@100
✓					7B	32.9%	63.3%	85.3%	45.4%	67.5%	81.6%
					13B	**36.0%**	**71.9%**	**90.6%**	**48.4%**	**71.3%**	**83.9%**
✓	✓				7B	28.0%	58.6%	84.3%	42.8%	66.0%	80.0%
					13B	34.1%	68.0%	87.9%	47.6%	70.3%	83.3%
✓	✓	✓			7B	25.0%	51.3%	79.0%	37.4%	61.5%	75.6%
					13B	31.1%	62.9%	83.2%	46.0%	67.8%	80.9%
✓	✓	✓	✓		7B	24.4%	46.2%	73.1%	36.6%	58.5%	74.4%
					13B	29.3%	55.9%	81.1%	42.2%	63.6%	79.1%
✓	✓	✓	✓	✓	7B	26.8%	44.0%	65.3%	31.4%	55.1%	73.2%
					13B	25.6%	51.2%	76.8%	37.6%	60.6%	76.4%
Llama 2					7B	12.2%	25.2%	44.4%	20.8%	41.8%	65.5%
					13B	20.1%	34.8%	61.2%	27.6%	48.1%	69.5%

Datasets. We use the HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) benchmarks as in Code Llama.

Results. Table 8 shows the greedy decoding performance (*pass@1*) of all model training stages and model sizes starting at the Code Llama base model. It also shows the models' scores on *pass@10* and *pass@100* which were generated with p=0.95 and temperature=0.6. Each stage of compiler-centric training causes a slight regression in Python programming ability. pass@1 performance on HumanEval and MBPP declines by up to 18% and 5% for LLM Compiler and by up to 29% and 22% for LLM Compiler FTD after the additional flag tuning and disassembly fine-tuning. All models still outperform Llama 2 on both tasks.

6 Related work

Language models over code. There is increasing interest in LLMs for source code reasoning and generation (Jiang et al., 2024; Hou et al., 2023). The main enablers of progress in this area are pretrained foundational models made available for others to build upon, including Code Llama (Rozière et al., 2023), StarCoder (Lozhkov et al., 2024), Magicoder (Wei et al., 2024), DeepSeek-Coder (Guo et al., 2024), GPT-4 (OpenAI, 2023) and others (Wang et al.,

2023; Allal et al., 2023; Feng et al., 2020). Some of the existing models are open source (Rozière et al., 2023; Lozhkov et al., 2024; Wei et al., 2024; Allal et al., 2023) while others are closed source (Chen et al., 2021; OpenAI, 2023; Li et al., 2022; Gunasekar et al., 2023). We extend the collection of foundational models for code with a family of models specifically trained on intermediate code representation with a license that allows wide reuse.

Language models have been adapted to perform program fuzzing (Xia et al., 2023a; Deng et al., 2023), test generation (Schäfer et al., 2023), automated program repair (Xia et al., 2023b), and source-level algorithmic optimization Madaan et al. (2023). The introduction of fill-in-the-middle capabilities is especially useful for software engineering use cases such as code completion, and has become common in recent code models such as InCoder (Fried et al., 2023), SantaCoder (Allal et al., 2023), StarCoder (Lozhkov et al., 2024), and Code Llama (Rozière et al., 2023). A large number of useful applications have been explored for LLMs, however, only very few are directly focused on compilation tasks.

Language models over IR. While LLMs have found broad adoption for coding tasks, few operate at the level of compilers. Gallagher et al. (2022) train a RoBERTA architecture on LLVM-IR for the purpose of code weakness identification, and Transcoder-IR (Szafraniec et al., 2022) uses LLVM-IR as a pivot point for source-to-source translation. Few LLMs include compiler IRs in their training, and of those that do, IRs comprise a tiny fraction of the data compared to other programming languages. StarCoder 2 Lozhkov et al. (2024) and DeepSeek-Coder Guo et al. (2024) include 7.7 GB (0.4%) and 0.91 GB (0.1%) of LLVM-IR respectively in their training data. LLM Compiler is pretrained on 422 GB of LLVM-IR, and additional LLVM-IR during fine-tuning, and assembly code which makes up at least 85% of the total training data.

Paul et al. (2024) create SLTrans, a 26 B token dataset which pairs high level source code with corresponding LLVM-IR. Like our dataset, they include different source languages and optimization levels for their IR, however, their optimization is limited to *-Oz* and *-O3*. They train IRCoder on 800 M tokens of SLTrans and demonstrate how it improves the code reasoning capabilities of underlying base models. IRCoder and StarCoder 2 present their models with LLVM-IR. We include both LLVM-IR as well as native assembly code from multiple source languages and for multiple architecture targets.

With the increasing interest in IR to improve the performance of code generation models, new datasets are emerging. For example, ComPile (Grossman et al., 2024), a 2.4 TB dataset of unoptimized LLVM-IR.

Machine Learning in Compilers. Many works have applied machine learning in compilers (Leather & Cummins, 2020; Ashouri et al., 2022; Cummins et al., 2017; Phothilimthana et al., 2021; Seeker et al., 2024). Compiler pass ordering has been exploited for decades. Over the years there have been several approaches using machine learning (Liang et al., 2023; Agakov et al., 2006; Ogilvie et al., 2017; Jayatilaka et al., 2021; Queiroz Jr & da Silva, 2023; Grubisic et al., 2024a). Neural machine translation is an emerging field that uses language models to transform code from one language to another. Prior examples include compiling C to assembly (Armengol-Estapé & O'Boyle, 2021), assembly to C (Armengol-Estapé et al., 2024; Hosseini & Dolan-Gavitt, 2022), and source-to-source (Lachaux et al., 2020).

7 Discussion

In this paper, we introduced LLM Compiler, a novel family of large language models specifically designed to address the challenges of code and compiler optimization. By extending the capabilities of the foundational Code Llama model, LLM Compiler provides a robust, pre-trained platform that significantly enhances the understanding and manipulation of compiler intermediate representations and assembly language.

We release LLM Compiler under a bespoke commercial license to facilitate widespread access and collaboration, enabling both academic researchers and industry practitioners to explore, modify, and extend the model according to their specific needs.

7.1 Limitations

We have shown that LLM Compiler performs well at compiler optimization tasks and has improved understanding of compiler representations and assembly code over prior works, but there are limitations. The main limitation is the finite sequence length of inputs (context window). LLM Compiler supports a 16k token context windows, but program codes may be far longer. For example, 67% of MiBench translation units exceeded this context window when formatted as flag tuning prompts, shown in Table 10. To mitigate this we split larger translation units into individual functions, though this limits the scope of optimization that can be performed, and still 18% of the split

translation units remain too large for the model to accept as input. Researchers are adopting ever-increasing context windows (Ding et al., 2023), but finite context windows remain a common concern with LLMs.

A second limitation, common to all LLMs, is the accuracy of model outputs. Users of LLM Compiler are advised to assess their models using evaluation benchmarks specific to compilers. Given that compilers are not bug-free, any suggested compiler optimizations must be rigorously tested. When a model decompiles assembly code, its accuracy should be confirmed through round trip, manual inspection, or unit testing. For some applications LLM generations can be constrained to regular expressions (Grubisic et al., 2024b), or combined with automatic verification to ensure correctness (Taneja et al., 2024).

B Prompts

B.1 Compiler emulation prompts

Prompt:

```
[INST] Give the LLVM-IR for the following code when optimized using opt -p 'module(default<Oz>)':

<code>; ModuleID = '<stdin>'
source_filename = "-"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

; Function Attrs: minsize nounwind optsize uwtable
define dso_local i32 @add_two(i32 noundef %0, i32 noundef %1) #0 {
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  store i32 %0, ptr %3, align 4, !tbaa !5
  store i32 %1, ptr %4, align 4, !tbaa !5
  %5 = load i32, ptr %3, align 4, !tbaa !5
  %6 = load i32, ptr %4, align 4, !tbaa !5
  %7 = add nsw i32 %5, %6
  ret i32 %7
}

attributes #0 = { minsize nounwind optsize uwtable "min-legal-vector-width"="0" "no-trapping-math"="true"
 ↪ "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse
 ↪,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3}
!llvm.ident = !{!4}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{!"clang version 17.0.6 (git@github.com:fairinternal/CodeGen.git
 ↪b05db9bbf7a92019267416c1bb9996fe6134e3f1)"}
!5 = !{!6, !6, i64 0}
!6 = !{!"int", !7, i64 0}
!7 = !{!"omnipotent char", !8, i64 0}
!8 = !{!"Simple C/C++ TBAA"}
</code>

The input code has instruction count 8 and binary size 65 bytes. [/INST]
```

Answer:

```
The LLVM-IR will have instruction count 2 and binary sise 53 bytes:

<code>; ModuleID = '<stdin>'
source_filename = "-"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

; Function Attrs: minsize mustprogress nofree norecurse nosync nounwind optsize willreturn memory(none)
 ↪uwtable
define dso_local i32 @add_two(i32 noundef %0, i32 noundef %1) local_unnamed_addr #0 {
  %3 = add nsw i32 %1, %0
  ret i32 %3
}

attributes #0 = { minsize mustprogress nofree norecurse nosync nounwind optsize willreturn memory(none)
 ↪uwtable "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "
 ↪target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3}
!llvm.ident = !{!4}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{!"clang version 17.0.6 (git@github.com:fairinternal/CodeGen.git
 ↪b05db9bbf7a92019267416c1bb9996fe6134e3f1)"}
</code>
```

Listing 2: Prompt used for $IR \rightarrow IR$ compiler emulation.

Prompt:

```
[INST] Give the assembly for the following code when optimized using opt -p 'module(default<Oz>)':

<code>; ModuleID = '<stdin>'
source_filename = "-"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

; Function Attrs: minsize nounwind optsize uwtable
define dso_local i32 @add_two(i32 noundef %0, i32 noundef %1) #0 {
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  store i32 %0, ptr %3, align 4, !tbaa !5
  store i32 %1, ptr %4, align 4, !tbaa !5
  %5 = load i32, ptr %3, align 4, !tbaa !5
  %6 = load i32, ptr %4, align 4, !tbaa !5
  %7 = add nsw i32 %5, %6
  ret i32 %7
}

attributes #0 = { minsize nounwind optsize uwtable "min-legal-vector-width"="0" "no-trapping-math"="true"
 ↪ "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse
 ↪,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3}
!llvm.ident = !{!4}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{!"clang version 17.0.6 (git@github.com:fairinternal/CodeGen.git
 ↪b05db9bbf7a92019267416c1bb9996fe6134e3f1)"}
!5 = !{!6, !6, i64 0}
!6 = !{!"int", !7, i64 0}
!7 = !{!"omnipotent char", !8, i64 0}
!8 = !{!"Simple C/C++ TBAA"}
</code>

The input code has instruction count 8 and binary size 65 bytes. [/INST]
```

Answer:

```
The assembly will have instruction count 2 and binary sise 53 bytes:

<code>  .text
  .file "-"
  .globl  add_two                         # -- Begin function add_two
  .type add_two,@function
add_two:                                # @add_two
  .cfi_startproc
# %bb.0:
  movl  %esi, %eax
  addl  %edi, %eax
  retq
.Lfunc_end0:
  .size add_two, .Lfunc_end0-add_two
  .cfi_endproc
                                        # -- End function
  .ident  "clang version 17.0.6 (git@github.com:fairinternal/CodeGen.git
   ↪b05db9bbf7a92019267416c1bb9996fe6134e3f1)"
  .section  ".note.GNU-stack","",@progbits
  .addrsig
</code>
```

Listing 3: Prompt used for *IR* → *assembly* compiler emulation.

B.2 Flag tuning prompts

Prompt:

```
[INST] Tell me how to optimize this LLVM-IR for object file size:

<code>; ModuleID = '<stdin>'
source_filename = "-"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

; Function Attrs: minsize nounwind optsize uwtable
define dso_local i32 @add_two(i32 noundef %0, i32 noundef %1) #0 {
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  store i32 %0, ptr %3, align 4, !tbaa !5
  store i32 %1, ptr %4, align 4, !tbaa !5
  %5 = load i32, ptr %3, align 4, !tbaa !5
  %6 = load i32, ptr %4, align 4, !tbaa !5
  %7 = add nsw i32 %5, %6
  ret i32 %7
}

attributes #0 = { minsize nounwind optsize uwtable "min-legal-vector-width"="0" "no-trapping-math"="true"
 ↪ "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse
 ↪,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3}
!llvm.ident = !{!4}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{!"clang version 17.0.6 (git@github.com:fairinternal/CodeGen.git
 ↪b05db9bbf7a92019267416c1bb9996fe6134e3f1)"}
!5 = !{!6, !6, i64 0}
!6 = !{!"int", !7, i64 0}
!7 = !{!"omnipotent char", !8, i64 0}
!8 = !{!"Simple C/C++ TBAA"}</code> [/INST]
```

Answer:

```
The code has object file size of 65 which can be reduced to 53 by running:

‘opt -p 'module(default<Oz>)'‘

This will produce code:

<code>; ModuleID = '<stdin>'
source_filename = "-"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

; Function Attrs: minsize mustprogress nofree norecurse nosync nounwind optsize willreturn memory(none)
 ↪uwtable
define dso_local i32 @add_two(i32 noundef %0, i32 noundef %1) local_unnamed_addr #0 {
  %3 = add nsw i32 %1, %0
  ret i32 %3
}

attributes #0 = { minsize mustprogress nofree norecurse nosync nounwind optsize willreturn memory(none)
 ↪uwtable "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "
 ↪target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3}
!llvm.ident = !{!4}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{!"clang version 17.0.6 (git@github.com:fairinternal/CodeGen.git
 ↪b05db9bbf7a92019267416c1bb9996fe6134e3f1)"}</code>
```

Listing 4: Prompt used for flag tuning.

Prompt:

```
[INST] Tell me how to optimize this LLVM-IR for object file size:

<code>; ModuleID = '<stdin>'
source_filename = "-"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

; Function Attrs: minsize mustprogress nofree norecurse nosync nounwind optsize willreturn memory(none)
 ↪uwtable
define dso_local i32 @add_two(i32 noundef %0, i32 noundef %1) local_unnamed_addr #0 {
  %3 = add nsw i32 %1, %0
  ret i32 %3
}

attributes #0 = { minsize mustprogress nofree norecurse nosync nounwind optsize willreturn memory(none)
 ↪uwtable "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-protector-buffer-size"="8" "
 ↪target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3}
!llvm.ident = !{!4}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{!"clang version 17.0.6 (git@github.com:fairinternal/CodeGen.git
 ↪b05db9bbf7a92019267416c1bb9996fe6134e3f1)"}</code> [/INST]
```

Answer:

```
The code has object file size of 53 which cannot be reduced further
```

Listing 5: Prompt and answer format for flag tuning task when no improvement can be made.

```
Provide your answer as a list of command line options to opt version 17.0.6, using the format:

"$ opt -p '<passes>'"

Where <passes> is a list of passes for the new pass manager, e.g. "function(dce),module(default<Oz>),
 ↪function(load-store-vectorizer)".

Only include the passes list. Do not include file paths or other flags such as -o. Terminate the opt
 ↪command line options with a newline.

Then report the optimized code that will be produced, delimited by <code> and </code> tags.

Finally, report the binary size the code before and after optimization using the template:

"Before optimization: X bytes. After optimization: Y bytes."

Where X and Y are placeholders for integer binary sizes in bytes. Binary size is the summation of the .
 ↪text and .data segment sizes of the object file generated by `clang-17 output.bc -c`, as reported by
 ↪the `size` tool.

Include no other text in your response.
```

Listing 6: Additional suffix appended to flag tuning prompts when evaluating GPT-4 and Code Llama - Instruct. We found that this additional context helped improve the coherence of model responses.

B.3 Disassembly prompts

Prompt:

```
[INST] Disassemble this code to LLVM-IR:

<code>  .text
  .file "-"
  .globl  add_two                         # -- Begin function add_two
  .type add_two,@function
add_two:                                  # @add_two
  .cfi_startproc
# %bb.0:
  movl  %edi, -4(%rsp)
  movl  %esi, -8(%rsp)
  movl  -4(%rsp), %eax
  addl  -8(%rsp), %eax
  retq
.Lfunc_end0:
  .size add_two, .Lfunc_end0-add_two
  .cfi_endproc
                                          # -- End function
  .ident  "clang version 17.0.6 (git@github.com:fairinternal/CodeGen.git
   ↪b05db9bbf7a92019267416c1bb9996fe6134e3f1)"
  .section  ".note.GNU-stack","",@progbits
  .addrsig
</code> [/INST]
```

Answer:

```
<code>; ModuleID = '<stdin>'
source_filename = "-"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

; Function Attrs: minsize nounwind optsize uwtable
define dso_local i32 @add_two(i32 noundef %0, i32 noundef %1) #0 {
  %3 = alloca i32, align 4
  %4 = alloca i32, align 4
  store i32 %0, ptr %3, align 4, !tbaa !5
  store i32 %1, ptr %4, align 4, !tbaa !5
  %5 = load i32, ptr %3, align 4, !tbaa !5
  %6 = load i32, ptr %4, align 4, !tbaa !5
  %7 = add nsw i32 %5, %6
  ret i32 %7
}

attributes #0 = { minsize nounwind optsize uwtable "min-legal-vector-width"="0" "no-trapping-math"="true"
 ↪ "stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="+cmov,+cx8,+fxsr,+mmx,+sse
 ↪,+sse2,+x87" "tune-cpu"="generic" }

!llvm.module.flags = !{!0, !1, !2, !3}
!llvm.ident = !{!4}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{!"clang version 17.0.6 (git@github.com:fairinternal/CodeGen.git
 ↪b05db9bbf7a92019267416c1bb9996fe6134e3f1)"}
!5 = !{!6, !6, i64 0}
!6 = !{!"int", !7, i64 0}
!7 = !{!"omnipotent char", !8, i64 0}
!8 = !{!"Simple C/C++ TBAA"}
</code>
```

Listing 7: Prompt used for disassembly.

```
Use LLVM version 17.0.6. Provide the IR enclosed by <code> and </code> tags.

Include no other text.
```

Listing 8: Additional suffix appended to disassembly prompts when evaluating GPT-4 and Code Llama - Instruct. We found that this additional context helped improve the coherence of model responses.

C Optimization Passes

Table 9: List of *opt* 17.0.6 transformation passes used to generate data for code optimization self-training and flag-tuning tasks.

Pass Name	Level	Pass Name	Level	Pass Name	Level
O0	Module	no-op-cgscc	CGSCC	lcssa	Function
O1	Module	inline	CGSCC	loop-data-prefetch	Function
O2	Module	coro-split	CGSCC	loop-load-elim	Function
O3	Module	function-attrs	CGSCC	loop-fusion	Function
Os	Module	aa-eval	Function	loop-distribute	Function
Oz	Module	adce	Function	loop-versioning	Function
always-inline	Module	add-discriminators	Function	pa-eval	Function
attributor	Module	aggressive-instcombine	Function	place-safepoints	Function
annotation2metadata	Module	assume-builder	Function	reassociate	Function
openmp-opt	Module	assume-simplify	Function	redundant-dbg-inst-elim	Function
openmp-opt-postlink	Module	alignment-from-assumptions	Function	reg2mem	Function
called-value-propagation	Module	annotation-remarks	Function	scalarize-masked-mem-intrin	Function
canonicalize-aliases	Module	bdce	Function	scalarizer	Function
constmerge	Module	break-crit-edges	Function	separate-const-offset-from-gep	Function
coro-early	Module	callsite-splitting	Function	sccp	Function
coro-cleanup	Module	consthoist	Function	sink	Function
cross-dso-cfi	Module	count-visits	Function	slp-vectorizer	Function
deadargelim	Module	constraint-elimination	Function	slsr	Function
elim-avail-extern	Module	chr	Function	speculative-execution	Function
extract-blocks	Module	coro-elide	Function	strip-gc-relocates	Function
forceattrs	Module	correlated-propagation	Function	tailcallelim	Function
globalopt	Module	dce	Function	vector-combine	Function
globalsplit	Module	dfa-jump-threading	Function	tlshoist	Function
hotcoldsplit	Module	div-rem-pairs	Function	declare-to-assign	Function
inferattrs	Module	dse	Function	early-cse	Function
inliner-wrapper	Module	fix-irreducible	Function	ee-instrument	Function
inliner-wrapper-no-mandatory-first	Module	flattencfg	Function	hardware-loops	Function
iroutliner	Module	make-guards-explicit	Function	lower-matrix-intrinsics	Function
lower-global-dtors	Module	gvn-hoist	Function	loop-unroll	Function
lower-ifunc	Module	gvn-sink	Function	simplifycfg	Function
lowertypetests	Module	infer-address-spaces	Function	loop-vectorize	Function
mergefunc	Module	instcombine	Function	instcombine	Function
name-anon-globals	Module	instsimplify	Function	mldst-motion	Function
partial-inliner	Module	irce	Function	gvn	Function
recompute-globalsaa	Module	float2int	Function	sroa	Function
rel-lookup-table-converter	Module	libcalls-shrinkwrap	Function	loop-flatten	Loop
rewrite-statepoints-for-gc	Module	inject-tli-mappings	Function	loop-interchange	Loop
rewrite-symbols	Module	instnamer	Function	loop-unroll-and-jam	Loop
rpo-function-attrs	Module	lower-expect	Function	canon-freeze	Loop
scc-oz-module-inliner	Module	lower-guard-intrinsic	Function	loop-idiom	Loop
strip	Module	lower-constant-intrinsics	Function	loop-instsimplify	Loop
strip-dead-debug-info	Module	lower-widenable-condition	Function	loop-deletion	Loop
strip-dead-prototypes	Module	guard-widening	Function	loop-simplifycfg	Loop
strip-debug-declare	Module	load-store-vectorizer	Function	loop-reduce	Loop
strip-nondebug	Module	loop-simplify	Function	indvars	Loop
strip-nonlinetable-debuginfo	Module	loop-sink	Function	loop-unroll-full	Loop
synthetic-counts-propagation	Module	lowerswitch	Function	loop-predication	Loop
wholeprogramdevirt	Module	mem2reg	Function	guard-widening	Loop
module-inline	Module	memcpyopt	Function	loop-bound-split	Loop
pseudo-probe-update	Module	mergeicmps	Function	loop-reroll	Loop
globaldce	Module	mergereturn	Function	loop-versioning-licm	Loop
ipsccp	Module	move-auto-init	Function	simple-loop-unswitch	Loop
embed-bitcode	Module	nary-reassociate	Function	loop-rotate	Loop
argpromotion	CGSCC	newgvn	Function	licm	LoopMssa
attributor-cgscc	CGSCC	jump-threading	Function	lnicm	LoopMssa
openmp-opt-cgscc	CGSCC	partially-inline-libcalls	Function		

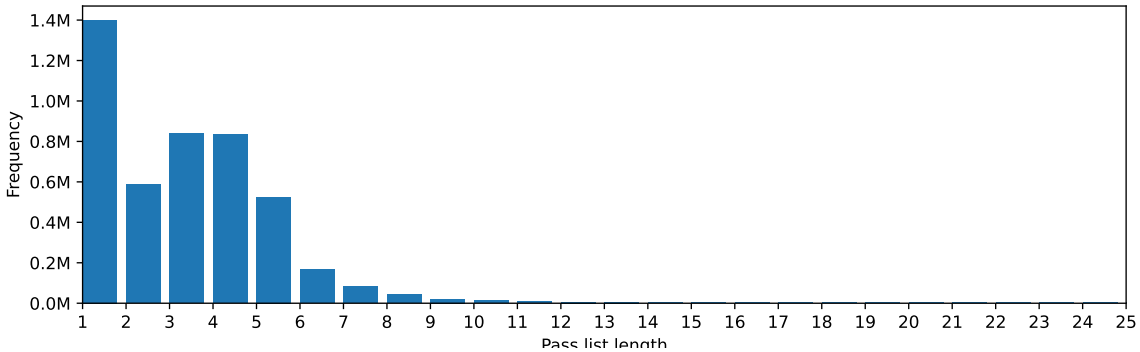


Figure 9: Length of autotuned pass lists.

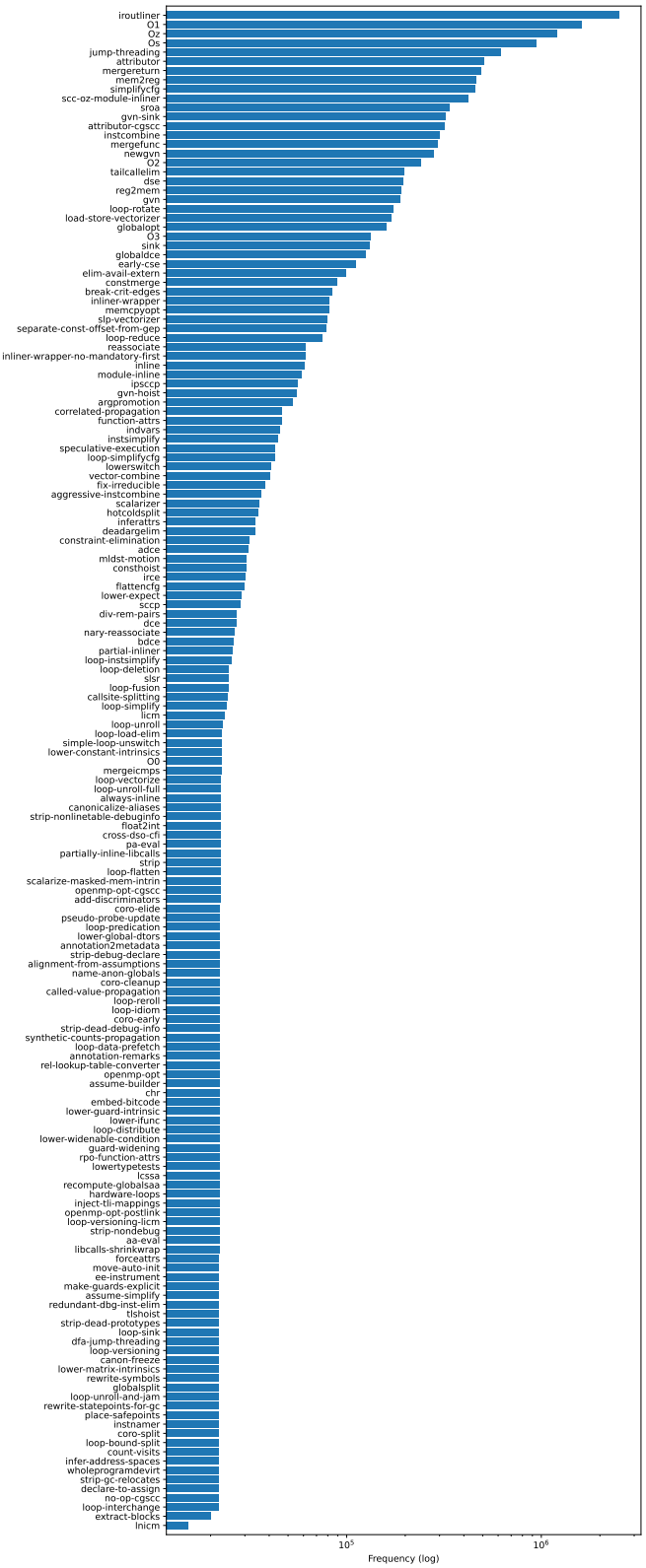


Figure 10: The frequency of passes in the best pass lists generated by the autotuner on our training programs.

D Benchmarks

Table 10: MiBench benchmarks used for flag tuning task evaluation.

		Without split		With split	
	Binary size	Translation units	Truncated prompts	Translation units	Truncated prompts
adpcm	816.7 kB	2		2	
basicmath	931.7 kB	4		4	
bitcount	821.1 kB	8		8	
blowfish	830.6 kB	7	3 (43%)	7	2 (29%)
crc32	818.4 kB	1		1	
dijkstra	946.0 kB	1		1	
fft	844.8 kB	3		3	
ghostscript	1.9 MB	296	222 (75%)	1,052	162 (15%)
gsm	58.8 kB	23	12 (52%)	37	10 (27%)
ispell	91.5 kB	12	8 (67%)	39	6 (15%)
jpeg_c	112.5 kB	54	39 (72%)	170	22 (13%)
jpeg_d	151.7 kB	54	39 (72%)	164	18 (11%)
lame	289.2 kB	32	22 (69%)	92	24 (26%)
patricia	949.3 kB	2	1 (50%)	3	
qsort	944.3 kB	1		1	
rsynth	151.4 kB	19	10 (53%)	27	3 (11%)
sha	5.3 kB	2	1 (50%)	3	
stringsearch	821.5 kB	4		4	
susan	911.4 kB	1	1 (100%)	13	7 (54%)
tiff2bw	442.1 kB	34	19 (56%)	134	24 (18%)
tiff2rgba	492.7 kB	34	19 (56%)	134	23 (17%)
tiffdither	441.2 kB	34	19 (56%)	133	23 (17%)
tiffmedian	453.0 kB	34	19 (56%)	139	26 (19%)
typeset	2.0 MB	51	43 (84%)	227	89 (39%)
Total		713	477 (67%)	2,398	439 (18%)

Table 11: MiBench benchmarks used for disassembly task evaluation.

	Translation units	Truncated prompts
adpcm	3	
basicmath	2	
bitcount	8	
blowfish	3	
crc32	1	
dijkstra	2	
fft	1	
ghostscript	1,264	2
gsm	35	
ispell	45	
jpeg_c	24	
jpeg_d	177	
lame	87	1
patricia	3	
qsort	1	
rsynth	33	1
sha	3	
stringsearch	5	
susan	7	
tiff2bw	3	
tiff2rgba	5	
tiffdither	2	
tiffmedian	158	
typeset	143	
Total	2015	4

E Model card

Model details	
Model Developers	Meta AI
Variations	LLM Compiler comes in two model sizes: 7B and 13B parameters. Both variations have been trained on the same data. LLM Compiler FTD, available in the same sizes, extends these with further training.
Input	Models input text only.
Output	Models output text only.
Model Architecture	LLM Compiler and its variants are autoregressive language models using optimized transformer architectures. All models were fine-tuned with up to 16K tokens.
Model Dates	LLM Compiler and its variants have been trained between January and May 2024.
Status	This is a static model trained on an offline dataset.
Licence	A custom commercial license is available at: `ai.meta.com/resources/models-and-libraries/llama-downloads/`.
Where to send comments	Instructions on how to provide feedback or comments on the model can be found in the model README.
Intended Use	
Intended Use Cases	LLM Compiler and its variants are intended for commercial and research use in English and relevant programming languages. The foundation model LLM Compiler can be adapted for a variety of code optimization and understanding tasks.
Out-of-Scope Uses	Use in any manner that violates applicable laws or regulations (including trade compliance laws). Use in languages other than English. Use in any other way that is prohibited by the Acceptable Use Policy and Licensing Agreement for LLM Compiler and its variants.
Hardware and Software	
Training Factors	We used custom training libraries. The training and fine-tuning of the released models have been performed on Meta's Research Super Cluster.
Carbon Footprint	In aggregate, training all 4 LLM Compiler models required 264K GPU hours of computation on hardware of type A100-80GB (TDP of 350-400W). Estimated total emissions were 64.12 tCO2eq, 100% of which were offset by Meta's sustainability program.
Training Data	
All experiments reported here and the released models have been trained and fine-tuned using the same data as Code Llama with different weights (see Section 2 and Table 1).	
Evaluation Results	
See evaluations for the main models and detailed ablations Section 5.	
Ethical Considerations and Limitations	
LLM Compiler and its variants are a new technology that carries risks with use. Testing conducted to date has been in English, and has not covered, nor could it cover all scenarios. For these reasons, as with all LLMs, LLM Compiler 's potential outputs cannot be predicted in advance, and the model may in some instances produce inaccurate or objectionable responses to user prompts. Therefore, before deploying any applications of LLM Compiler, developers should perform safety testing and tuning tailored to their specific applications of the model. Please see the Responsible Use Guide available available at `https://ai.meta.com/llama/responsible-user-guide`.	

Table 12: Model card (Mitchell et al., 2019) for LLM Compiler and LLM Compiler FTD.

On the advantages of using relative phase Toffolis with an application to multiple control Toffoli optimization

Dmitri Maslov[1,*]

[1] *National Science Foundation, Arlington, Virginia, USA*

Various implementations of the Toffoli gate up to a relative phase have been known for years. The advantage over regular Toffoli gate is their smaller circuit size. However, their use has been often limited to a demonstration of quantum control in designs such as those where the Toffoli gate is being applied last or otherwise for some specific reasons the relative phase does not matter. It was commonly believed that the relative phase deviations would prevent the relative phase Toffolis from being very helpful in practical large-scale designs.

In this paper, we report three circuit identities that provide the means for replacing certain configurations of the multiple control Toffoli gates with their simpler relative phase implementations, up to a selectable unitary on certain qubits, and without changing the overall functionality. We illustrate the advantage via applying those identities to the optimization of the known circuits implementing multiple control Toffoli gates, and report the reductions in the CNOT-count, T-count, as well as the number of ancillae used. We suggest that a further study of the relative phase Toffoli implementations and their use may yield other optimizations.

PACS numbers: 03.67.Lx, 03.67.Ac

I. INTRODUCTION

Multiple control Toffoli gates are the staple of quantum arithmetic and reversible circuits. They are employed widely within quantum algorithms, including in reversible transformations, such as arithmetic circuits and all sorts of Boolean operations over quantum registers, as well as subroutines within other specialized quantum transforms. Unfortunately, multiple control Toffoli gates are not simple operations, and require to be implemented using a certain library of elementary gates—physically attainable transformations for physical-level implementations, and fault-tolerant gates on the logical level. As of the time of this writing, most advanced and developed trapped ions [1] and superconducting [2] quantum information processing approaches allow computations over at most a few dozen qubits using at most a few dozen two-qubit gates. The smallest of the multiple control Toffoli gates, the three-qubit Toffoli gate, requires six CNOT gates as a physical-level circuit over controlling apparatus allowing the application of the CNOT and arbitrary single qubit gates, and seven T gates, as a logical fault-tolerant circuit over Clifford+T library without ancillae. The known implementations of larger multiple control Toffoli gates come at a substantially higher cost. This makes the multiple control Toffoli gates be expensive computing primitives. As such, the ability to replace them with their simpler counterparts that nevertheless can guarantee the overall functional integrity, as well as their optimization (multiple control Toffoli gates are implemented using smaller size multiple control Toffolis [3, 4]) are important in practice. Ultimately, the difficulty of implementing Toffoli gates may even be a deciding factor in the ability to run an experiment of a desired size. Indeed, consider a scenario where only a fixed number of certain elementary gates can be applied. Imagine the goal is to run a discrete logarithm type computation [4]. Since circuits implementing such an algorithm are dominated by reversible arithmetic operations, which in turn rely on the Toffoli gates, it is conceivable that optimizing Toffoli implementations would yield a resource count that is possible to execute for a desired size computation. Multiple control Toffoli gates are, of course, important beyond just the discrete logarithm type algorithms.

The goal of this paper is to provide a framework for replacing multiple control Toffoli gates with their simpler relative phase implementations. The advantage is illustrated through an optimization of the implementations of the multiple control Toffoli gates. The reported optimization is viewed as a motivating example rather than a complete and finished study. An in-depth look at the implementations of the relative phase multiple control Toffoli gates and their use in the optimization of arbitrary quantum circuits may likely yield more results.

To draw a classical analogy, relative phase Toffoli gates may turn out to play a role analogous to the classical NAND gates: while classical (quantum/reversible) circuits are designed using a convenient for a human set of operations (multiple control Toffolis), a compiler may decompose those into NAND gates (relative phase multiple control Toffolis) before they are mapped into lowest-level transistors (elementary quantum gates).

* mailto:dmitri.maslov@gmail.com; Joint Center for Quantum Information and Computer Science, University of Maryland, College Park, MD, USA

II. DEFINITIONS

In this paper, we will work with pure n-qubit quantum states $\sum_{i=0}^{2^n-1} \alpha_i |i\rangle$ and quantum transformations described by the $2^n \times 2^n$ unitary matrices U. Recall that a square matrix U is called unitary if its inverse equals to its conjugate transpose, $U^{-1} = U^\dagger$. While the property of unitarity defines evolutions that are possible to attain physically, it does not prescribe which ones may be implemented directly. To assist with the presentation of the material, we will discretize the family of transformations that may be obtained physically, and call them elementary quantum gates. This does not limit the applicability of the results—indeed, discrete circuits may be thought of as certain versions of continuous Hamiltonians, but are otherwise easier to work with. In particular, in this work we will rely on the following elementary gates: Pauli-X, $X = NOT = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, Pauli-$Z$, $Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, and its roots Phase, $P = \sqrt{Z} = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$, $T = \sqrt[4]{Z} = \begin{pmatrix} 1 & 0 \\ 0 & \frac{1+i}{\sqrt{2}} \end{pmatrix}$, and Pauli-$Y$, $Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$. A fourth root of Y will be mentioned in some constructions, in the form of $R_Y(\pi/4)$, that is equivalent to the fourth root of Y up to a global phase. Recall that $R_Y(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}$. Finally, for completeness we will need the Hadamard gate, $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, and the two-qubit CNOT gate that we introduce via the mapping of kets, rather than the 4×4 matrix, as $\text{CNOT}(a, b) : |a, b\rangle \mapsto |a, b \oplus a\rangle$, and everywhere else by linearity, due to the simplicity of such a definition.

Quantum circuits are defined as the strings of quantum gates, or otherwise products of matrices that correspond to the individual gates. For multiple qubit circuit computations via matrices, a proper Kronecker product needs to be taken to compute matrix products. For example, a two-qubit operation corresponding to the Hadamard gate on the first qubit is given by the matrix $H \otimes Id$, where Id is the identity applied to the second qubit. Recall that the product of matrices is taken in reverse order with respect to the order of gates in the corresponding circuit. Following the standard notations, circuits/unitaries composed of quantum gates/matrices X, Y, Z, P, H, and CNOT are called Clifford. These unitaries play an important role in quantum error correction, but are not complete (moreover, simulable classically with a polynomial size effort) for quantum computation. As such, for completeness, a circuit library needs to contain a non-Clifford gate, such as the T gate. The addition of any non-Clifford gate to the Clifford circuits furthermore turns out to result in the computational universality [4].

The above is meant to be a quick reminder of some basic facts and an introduction of the notations used in this paper. For an in-depth review we refer the reader to [4].

For convenience, we furthermore use the following notations: for a set of variables/qubits $X = \{x_1, x_2, ..., x_n\}$, $|X|$ equals n, being the number of individual qubits in this set, and the conjugation (Boolean AND) of variables, $x_1 \& x_2 \& ... \& x_n$ is denoted as simply x. When the number of variables in the set X is zero, we assign x the value of 1. When the set of variables X consists of a single element, $\{x\}$, the conjugation of the variables within the set, as well as the name of the variable, coincide; this does not however cause any issues.

We next define the multiple control Toffoli gates.

Definition 1. A *multiple control Toffoli gate* over a set of n qubits with the set $X = \{x_1, x_2, ..., x_{n-1}\}$ being the controls, and qubit y being the target, $TOF^n(X; y)$, is defined as the matrix

$$diag\left\{1, 1, ..., 1, \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\right\}.$$

We will sometimes omit the superscript and write $TOF(X; y)$ when the controls and the target are explicitly specified and the size of the multiple control Toffoli gate can thus be restored. Similarly, we may omit the specification of the qubits the gate operates on and write TOF^n when we are only concerned with the size of the gate. Finally, we may write TOF when the goal is to specify the kind of gate being the Toffoli and distinguish it from other kinds of gates. Observe, that when $|X| = 0$ the above definition reports the Pauli-X (NOT) gate, for $|X| = 1$ the definition introduces the CNOT gate, when $|X| = 2$, it reduces to the usual Toffoli gate TOF^3, and for larger sets X, the multiply-controlled Toffolis—Toffoli-4, Toffoli-5, *etc.*

An alternate definition of the multiple control Toffoli gate may cast it in the form of the mapping of kets, as follows, $TOF^n(X; y) : |X; y\rangle \mapsto |X, y \oplus x\rangle$. In some cases, the mapping of kets may be easier to operate with than the corresponding unitary matrix.

In our constructions, relative phase implementations of quantum unitary transformations play a major role. For the purpose of this work, we define relative phase implementations as follows.

Definition 2. A *relative phase* version of a quantum n-qubit unitary operation $U = \{u_{i,j}\}|_{i,j=0..2^n-1}$ is any n-qubit unitary $V = \{v_{i,j}\}|_{i,j=0..2^n-1}$ such that $|v_{i,j}| = |u_{i,j}|$ for all i and j.

In other words, a relative phase version or otherwise implementation of a unitary U is a unitary V such that the elements of the two matrices differ by $e^{i\pi\phi}$, where $\phi \in \mathbb{R}$, and ϕ may be different for different matrix elements. Observe that $e^{i\pi\phi}0 = 0$, therefore relative phase versions of unitaries have zeroes everywhere the original unitary does.

To illustrate, a relative phase multiple control Toffoli gate over the set of controls $X = \{x_1, x_2, ..., x_{n-1}\}$ with

the target y, $RTOF(X;y)$, can be written as follows,

$$diag\left\{z_0, z_1, ..., z_{2^n-3}, \begin{pmatrix} 0 & z_{2^n-2} \\ z_{2^n-1} & 0 \end{pmatrix}\right\},$$

where z_i are arbitrary length-1 complex numbers. Prefix "R" is used to distinguish the relative phase version from the multiple control Toffoli gate itself. Observe that when all $z_i = 1$, the respective relative phase Toffoli gate $RTOF(X;y)$ becomes the multiple control Toffoli gate $TOF(X;y)$, and when all z_i take the same but fixed value z, the respective relative phase Toffoli gate $RTOF(X;y)$ implements the multiple control Toffoli gate $TOF(X;y)$ up to an undetectable global phase z.

A relative phase multiple control Toffoli gate $RTOF^n$ may be thought of as a product of the multiple control Toffoli gate TOF^n and an n-qubit diagonal unitary D^n. Indeed, for a diagonal unitary $D^n := diag\{z_0, z_1, ..., z_{2^n-1}\}$ circuit TOF^nD^n implements a generic relative phase multiple control Toffoli gate $R_1TOF^n = diag\left\{z_0, z_1, ..., z_{2^n-3}, \begin{pmatrix} 0 & z_{2^n-2} \\ z_{2^n-1} & 0 \end{pmatrix}\right\}$, whereas circuit D^nTOF^n implements a generic relative phase multiple control Toffoli gate $R_2TOF^n = diag\left\{z_0, z_1, ..., z_{2^n-3}, \begin{pmatrix} 0 & z_{2^n-1} \\ z_{2^n-2} & 0 \end{pmatrix}\right\}$. Observe how both gates are relative phase multiple control Toffoli gates, but different in the last two non-zero elements, that are being permuted. We will exploit this property in the circuit diagrams. In particular, of the two possible decompositions of the relative phase multiple control Toffoli gate into a product of the multiple control Toffoli and a diagonal unitary, we will select TOF^nD^n to be the canonic one, and draw the respective relative phase multiple control Toffoli gate with same controls as the diagonal gate D^n and a distorted target, such as illustrated in Figure 1(c). The helpful intuition behind this pictorial representation is as follows: a Toffoli gate $TOF(X;y)$ may be combined with a diagonal gate $D(Z)$, $Z \in \{X, y\}$, following it to obtain a relative phase Toffoli gate, or a Toffoli gate $TOF(X;y)$ may be combined with a diagonal gate $D(Z)$, $Z \in \{X, y\}$, preceding it to obtain the inverse of a relative phase Toffoli gate; conversely, each relative phase Toffoli gate or its inverse may be broken down into a suitable pair of the multiple control Toffoli gate and the diagonal gate.

An important property of the relative phase multiple control Toffoli gates is that every one of those is an inverse of some other relative phase multiple control Toffoli gate. Indeed, for $R_1TOF^n = diag\left\{z_0, z_1, ..., z_{2^n-3}, \begin{pmatrix} 0 & z_{2^n-1} \\ z_{2^n-2} & 0 \end{pmatrix}\right\}$ and $R_2TOF^n = diag\left\{w_0, w_1, ..., w_{2^n-3}, \begin{pmatrix} 0 & w_{2^n-1} \\ w_{2^n-2} & 0 \end{pmatrix}\right\}$ $R_1TOF^n = R_2^{-1}TOF^n$ when $w_i = z_i^{-1}$ for $i = 0...2^n-3$, $w_{2^n-2} = z_{2^n-1}^{-1}$, and $w_{2^n-1} = z_{2^n-2}^{-1}$.

We next define special form relative phase multiple control Toffoli gates, that are important in some of the constructions that follow.

Definition 3. For a set $X = \{x_1, x_2, ..., x_n\}$, and its subset $X' = \{x_{i_1}, x_{i_2}, ..., x_{i_k}\}$ a *type-X' special form relative phase multiple control Toffoli gate*, $S^{X'}RTOF(x_1, x_2, ..., x_{n-1}; x_n)$, is defined as the matrix

$$diag\left\{z_0, z_1, ..., z_{2^n-3}, \begin{pmatrix} 0 & z_{2^n-2} \\ z_{2^n-1} & 0 \end{pmatrix}\right\},$$

where every pair of complex numbers z_s and z_t are equal whenever the binary expansions of s and t are different only in the digits $i_1, i_2, ..., i_{k-1}$, and i_k.

To illustrate, a type-$\{x_1\}$ $S^{x_1}RTOF(x_1, x_2, ..., x_{n-1}; x_n)$ is given by the matrix

$$diag\{z_0, z_1, ..., z_{2^{n-1}-1}, z_0, z_1, ..., z_{2^{n-1}-3}, \begin{pmatrix} 0 & z_{2^{n-1}-2} \\ z_{2^{n-1}-1} & 0 \end{pmatrix}\}.$$

The type-$\{x_1\}$ special form relative phase Toffoli gate $S^{x_1}RTOF$ has half the number of the degrees of freedom compared to the equal size unrestricted relative phase Toffoli gate $RTOF$. In practice, this suggests that it should be easier to find an efficient circuit implementing a relative phase Toffoli gate than it is to find one of the same size for a type-$\{x_1\}$ special form relative phase Toffoli gate. To give another example, a type-X $S^XRTOF(x_1, x_2, ..., x_{n-1}; x_n)$ is the most restrictive of the kind. It is equal to the respective Toffoli gate up to a global phase, and thereby does not give much freedom in implementing by a circuit over the $TOF(x_1, x_2, ..., x_{n-1}; x_n)$. This means that in the practical constructions, and whenever possible, we will try to use a type-X' special form relative phase multiple control Toffoli gate with the smallest size set X'.

An alternate and equivalent definition of a type-X' special form relative phase Toffoli gate is via a transformation given by the circuit $TOF(x_1, x_2, ..., x_{n-1}; x_n)D(X \setminus X')$. It furthermore serves as a basis for how we draw $SRTOF$ gates in the circuit diagrams. Compared to the multiple control Toffoli gate, every control/target in the set $X \setminus X'$ of $S^{X'}RTOF$ appears distorted by the dingbat originating from the respective $D(X \setminus X')$, and every control/target in the set X' appears undistorted, see Figure 1(d).

Beyond having fewer degrees of freedom compared to an unrestricted relative phase Toffoli gate, there is one more important difference between the special form relative phase Toffoli gates and the relative phase Toffoli gates: the inverse of a type-X' special form relative phase Toffoli gate is not always a type-X' special form relative phase Toffoli gate.

The use of subscripts allows to distinguish different versions of the relative phase and special form relative phase multiple control Toffoli gates. For instance, notations R_1TOF and R_2TOF indicate that both gates are some relative phase Toffoli gates, but they are not necessarily related. In contrast, an $R_1^{-1}TOF$ is the inverse

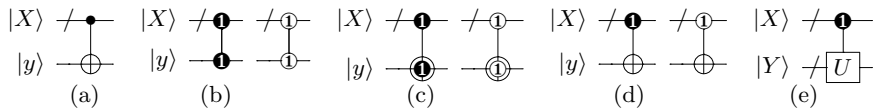
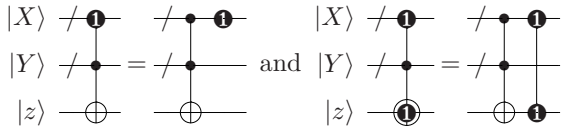


FIG. 1: (a) a multiple control Toffoli gate $TOF(X;y)$, (b) a diagonal gate $D_1(X;y)$ and its inverse; observe how different diagonal gates can be visually distinguished by the number within the control, and a diagonal gate and its inverse are related by the different color of the control, (c) a relative phase multiple control Toffoli gate $R_1TOF(X;y)$ and its inverse $R_1^{-1}TOF(X;y)$, (d) a type-y special form $S^yR_1TOF(X;y)$, and its inverse, and (e) a controlled-unitary U implemented up to some relative phase. The $\not+$ symbol denotes a multiqubit register.

of the R_1TOF. Recall, that a circuit implementing the inverse operation may be constructed by conjugating the gates in the circuit implementing the given unitary and inverting their order. Observe further that any two TOF gates of the same size are represented by identical matrices; this is not always true for some two $RTOF$ or a pair of $SRTOF$, therefore the ability to distinguish different versions of the relative phase implementations is important, as these could be different gates.

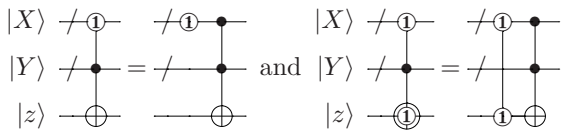
We will draw quantum gates and circuits using standard notations, including the relative phase gates per diagrams found in Figure 1, with time propagating from left to right. Some useful circuit identities clarifying and summarizing the above discussions are shown next.

1.



show canonic decomposition of $S^{Y,z}R_1TOF$ and S^YR_1TOF into the product of the multiple control Toffoli gate TOF and the diagonal gate D_1; read right-to-left, these rules show how to combine a suitable pair of the multiple control Toffoli gate and the diagonal gate into a (special form) relative phase Toffoli gate. When $Y=\emptyset$, second circuit illustrates the R_1TOF gate.

2.

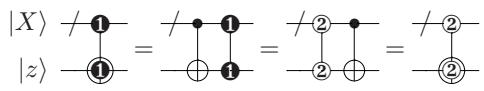


show canonic decomposition of $S^{Y,z}R_1^{-1}TOF$ and $S^YR_1^{-1}TOF$ into the product of the diagonal gate and the multiple control Toffoli gate. Indeed, looking at the second of the two identities,

$$\begin{aligned} & S^yR^{-1}TOF(X,Y;z) \\ &= (TOF(X,Y;z)D_1(X,z))^{-1} \\ &= D_1^{-1}(X,z)TOF(X,Y;z), \end{aligned}$$

being the circuit pictured on the right hand side.

3. ∀❶ ∃②:



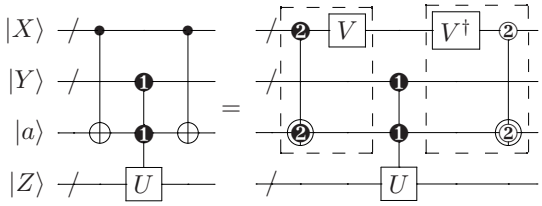
in other words, every R_1TOF is also an $R_2^{-1}TOF$ under the proper choice of relative phases.

In general, for any reversible gate $R(X)$ its relative phase version could be thought of as a product $R(X)D(X)$, for a proper diagonal unitary $D(X)$. This suggests a possible route in which the work reported in this paper may be extended.

III. MAIN RESULT

Our main result is summarized in the next three Propositions. We apply it to obtain multiple Corollaries, and to optimize multiple control Toffoli gates in the section that follows. The proofs of these three propositions rely on the three circuit identities concluding previous section, as well as the following notion: the controlled-U implemented up to a relative phase, $RCU(V,W;X)$, commutes with the controlled-V implemented up to a relative phase, $RCV(V,Y;Z)$, where the qubit sets V,W,X,Y, and Z are disjoint. This rule also applies to show that any two non-intersecting unitaries commute. We assume reader's familiarity with the above commutation rule, and do not explicitly prove it here.

Proposition 1. The conjugation of the controlled unitary U over the qubit set Z implemented up to a possible relative phase, $R_1CU(Y,a;Z)$, by a pair of multiple control Toffoli gates $TOF(X;a)$ allows the replacement of these multiple control Toffoli gates with their relative phase versions implemented up to any desired unitary $V(X)$, such as illustrated next:



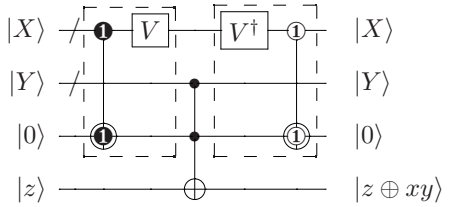
(1)

Proof: The proof is accomplished via the following set of circuit transformations:

$$\begin{aligned}
&TOF(X;a)R_1CU(Y,a;Z)TOF(X;a)\\
&= TOF(X;a)D_2(X;a)D_2^{-1}(X;a)\\
&\quad R_1CU(Y,a;Z)TOF(X;a)\\
&= TOF(X;a)D_2(X;a)R_1CU(Y,a;Z)\\
&\quad D_2^{-1}(X;a)TOF(X;a)\\
&= R_2TOF(X;a)R_1CU(Y,a;Z)R_2^{-1}TOF(X;a)\\
&= R_2TOF(X;a)V(X)V^{-1}(X)R_1CU(Y,a;Z)\\
&\quad R_2^{-1}TOF(X;a)\\
&= \big[R_2TOF(X;a)V(X)\big]R_1CU(Y,a;Z)\\
&\quad \big[V^{-1}(X)R_2^{-1}TOF(X;a)\big].
\end{aligned}$$

□

The result of Proposition 1 can be reduced to the following form once $RCU(Y,a;Z)$ is set to implement the Toffoli type gate, $TOF(Y,a;z)$:



Indeed, the corresponding circuit on the left hand side in (1) computes

$$\begin{aligned}
&|X,Y,0,z\rangle \overset{TOF(X;0)}{\mapsto} |X,Y,x,z\rangle \overset{TOF(Y,x;z)}{\mapsto}\\
&|X,Y,x,z\oplus xy\rangle \overset{TOF(X;x)}{\mapsto} |X,Y,0,z\oplus xy\rangle,
\end{aligned}$$

which is indicated by the formulas on the output side. This, in turn, leads to the following corollary.

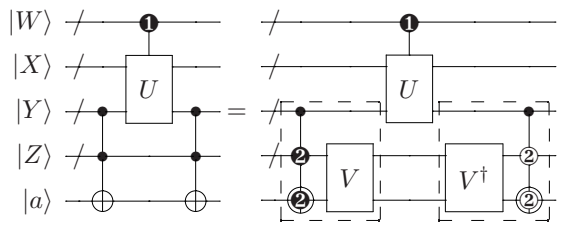
Corollary 1. An n-qubit Toffoli gate TOF^n can be implemented with the cost not exceeding the sum of twice the cost of an n-qubit relative phase Toffoli gate $RTOF^n$ and the cost of the CNOT gate, using one ancilla qubit set to and returned in the value $|0\rangle$. In other words, in the presence of such an ancilla,

$$Cost(TOF^n) \leq 2 \times Cost(RTOF^n) + Cost(CNOT).$$

This corollary may be reformulated for a different choice of the middle gate, *e.g.*, as follows: $Cost(TOF^n) \leq 2 \times Cost(RTOF^{n-1}) + Cost(TOF^3)$.

Other gate configurations are also supported by the relative phase Toffolis. The following Proposition complements the set of basic rules we base the proposed optimization approach on.

Proposition 2. Consider the conjugation of a controlled-U gate $R_1CU(W;X,Y)$ implemented possibly up to some relative phase, by a pair of identical multiple control Toffoli gates, such as illustrated in (2) on the left hand side. Then, the following circuit identity holds for any unitary transformation V over the qubit set $\{Z \cup a\}$ and any $S^Y R_2 TOF(Y,Z;a)$ (a type-Y special form relative phase Toffoli gate):



(2)

Proof: This proposition may be proved similarly to Proposition 1,

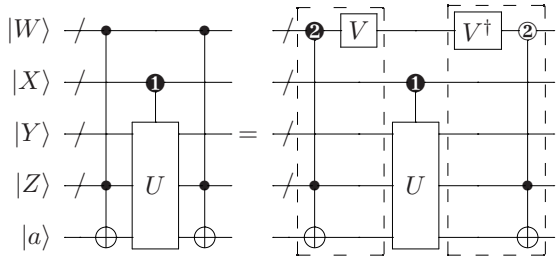
$$\begin{aligned}
&TOF(Y,Z;a)R_1CU(W;X,Y)TOF(Y,Z;a)\\
&= TOF(Y,Z;a)D_2(Z;a)D_2^{-1}(Z;a)\\
&\quad R_1CU(W;X,Y)TOF(Y,Z;a)\\
&= TOF(Y,Z;a)D_2(Z;a)R_1CU(W;X,Y)\\
&\quad D_2^{-1}(Z;a)TOF(Y,Z;a)\\
&= S^Y R_2TOF(Y,Z;a)R_1CU(W;X,Y)\\
&\quad S^Y R_2^{-1}TOF(Y,Z;a)\\
&= S^Y R_2TOF(Y,Z;a)V(Z;a)V^{-1}(Z;a)\\
&\quad R_1CU(W;X,Y)S^Y R_2^{-1}TOF(Y,Z;a)\\
&= \big[S^Y R_2TOF(Y,Z;a)V(Z;a)\big]R_1CU(W;X,Y)\\
&\quad \big[V^{-1}(Z;a)S^Y R_2^{-1}TOF(Y,Z;a)\big]
\end{aligned}$$

An alternate proof may be constructed via restricting W,X,Y, and Z to contain at most a single qubit each, and multiplying the corresponding matrices [5]. The benefit of considering such a matrix multiplication is in the ability to show that the $S^Y R_2 TOF(Y,Z;a)$ turns out to be the relative phase Toffoli gate that allows most freedom in selecting relative phases for a general unitary U, allowing to formulate this proposition as an "if-and-only-if" statement. Furthermore, looking at the matrices helps to expand the set of possible allowed relative phase replacements once U is known. □

The results of Propositions 1 and 2 may be generalized via introducing a control set P that controls all three gates on the left hand side and well as all five gates on the right hand side, and a control set Q that controls all gates except V.

Observe, that between the two Propositions they cover all situations when a relative phase controlled-U is conjugated by a pair of multiple control Toffoli gates such that the targets of those multiple control Toffoli gates do not intersect with the U, resulting in the ability to replace a pair of multiple control Toffoli gates with a pair of simpler gates. A similar circuit identity may be developed for the scenario when the target of the multiple control Toffolis intersects with the qubits used by the unitary U. This circuit identity relies on the special form relative phase Toffoli gates. We have not yet found practical examples where such circuit identity would yield an advantage and the results of Propositions 1 and 2 do not apply, but formulate the statement of the respective Proposition for completeness.

Proposition 3. The conjugation of the controlled unitary U implemented up to a relative phase, $R_1CU(X;Y,Z,a)$, by a pair of the multiple control Toffoli gates $TOF(W,Z;a)$ allows the replacement of these multiple control Toffoli gates with the type-$\{Z \cup a\}$ special form relative phase version (up to a multiplication by any desired unitary $V(W)$) and its inverse, as follows:



We do not include an explicit proof, but mention that it may be obtained similarly to that of Propositions 1 and 2. Furthermore, we note that the scenario where $R_1CU(X;Y,Z,a)$ is a diagonal gate, *e.g.*, a controlled-R_z implemented up to a possible relative phase, is better handled by applying Proposition 1 than Proposition 3 (*e.g.*, see item 4, Subsection III A). Indeed, Proposition 1 uses the most generic unspecified type relative phase Toffoli, and any controlled-R_z may be thought of as a targetless gate ($|Z| = 0$ in the statement of Proposition 1) or otherwise, one may introduce a new target qubit that applies a global phase [4, Figure 4.5].

A. Applications

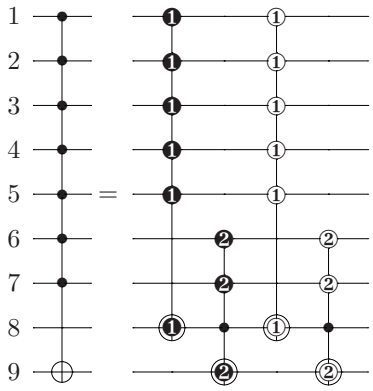
The principal circuit equalities (1) and (2) suggest a circuit optimization procedure by which a suitable pair of the multiple control Toffoli gates can be replaced with their relative phase or special form relative phase implementations up to the right hand multiplication by any desired unitary over the proper qubit set. The rules may be used interchangeably and combined. In particular, we next illustrate how the above approach can be applied to optimize the most popular constructs used to implement/decompose the multiple control Toffoli gates into simpler gates. In the following discussions, we will omit unitaries V, with the understanding that if needs be, they may be added back in.

Corollary 2. [Optimization of the construction reported in [3, Lemma 7.2].] A multiple control Toffoli gate TOF^n can be implemented by a circuit consisting of $4n-14$ relative phase Toffoli gates $RTOF^3$ and a type-$\{y\}$ special form relative phase Toffoli gate $S^yRTOF^3(x,y;z)$ and its inverse over a circuit with at least $2n-3$ qubits, such as illustrated in Figure 2.

Proof: The numeric order of subscripts in the special form and relative phase Toffoli gates indicates the order in which the circuit equalities (2) and (1) are applied to the original circuit reported in [3, Lemma 7.2] to obtain the desired simplified decomposition. Observe that when during this process a pair of Toffoli gates $TOF^3(a,b;c)$ is replaced with a special form or a relative phase implementation, the circuit in the middle may be equivalent to a combination of a suitable multiple control Toffoli gate—possibly up to a relative phase, and a transformation on the qubits outside the set $\{a,b,c\}$. This latter transformation may be factored out, thereby allowing all circuit alternations to retain the original functional correctness.

Finally, observe that the identities (1) and (2) may be used in a number of different ways, resulting in different constructions, and not just the particular one selected in the statement of the Corollary. In Figure 2(b) we used one of such constructions that minimizes the number of the special form relative phase Toffoli gates to gain most freedom in substituting Toffoli gates with their relative phase implementations. In Figure 2(c) we furthermore restricted the number of potentially different $RTOF$ gates via making the following assignments: $R_4TOF := R_3TOF$, $R_5TOF := R_3^{-1}TOF$, and $R_6TOF := R_3^{-1}TOF$. This implementation will be used later in the paper. □

Corollary 3. [Optimization of the construction reported in [3, Lemma 7.3].] A multiple control Toffoli gate TOF^n can be implemented by a circuit consisting of two relative phase Toffoli gates $RTOF^k$ and two special form relative phase Toffoli gates $SRTOF^{n-k+2}$ over a circuit with at least $n+1$ qubits, such as illustrated next:



Proof: To obtain this construction, both circuit identities (1) and (2) need to be applied once, in any order. □

Corollary 4. [Optimization of the construction in [4, page 184].] A multiple control gate C^nU can be implemented by a circuit consisting of $2n-2$ relative phase Toffoli gates $RTOF^3$ and one CU gate over a circuit with at least $2n$ qubits of which some $n-1$ qubits are set to and

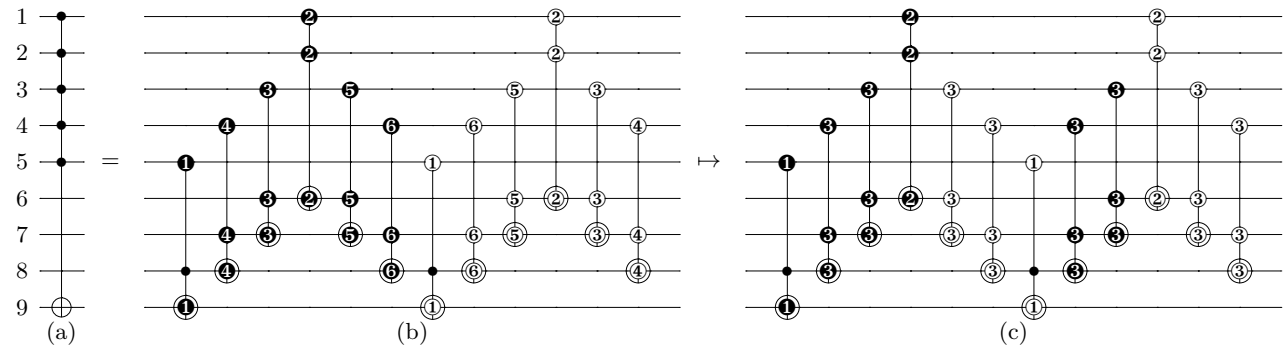
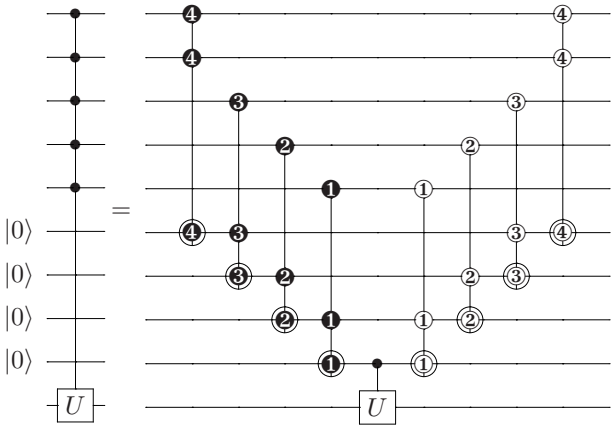


FIG. 2: (a-b) Implementation of TOF^6 on the qubits $1-9$ using $S^{\{8\}}RTOF^3$ and its inverse, and 10 $RTOF^3$ gates. (a-c) Implementation of TOF^6 using a narrow selection of the relative phase and special form relative phase Toffoli gates.

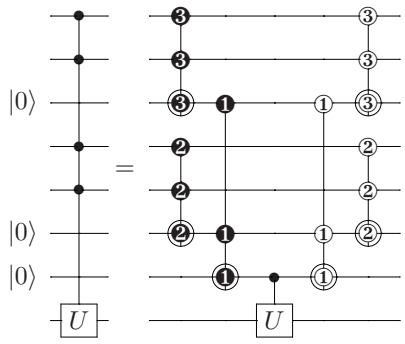
returned in the value $|0\rangle$, such as illustrated next:



Proof: The circuit identity (11) is applied $n-1$ times. □

The implementation in [7, equation (13)] optimizes the depth of the circuit [4, page 184], but does not prevent our construction from being applied. We formalize this observation in the following Corollary.

Corollary 5. [Optimization/generalization of the construction in [7, equation (13)].] A multiple control gate C^nU can be implemented by a circuit consisting of $2n-2$ relative phase Toffoli gates $RTOF^3$ and one CU gate over a circuit with at least $2n$ qubits of which some $n-1$ qubits are set to and returned in the value $|0\rangle$, such as illustrated next:



Some other optimizations include the following.

1. Circuit in [3, Lemma 7.5] may rely on the simpler relative phase multiple control Toffoli gate and its inverse, rather than two multiple control Toffoli gates (gates #2 and #4 on the right hand side).

2. Circuit in [3, Lemma 7.9] may rely on the simpler special form relative phase multiple control Toffoli gate and its inverse, rather than two multiple control Toffoli gates (gates #2 and #4 on the right hand side).

3. Circuit in [3, Lemma 7.11] may rely on the simpler relative phase multiple control Toffoli gate and its inverse, rather than two multiple control Toffoli gates (gates #1 and #3 on the right hand side).

4. Circuit in [6, Figure 3] may rely on the simpler relative phase Toffoli gates and their inverses, as is best seen via applying Proposition 1.

IV. OPTIMIZING IMPLEMENTATIONS OF THE MULTIPLE CONTROL TOFFOLI GATES USING THE EXISTING RELATIVE PHASE TOFFOLI CIRCUITS

In this section we study in detail how to optimize the implementations of the multiple control Toffoli gates, show that all of the known optimized implementations can be explained by the means of the relative phase Toffoli substitutions described in this work, and report some new optimized circuits.

A. Circuit cost

The question of the efficiency of implementing a certain transformation requires one to formally define a circuit cost. Depending on the definition of cost, certain circuits will be preferred over other circuits.

There are a number of different definitions of the circuit cost used in the literature, each originating from considering certain specific requirements. At the highest abstraction level, firstly, one needs to determine if they are dealing with logical level or physical level circuits.

In the former case, one has to derive the protocols and compute the costs of the constructible fault-tolerant gates, given the selected approach to error correction. Within this framework Clifford+T circuits received a significant attention. This is because Clifford gates such as Pauli-X, Y, Z, Hadamard, Phase, and CNOT are believed to be relatively inexpensive to implement fault tolerantly on the logical level. The non-Clifford gate T, or any other constructible non-Clifford gate required for computational universality, is more difficult to generate. The known approaches employ state purification and gate teleportation as a means of generating the T gate, that can get quite costly in the realistic systems [8]. As a result, the cost of the implementation of a logical circuit can be very crudely approximated by the number of the T gates used.

In the case of physical level circuits, one is limited to the ability of the controlling apparatus to apply transformations to the physical quantum information processing system of choice. There is a great variety of the possibilities here. We consider a simple and popular weak interaction model, where the single-qubit gates can be implemented efficiently, and of the two-qubit gates, that take considerably more effort to implement, we have just the CNOT gate. The cost of the circuits can thus be evaluated via counting the number of the CNOT gates in the single-qubit and CNOT gate circuits. Despite apparent oversimplification, there is a specific promising quantum information processing approach, where exactly this formula describes the circuit cost at a high abstraction level. Indeed, trapped ions with Molmer-Sorenson gate [9] operate in the weak coupling regime (two-qubit gates take roughly $10-20$ fold effort to implement compared to arbitrary single-qubit gates), and Molmer-Sorenson gate itself is equivalent to the CNOT up to a conjugation by a pair of $R_Z(a)$ and $R_Z(-a)$ gates on both qubits, for a proper choice of parameter a, and a few single-qubit Phase and Hadamard gates.

An advantage of measuring the cost of the circuit implementations by the T-count and the CNOT-count is due to the popularity of these circuit cost metrics in the literature, and the ability to compare relative phase inspired implementations developed in this work to the known ones.

Disadvantages of using either one of these two circuit cost metrics are numerous. Neither circuit metric accounts for:

- the depth, that could be more important than the gate count, especially when one is, quite naturally, concerned with the speed of the computation given by a quantum circuit rather than just its size;
- the connectivity pattern of the qubits. Indeed, physical space spans only three dimensions, and every qubit cannot be connected to every other qubit in a scalable fashion within a finite-dimensional space; or
- the number of ancillary qubits used, that is particularly important on the physical level. The number of ancillary qubits used also influences the efficiency of connections between primary qubits. This is because both primary qubits and ancillary qubits share same physical space and yet need to be as close to each other as possible for higher efficiency.

These are all very important practical considerations. However, our goal is to demonstrate the advantages of the framework introduced in this paper for designing efficient circuits, therefore we restrict the attention to the above two simplistic metrics. We furthermore encourage to apply the techniques from this paper to designing efficient circuits in the scenario where the details of the circuit cost function are known.

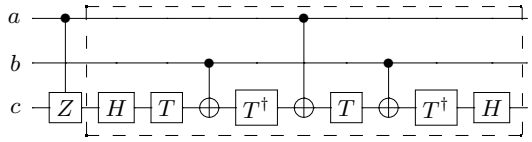
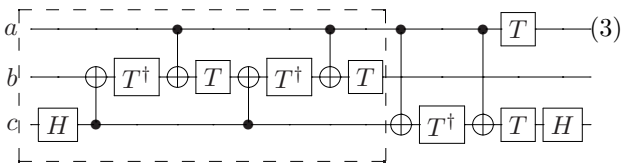


FIG. 3: Toffoli gate implemented up to a relative phase: gates 1-10 implement a type-$\{c\}$ special relative phase Toffoli gate, known as the controlled-controlled-iX in [7], whereas circuit with gates 2-10 implements some generic relative phase Toffoli gate. The controlled-Z gate $CZ(a;c)$ may commute through the Hadamard $H(c)$, at which point it will change into $CNOT(a;c)$, and the circuit will show in an alternate form. It may be established, via applying the result of Corollary 1, that the CNOT count of the circuit with gates 2-10 is optimal.

B. Toffoli and Toffoli-4 gates up to a relative phase

Firstly, recall a circuit implementing the Toffoli gate $TOF(a,b;c)$ itself:



(3)

This circuit may be drawn in many different ways using no more than the minimal numbers of 6 CNOT gates and 7 $T/T^\dagger$ gates, however, we prefer this form since it has the largest number of gates operating on the qubits a and c after no more gates are being applied to the qubit b.

Literature encounters two apparently related implementations of the Toffoli gate up to a relative phase [4],

page 183] and [7], that we summarize in one distilled picture, see Figure 3. There are more symmetries and properties to this circuit than those that necessarily meet the eye on the first glance. In particular,

- Gates 1-10 implement a type-$\{c\}$ special form relative phase Toffoli gate $S^cRTOF(a,b;c) = diag\left\{1,1,1,1,1,1,\begin{pmatrix} 0 & i \\ i & 0 \end{pmatrix}\right\}$, whereas gates 2-10 implement a relative phase Toffoli gate $RTOF(a,b;c) = diag\left\{1,1,1,1,1,-1,\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}\right\}$.

- First gate, the controlled-Z, can be moved to the end of the circuit, resulting in the construction of the type-$\{c\}$ special form relative phase Toffoli gate $S^cRTOF(a,b;c) = diag\left\{1,1,1,1,1,1,\begin{pmatrix} 0 & -i \\ -i & 0 \end{pmatrix}\right\}$.

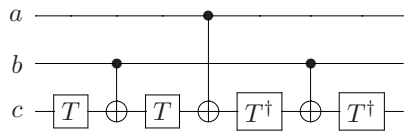
- Simultaneous substitution $T \mapsto T^\dagger$ and $T^\dagger \mapsto T$ allows constructing more circuits implementing a relative phase Toffoli gate.

- The circuit given by the gates 2-10 is self-inverse.

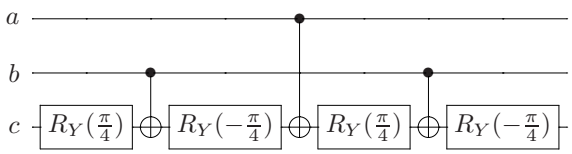
- Qubits a and b may be interchanged. Applying this operation gives modified relative phase Toffoli circuits.

- Adding gates $T^m(a)$ and $T^n(b)$ (powers of the T gate), where $m,n \in \{0,1,...,7\}$, to both the beginning and the end of the circuit in Figure 3 allows constructing more relative phase Toffoli gates.

- Consider gates 3-9. Using the CNOT-T algebra terminology [7, 10], the T gate is being applied to $\{c, -(b\oplus c), a\oplus b\oplus c, -(a\oplus c)\}$ (negative sign indicates the application of $T^\dagger$). Instead, we may apply the T gate to $\{c, b\oplus c, -(a\oplus b\oplus c), -(a\oplus c)\}$. Then, the circuit we obtain looks as follows:



Observe how similar it is to [4, page 183]—essentially, Y rotations are replaced by Z rotations. Optimality of the above circuit employing R_Y rotations in place of T (sometimes known as Margolus gate) was shown in [11]. Conjugating this circuit by a pair of Hadamard gates on the qubit c allows to obtain a relative phase Toffoli $RTOF(a,\bar{b};c)$, where $\bar{b}$ denotes the negative control. Similarly, if the $T/T^\dagger$ gates of the circuit in Figure 3, gates 3-9, were replaced with $R_Y(\pi/4)/R_Y(-\pi/4)$, as illustrated next,



we would have obtained an $RTOF(a,\bar{b};c)$.

We found no relative phase Toffoli-4 implementations in the literature, but realized that one may be constructed as follows. Consider circuit in Figure 3, gates 2-10. Replace $CNOT(a;c)$ with a type-$\{c\}$ special form relative phase Toffoli gate $S^cRTOF(x,a;c)$; this operation introduces a new qubit, x. The result is an $RTOF(x,a,b;c)$. Figure 4 illustrates the result of such a procedure for $SRTOF$ selection per Figure 3 (observe that the controlled-Z was commuted through the Hadamard gate to obtain the CNOT). In the matrix form, the gate looks as follows, $diag\left\{1,1,1,1,1,1,1,1,1,1,1,1,i,-i,\begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}\right\}$.

C. Results of the simplification

Since T-count optimal and CNOT-count optimal implementations of the three-qubit Toffoli gate are known, we will concentrate on the Toffoli-4 and larger gates. This section is not meant to report complete results of the optimization that is possible to obtain (indeed, there is no guarantee there are no better relative phase Toffoli-4 gates to be used, and we did not look for the relative phase Toffoli-5 and larger gates), rather show a clear advantage of using relative phase and special form relative phase Toffoli gates and motivate their further in-depth study.

Consider Toffoli-4 implementation via a circuit with Clifford+T gates. Using matrix determinant argument, one may establish that the Toffoli-4 may not be implemented unless at least one ancilla qubit is available. This is because the determinant of the 16×16 matrix representing the Toffoli-4 evaluates to the number (-1), whereas the determinants of all Clifford+T library gates, when viewed as 16×16 matrices, are equal to 1. By composing the products of matrices with determinant 1 it is impossible to obtain a matrix with determinant (-1). As a result, at least one ancilla is required.

Once we have established that an ancilla qubit is required, there are two options for the kind of ancilla qubit it is. One, more restrictive, prescribes that the ancilla be available in the state $|0\rangle$; the other provides the ancilla in some unknown state, $|x\rangle$. In both cases, when implementing Toffoli-4 with the help of an ancilla, special care needs to be taken to return the value of ancilla to its original state. We consider both cases next.

Optimization of Toffoli-4.

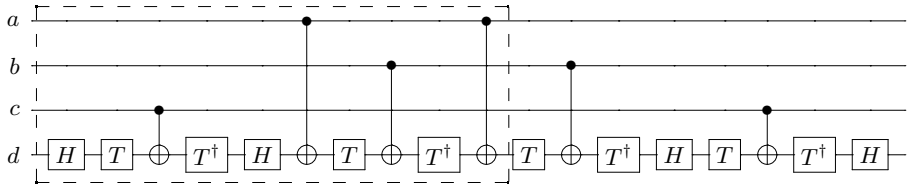
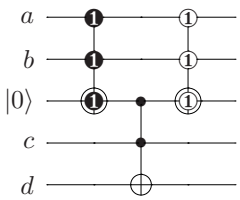


FIG. 4: Circuit implementing Toffoli-4 up to a relative phase, $RTOF(a, b, c; d)$.

- *Ancilla $|0\rangle$, minimizing T count.* Literature encounters two results, [10] and [7], both based on the optimization of [4, page 184]. In particular, [10] reports an optimized circuit with 15 T gates (down from unoptimized 21), and [7] observes that two Toffolis can be replaced with the relative phase Toffoli called the controlled-controlled-iX, Figure 3, which explains the optimization obtained in [10]. Our solution uses a somewhat simpler relative phase Toffoli, see Figure 3, dashed (gates 2-10), to obtain $TOF^4(a, b, c; d)$:

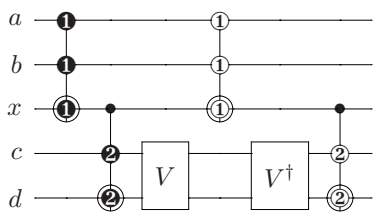


(4)

  There is no advantage in the number of T gates. However, our solution explains both known circuits and features a smaller overall gate count.

- *Ancilla $|0\rangle$, minimizing CNOT count.* [7] uses controlled-controlled-iX to obtain an implementation with 14 CNOTs. To our knowledge, this was the best known result in the literature to date. Our construction, (4), requires only $12 = 3 + 6 + 3$ CNOT gates, since our relative phase Toffoli (Figure 3, dashed) requires one less CNOT gate. Observe, that per [13] the lower bound for the number of CNOT gates is 8. Therefore, our 12-CNOT construction may not be improved by more than 4 CNOT gates.

- *Arbitrary single-qubit ancilla, minimizing T count.* The best known solution, [10], optimizes the 28 T gate implementation from [3, Lemma 7.2]. The result is a circuit with 16 T gates. Our solution matches this solution, and in fact explains how it works. Indeed, we obtain the desired $TOF^4(a, b, c; d)$ as follows:



(5)

  where x is the ancilla qubit in an unknown state, $R_1TOF(a, b; x)$ is the relative phase Toffoli per Figure 3, dashed; and $S^x R_2TOF(x, c; d)V(c, d)$ pair is given by (3)–dashed. Essentially, $V(c, d)$ is designed such as to undo all gates applied to the qubits c and d at the end of the implementation given by (3). We have not found a suitable special relative phase Toffoli gate implementation that is different from the implementation of the Toffoli gate itself, per (3), and giving a better optimization once combined with proper $V(c, d)$. The resulting T-count of our construction is thus $16 = 4 + (7 - 3) + 4 + (7 - 3)$. Apart from the matching number of T gates, our solution contains fewer Clifford gates (*e.g.*, 14 CNOTs vs 54 CNOTs in [10]), and may also be rewritten as a T-depth 4 circuit (T-depth 1 per each relative phase Toffoli stage) at the cost of a higher number of ancillae and a higher number of CNOT gates.

- *Arbitrary single-qubit ancilla, minimizing CNOT count.* Using CNOT-optimal implementation of the controlled-controlled-iX from [7] over [3, Lemma 7.2] would yield a circuit with 20 CNOT gates, as is done in [12]. The original circuit, [3, Lemma 7.2], uses 24 CNOT gates after each Toffoli is substituted with their CNOT-optimal implementation. Our construction, (5), contains $14 = 3 + 4 + 3 + 4$ CNOT gates.

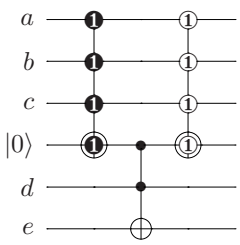
Observe that the above implementations, if considered as circuits over Clifford+T library, use the minimal number of ancillae, being one.

Optimization of Toffoli-5.

One may once again apply the determinant argument to establish that the Toffoli-5 gate needs at least one ancilla to be available before it may be implemented as a Clifford+T circuit.

- *All ancillae in the state $|0\rangle$, minimizing T count.* The best known solution is given by [10] via an optimization of the construction in [4, page 184], and explained by [7] to be a four controlled-controlled-iX and one Toffoli circuit. The T-count is 23 and both known solutions use two ancillae. Our solu-

tion implementing $TOF^5(a,b,c,d;e)$ is as follows:

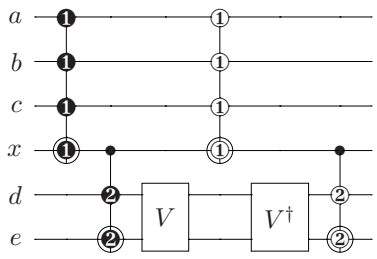


(6)

per R_1TOF^4 implementation found in Figure 4 and Toffoli implementation from (3). Our solution uses $23 = 8 + 7 + 8$ T gates, relies on only one ancilla, and has a smaller total number of gates compared to the previously known constructions.

- *All ancillae in the state $|0\rangle$, minimizing CNOT count.* The construction from [7] gives the best known CNOT count of 22 over a circuit that uses two ancillae. Our circuit (6) contains 18 CNOTs and uses only one ancilla. Recall that the lower bound for the number of CNOT gates is 10 [13].

- *All ancillae in an unknown state, minimizing T count.* The best known solution is given in [10] and features 28 T gates. Our solution implementing $TOF^5(a,b,c,d;e)$ is as follows:



(7)

where x is the ancilla qubit in an unknown state, R_1TOF^4 is the relative phase Toffoli from Figure 4, and $S^xR_2TOF(x,d;e)V(d,e)$ pair is given by (3)–dashed. Observe, that the overall number of T gates is $24 = 8 + (7-3) + 8 + (7-3)$, we use one less ancilla compared to the best known construction, and a smaller overall number of the non-T gates.

We can furthermore explain how to obtain the solution with $12k - 20$ T gates to implement a k-controlled Toffoli gate using $k-2$ unrestricted ancillae featured in [10] without resorting to a computer optimization. This is done via the use of the relative phase Toffoli and Toffoli-V pair from Figure 3, dashed, and (3)–dashed, over the construction reported in Corollary 2. We illustrate how this works using the circuit from Figure 2(c) and observe that the arguments easily generalize to arbitrary k. Substituting relative phase and special relative phase-V pair implementations into the construction in Figure 2(c) replaces each relative phase Toffoli with a circuit containing 4 T gates. The total number of the T gates would thus be 48 (for arbitrary k, $16k - 32$), higher than 40 [10]. However, observe that R_3TOF and $R_3^{-1}TOF$ are inverses of each other. This means that the gates $T^\dagger$ and H on the target qubit that the R_3TOF ends with would cancel with H and T that the $R_3^{-1}TOF$ begins with. This cancellation happens between all four such pairs $\{R_3TOF, R_3^{-1}TOF\}$ found in the circuit. The total reduction is thus by 8 T gates (for arbitrary k, $4k-12$), leading to a circuit with 40 T gates (for arbitrary k, $16k-32-4k+12 = 12k-20$).

- *All ancillae in an unknown state, minimizing CNOT count.* [12] includes an implementation where the controlled-controlled-iX is used within [3, Lemma 7.2] for all but two gates. This construction relies on 36 CNOT gates. For arbitrary n, the CNOT count is $16n-44$, which we further refer to as cc-iX implementation in Table I. Observe that [14] reports an implementation with 26 two-qubit gates using two ancillae. The optimization in [14] is motivated by a computational model where the two-qubit interaction given by $diag\{I, X^{\pm t}\}$, where $t \in \mathbb{R}[0,1]$ and X is Pauli-X, is tunable and parametrized by time. Therefore, for example, a controlled-$\sqrt{NOT}$ would cost half as much as the CNOT, as it only needs to be evolved for half the time. In our calculations given here, we do not allow such things to happen, but observe that it would be interesting to apply the reported relative phase Toffoli constructions within that framework. Controlled-$\sqrt{NOT}$ may be implemented as a 2-CNOT circuit [4, Figure 4.6]. The 26 two-qubit gate circuit of [14] has 18 controlled-$\sqrt{NOT}$ gates and 8 CNOT gates, therefore it would be transformed into one with 44 CNOT gates. Note, however, that it would make little sense from the point of view of the computational model considered in [14], as a length-0.5 interaction is being replaced with a length-2 interaction.

  In comparison, our solution, given by (7), is a circuit with 20 $(= 6 + 4 + 6 + 4)$ CNOT gates that uses only one ancilla—latter being provably optimal within the framework of Clifford+T circuits.

We generalize the above examples of Toffoli-4 and Toffoli-5 optimization to any number of qubits in the following two Propositions.

Proposition 4. A size $n \geq 4$ multiple control Toffoli gate TOF^n may be implemented using $\lceil \frac{n-3}{2} \rceil$ ancillary qubits, set to and returned in the value $|0\rangle$, by a circuit with:

- $8n - 17$ T gates,
- $6n - 12$ CNOT gates, and
- $4n - 10$ Hadamard gates.

Proof: The proof is by induction. The statement is clearly true for $n = 4$ and $n = 5$, as has been explicitly verified in the previous discussions. To prove the transition from an even $n = 2k$ to the odd $n = 2k + 1$ observe that the middle gate TOF^3 can be replaced with the circuit (4). This introduces an $RTOF^3$, Figure 3, dashed, and its inverse. Note that a new ancillary qubit is being introduced on this step, and the gate counts increase by $8 = 4+4$ for T, by $6 = 3+3$ for CNOT, and by $4 = 2+2$ for Hadamard. The transition from an odd $n = 2k + 1$ to the even $n = 2k + 2$ is accomplished via replacing $RTOF^3$ with $RTOF^4$, Figure 4, and its inverse with the inverse of $RTOF^4$. Observe that the gate counts grow by 8/6/4 for T/CNOT/Hadamard, but no new ancilla is being introduced. □

Note that [7] reports a circuit with $n - 3$ $|0\rangle$ ancillae, $8n - 17$ T gates, $8n - 18$ CNOT gates, and $4n - 10$ Hadamard gates.

Proposition 5. A size $n \geq 5$ multiple control Toffoli gate TOF^n may be implemented by a circuit using $\lceil \frac{n-3}{2} \rceil$ ancillary qubits residing in an arbitrary state and returned unchanged, by a circuit with:

- $8n - 16$ T gates,
- $8n - 20$ CNOT gates, and
- $4n - 10$ Hadamard gates.

Proof: To assist with proving this Proposition, define the following gates:

1. $RTL(a, b, c)$ per Figure 3, dashed. This is a relative phase Toffoli gate. The implementation contains 9 elementary gates: 4 T gates, 3 CNOTs, and 2 Hadamards.
2. $RTS(a, b, c)$ per Figure 3, gates 2-6. This is a relative phase Toffoli followed by a $V(b, c)$ that removes the last four gates. The circuit contains 5 elementary gates: 2 T gates, 2 CNOTs, and 1 Hadamard.
3. $SRTS(a, b, c)$ per circuit (3), dashed. This is a Toffoli gate (as such it is also a type-$\{b\}$ special form relative phase Toffoli) followed by a $V(a, c)$ that removes last six gates. $SRTS$ contains 9 elementary gates: 4 T gates, 4 CNOTs, and 1 Hadamard.
4. $RT4L(a, b, c, d)$ per Figure 4. This is a 4-qubit relative phase Toffoli. It contains 8 T gates, 6 CNOTs, and 4 Hadamards.
5. $RT4S(a, b, c, d)$ per Figure 4, dashed. This is a relative phase Toffoli-4 $RT4L(a, b, c, d)$ followed by a $V(b, c, d)$ that removes last 8 gates. It is composed of the following elementary gates: 4 T gates, 4 CNOTs, and 2 Hadamards.

We first prove the Proposition for the resource count of $n - 3$ ancillae, $8n - 16$ T gates, $8n - 18$ CNOT gates, and $4n - 10$ Hadamard gates, and then introduce the $RT4L/RT4S$ gates that further improve the ancilla and CNOT count. The proof relies on the construction found in Figure 2(c). Assuming qubits are numbered 1 to $2n-3$ and we are attempting to implement $TOF^n(1, 2, ..., n - 1; 2n - 3)$, select the gates in Figure 2(c) as follows:

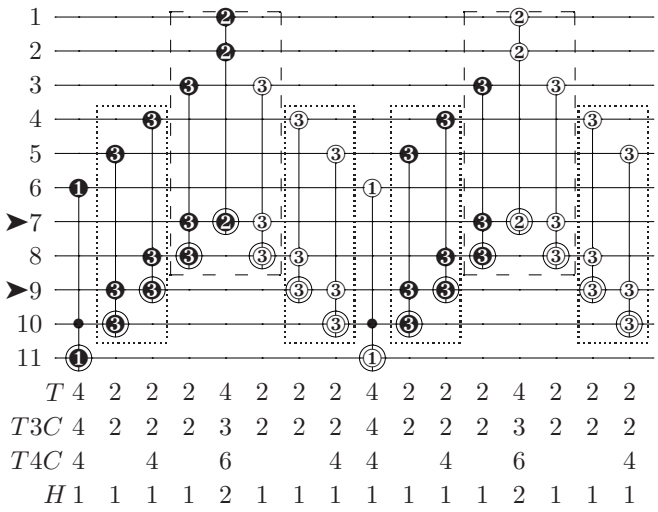
1. First gate is $SRTS(n - 1, 2n - 4, 2n - 3)$.
2. Next $k = 1..n - 4$ gates are $RTS(2n - 4 - k, n - 1 - k, 2n - 3 - k)$.
3. Next gate is $RTL(1, 2, n)$.
4. Next $k = 1..n - 4$ gates are $RTS^{-1}(n - 1 + k, k + 2, n + k)$ (inverses of the gates in item 2 read in reverse order).
5. Next gate is $SRTS^{-1}(n - 1, 2n - 4, 2n - 3)$ (this is the matching inverse pair for the gate in item 1).
6. Next $k = 1..n - 4$ gates are $RTS(2n - 4 - k, n - 1 - k, 2n - 3 - k)$ (same as item 2).
7. Next gate is $RTL^{-1}(1, 2, n)$ (this is the matching inverse for the gate in item 3).
8. Last $k = 1..n - 4$ gates are $RTS^{-1}(n - 1 + k, k + 2, n + k)$ (same as item 4).

Observe that the desired preliminary gate counts are satisfied. Next step is introducing $RT4L/RT4S$ gates to replace as many RTL and RTS as possible.

1. First, replace the circuit $RTS(n, 3, n + 1)RTL(1, 2, n)RTS^{-1}(n, 3, n + 1)$ (last gate in item 2, the gate in item 3, and first gate in item 4) with $RT4L(1, 2, 3, n + 1)$ and $RTS(n, 3, n+1)RTL^{-1}(1, 2, n)$ $RTS^{-1}(n, 3, n+1)$ (last gate in item 6, the gate in item 7, and first gate in item 8) with $RT4L^{-1}(1, 2, 3, n + 1)$. Note that this procedure may only apply for $n \geq 5$. It furthermore reduces the CNOT count from $7 = 2 + 3 + 2$ to 6 twice, for a total saving of 2 CNOTs. Finally, observe that the qubit n is no more used. Thus, we save one ancillary qubit worth of computational space.
2. For $k = 1..\lceil \frac{n-6}{2} \rceil$ we introduce four $RT4S$ gates by replacing a pair of neighbouring RTS on the left and right hand sides of the previous step. In particular, we replace $RTS(n+2k, 2k+3, n+2k+1)RTS(n - 1 + 2k, 2k + 2, n + 2k)$ (item 2) with $RT4S(n - 1 + 2k, 2k + 2, 2k + 3, n + 2k + 1)$ and $RTS^{-1}(n-1+2k, 2k+2, n+2k)RTS^{-1}(n+2k, 2k+3, n+2k+1)$ (item 4) with $RT4S^{-1}(n-1+2k, 2k+2, 2k+3, n+2k+1)$, and similarly in the second half of the circuit (items 6, 8). Observe that this operation does not change the gate counts, but frees up qubit $n + 2k$ that is no more used, providing a reduction of one ancilla.

The total reductions from the above construction are a pair of CNOT gates, and $\lfloor \frac{n-3}{2} \rfloor$ qubits, leading to the resource counts as announced in the statement of the Proposition.

Looking at the following circuit helps visualize all replacements and gate counts:



In the above, dashed gates are replaced with $RT4L(1,2,3,8)$ and its inverse, freeing qubit 7, and dotted gates are replaced with $RT4S(8,4,5,10)$ and its inverse, freeing qubit 9. Line starting with "T" reports the T count, line starting with "$T3C$" reports the $CNOT$ count when only $RTOF^3$ are being used, line starting with "$T4C$" reports the $CNOT$ count when $RTOF^4$ are allowed, and line starting with "H" reports the Hadamard gate count. □

We summarize the results in Table I and compare them against best known. The names of the columns are self-explanatory. Observe that [14] features multiple control Toffoli implementations using $12n - 34$ two-qubit gates over a circuit with $n - 3$ ancillae. In comparison, our implementation uses $8n - 20$ CNOT gates over a circuit with only $\lceil \frac{n-3}{2} \rceil$ ancillae. It is furthermore interesting to highlight that in terms of implementing a multiple control Toffoli gate the cost of moving away from using unrestricted ancillae to ancillae residing in the state $|0\rangle$ is only one T gate, but in terms of the CNOTs, it is a noticeable term, $2n - 8$.

V. OPEN PROBLEMS

The problem of systematically synthesizing and analyzing multiple control relative phase Toffoli implementations—both unrestricted as well as the special form, is important to address next. The results of such a search could be used directly to optimize implementations of the multiple control Toffoli gates, arithmetic parts of quantum algorithms, and reversible circuits.

How efficient may a relative phase multiple control Toffoli gate implementation be? In the 3-qubit case the answer is: it requires at least 3 CNOTs as a circuit over CNOT and any single-qubit gates library, as otherwise, per Corollary 11, we would come to a contradiction with any lower CNOT gate count [13]. If it is established that the Toffoli gate requires 7 T gates in the presence of ancillae, a similar argument can be applied towards showing that any relative phase Toffoli gate requires at least 4 T gates as a circuit over Clifford+T library.

The reported constructions obtain best solutions simultaneously for two circuit cost metrics arising from different considerations, the CNOT-count and the T-count. It may be that this is not a coincidence. Is there a relation between these two resource counts?

VI. CONCLUSION

In this paper, we reported an approach for systematic optimization of quantum circuits via replacing suitable pairs of the multiple control Toffoli gates with their relative phase implementations. This operation preserves the functional correctness. However, since the relative phase Toffolis are easier to implement than their regular counterparts, the advantage can be witnessed through the optimized resource counts. We have furthermore illustrated the advantage via optimizing and, when applicable, explaining the nature of best known implementations of the multiple control Toffoli gates. Our demonstrated optimizations include a simultaneous optimization of the T count by a factor of $\frac{4}{3}$ in the leading constant, the CNOT count by a factor 2 in the leading constant, and the number of ancillary qubits by a factor of 2 in the leading constant. The above refers to the optimization of the circuit implementing the multiple control Toffoli gate using arbitrary ancillae, whose construction resulted from employing the relative phase Toffoli gates.

ACKNOWLEDGEMENTS

I wish to thank anonymous reviewers for their useful comments.

Circuit diagrams were drawn using qcircuit.tex package, http://physics.unm.edu/CQuIC/Qcircuit/.

This material was based on work supported by the National Science Foundation, while working at the Foundation. Any opinion, finding, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the National Science Foundation.

Gate	Source	Optimization goal	# T	# CNOT	# H	# P/Z	# ancillae	Ancillae type
TOF^4	[10]	T	15	35	6	3	1	$\|0\rangle$
	[7]	T	15	14	6	0	1	$\|0\rangle$
	Ours	T, CNOT	15	12	6	0	1	$\|0\rangle$
	[10]	T	16	54	6	6	1	$\|x\rangle$
	cc-iX [12]	CNOT	22	20	8	0	1	$\|x\rangle$
	Ours	T, CNOT	16	14	6	0	1	$\|x\rangle$
TOF^5	[10]	T	23	63	10	6	2	$\|00\rangle$
	[7]	T	23	22	10	0	2	$\|00\rangle$
	Ours	T, CNOT	23	18	10	0	1	$\|0\rangle$
	[10]	T	28	90	10	13	2	$\|xx\rangle$
	cc-iX [12]	CNOT	38	36	16	0	2	$\|xx\rangle$
	Ours	T, CNOT	24	20	10	0	1	$\|x\rangle$
TOF^6	[10]	T	31	94	14	9	3	$\|000\rangle$
	[7]	T	31	30	14	0	3	$\|000\rangle$
	Ours	T, CNOT	31	24	14	0	2	$\|00\rangle$
	[10]	T	40	132	14	20	3	$\|xxx\rangle$
	cc-iX [12]	CNOT	46	52	24	0	3	$\|xxx\rangle$
	Ours	T, CNOT	32	28	14	0	2	$\|xx\rangle$
TOF^{11}	[10]	T	71	232	34	24	8	$\|00000000\rangle$
	[7]	T	71	70	34	0	8	$\|00000000\rangle$
	Ours	T, CNOT	71	54	34	0	4	$\|0000\rangle$
	[10]	T	100	328	34	55	8	$\|xxxxxxxx\rangle$
	cc-iX [12]	CNOT	134	132	64	0	8	$\|xxxxxxxx\rangle$
	Ours	T, CNOT	72	68	34	0	4	$\|xxxx\rangle$
$TOF^n, n \geq 5$	[10]	T	8n-17	N/A	N/A	N/A	n-3	$\|00...0\rangle$
	[7]	T	8n-17	8n-18	4n-10	0	n-3	$\|00...0\rangle$
	Ours	T, CNOT	8n-17	6n-12	4n-10	0	$\lceil \frac{n-3}{2} \rceil$	$\|00...0\rangle$
	[10]	T	12n-32	N/A	N/A	N/A	n-3	$\|xx...x\rangle$
	cc-iX [12]	CNOT	16n-42	16n-44	8n-24	0	n-3	$\|xx...x\rangle$
	Ours	T, CNOT	8n-16	8n-20	4n-10	0	$\lceil \frac{n-3}{2} \rceil$	$\|xx...x\rangle$

TABLE I: Optimization of the multiple control Toffoli gates using $RTOF^3$ and $RTOF^4$ gates.

[1] C. Monroe and J. Kim, Science **339**, 1164–1169 (2013).

[2] M. H. Devoret and R. J. Schoelkopf, Science **339**, 1169–1174 (2013).

[3] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. Smolin, and H. Weinfurter, Phys. Rev. A **52**, 3457–3467 (1995).

[4] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, New York (2000).

[5] Helpful Mathematica calculations are available online at http://www.umiacs.umd.edu/~ dmaslov/papers/mathematicacomputations.txt.

[6] A. M. Childs, R. Cleve, E. Deotto, E. Farhi, S. Gutmann, and D. A. Spielman, Proc. 35th ACM STOC, 59–68 (2003).

[7] P. Selinger, Phys. Rev. A **87**, 042302 (2013).

[8] S. Bravyi and A. Kitaev, Phys. Rev. A **71**, 022316 (2005).

[9] A. Sorensen and K. Molmer, Phys. Rev. Lett. **82**, 1971–1974 (1999).

[10] M. Amy, D. Maslov, and M. Mosca, IEEE Trans. CAD **33**(10), 1476–1489 (2014).

[11] G. Song and A. Klappenecker, Quantum Information and Computation **4**, 361–372 (2004).

[12] Quipper 0.5, http://www.mathstat.dal.ca/~ selinger/quipper/doc/frames.html, released September 2013.

[13] V. V. Shende and I. L. Markov, Quantum Information and Computation **9**(5-6), 461–486, (2009).

[14] D. Maslov, G. W. Dueck, D. M. Miller, and C. Negrevergne, IEEE Trans. CAD **27**(3), 436–444 (2008).

Open Quantum Assembly Language

Andrew W. Cross, Lev S. Bishop, John A. Smolin, Jay M. Gambetta

January 10th, 2017

1 Background

Software architectures, compilers, and languages specifically for quantum computing have been studied by the academic community for more than a decade ([1–4] and references therein). Researchers have implemented software and simulators that can be used in practice to study quantum algorithms at many scales. While we cannot survey this work here, we list a few of these projects, several of which include software that has been made readily available: Liquid [5, 6], Scaffold [7, 8], Quipper [9–11], ProjectQ [12, 13], QCL [14, 15], Quiddpro [16, 17], Chisel-q [18, 19], and Quil [20, 21].

Our goal in this document is to describe an interface language for the Quantum Experience that enables experiments with small depth quantum circuits. The language can be generated by the Composer, hand-written, or targeted by higher level software tools, such as those above. Before we do so, we discuss quantum programs in general to provide context. General quantum programs require coordination of quantum and classical parts of the computation. One way to think about general quantum programs is to identify their distinct phases of execution [11]. Fig. 1 shows a high-level diagram of the processes and abstractions involved in specifying a quantum algorithm, transforming the algorithm into executable form, running an experiment or simulation, and analyzing the results. A key idea throughout these processes is the use of intermediate representations. An intermediate representation (IR) of a computation is neither its source language description, nor the target machine instructions, but something in between. Compilers may use several IRs during the process of translating and optimizing a program.

Compilation. This phase takes place on a classical computer in a setting where specific problem parameters are not yet known and no interaction with the quantum computer is required, i.e. it is offline. The input is source code describing a quantum algorithm and any compile time parameters. The output is a combined quantum/classical program expressed using a high level IR. During this phase, it is possible to compile classical procedures into object code and make initial passes that do not require complete knowledge of the problem parameters.

Circuit generation. This takes place on a classical computer in an environment where specific problem parameters are now known, and some interaction with the quantum computer may occur, i.e. this is an online phase. The input is a quantum/classical program expressed using a high level IR, as well as all remaining problem parameters. The output

is a collection of quantum circuits, or quantum basic blocks, together with associated classical control instructions and classical object code needed at run-time. A basic block is a straight-line code sequence with no branches (except at the entry and exit points). Since feedback can occur on multiple time scales, the quantum circuits may include instructions for fast feedback. Other classical control instructions outside of the quantum circuit basic block include, for example, run-time parameter computations and measurement-dependent branches. External classical object code could include algorithms to process measurement outcomes into control flow conditions or results, or to generate new basic blocks on the fly. The output of circuit generation is expressed using a quantum circuit IR. Further circuit generation may occur based on processed measurement results.

Execution. This takes place on physical quantum computer controllers in a real-time environment, i.e. the quantum computer is active. The input is a collection of quantum circuits and associated run-time control statements expressed using a quantum circuit IR. The input is processed by a high-level controller into a stream of real-time instructions in a low-level format that corresponds to physical operations. These are executed on a low-level controller, and a corresponding results stream provides measurement data back to the high-level controller when needed. In general, the high level controller (or virtual machine) can execute classical control instructions and external object code. The output of circuit execution is a collection of processed measurement results returned from the high-level controller.

Post-processing. This takes place on a classical computer after all real-time processing is complete. The input is a collection of processed measurement results, and the output is intermediate results for further circuit generation and/or the final result of the quantum computation.

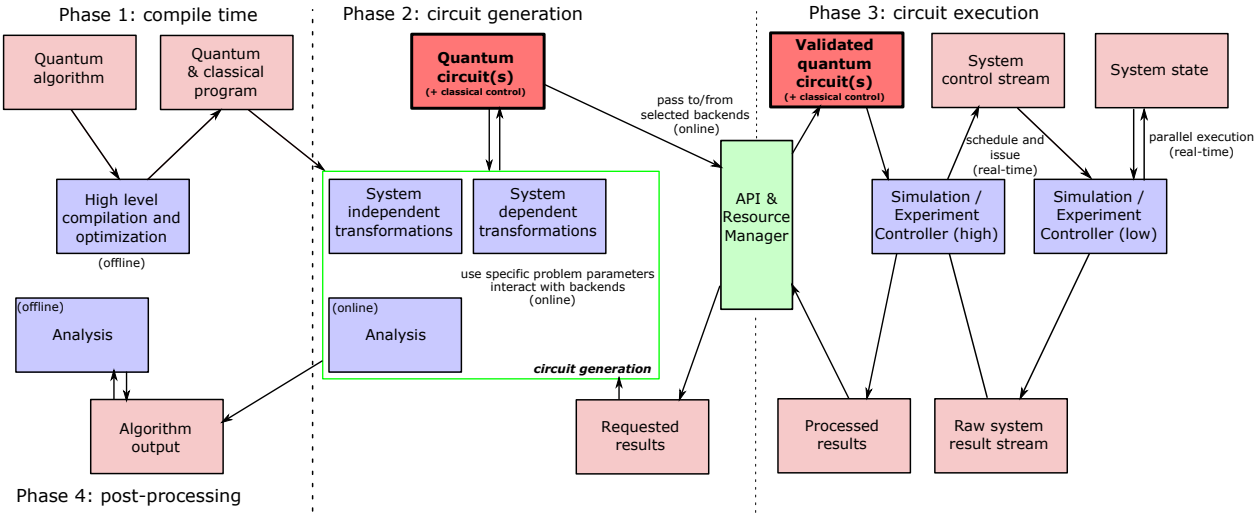


Figure 1: Block diagrams of processes (blue) and abstractions (red) to transform and execute a quantum algorithm. The emphasized quantum circuit abstraction is the main focus of this document. The API and Resource Manager (green) represents the gateway to backend processes for circuit execution. Dashed vertical lines separate offline, online, and real-time processes.

Our model of program execution on the Quantum Experience does not allow fully general classical computations in the loop with quantum computations, as described above, because qubits remain coherent for a limited time. Quantum programs are broken into distinct circuits whose quantum outputs cannot be carried over into the next circuit. Classical computation is done between quantum circuit executions. Users actively participate in the circuit generation phase and manually implement part of feedback path through the high level controller in Fig. 1, observing outcomes from the previous quantum circuit and choosing the next quantum circuit to execute. Making use of an API to the execution phase, users can write their own software for compilation and circuit generation that interacts with the hardware over a sequence of quantum circuit executions. After obtaining all of the processed results, users may post-process the data offline.

We specify part of a quantum circuit intermediate representation based on the quantum circuit model, a standard formalism for quantum computation [22]. The quantum circuit abstraction is emphasized in Fig. 1. The IR expresses quantum circuits with fast feedback, such as might constitute the basic blocks of a full-featured IR. A basic block is a straight-line code sequence with no branches (except at the entry and exit points). We have chosen to include statements that are essential for near-term experiments and that we believe will be present in any future IR. The representation will be quite familiar to experts.

The human-readable form of our quantum circuit IR is based on "quantum assembly language" [3, 23–26] or QASM (pronounced *kazm*). QASM is a simple text language that describes generic quantum circuits. QASM can represent a completely unrolled quantum program whose parameters have all been specified. Most QASM variants assume a discrete set of quantum gates, but our IR is designed to control a physical system with a parameterized gate set. While we use the term "quantum assembly language", this is merely an analogy and should not be taken too far.

Open QASM represents universal physical circuits, so we propose a built-in gate basis of arbitrary single-qubit gates and a two-qubit entangling gate (CNOT) [27]. We choose a simple language without higher level programming primitives. We define different gate sets using a subroutine-like mechanism that hierarchically specifies new unitary gates in terms of built-in gates and previously defined gate subroutines. In this way, the built-in basis is used to define hardware-supported operations via standard header files. The subroutine mechanism allows limited code reuse by hierarchically defining more complex operations [7, 26]. We also add instructions that model a quantum-classical interface, specifically measurement, state reset, and the most elemental classical feedback.

The remaining sections of this document specify Open QASM and provide examples.

2 Language

The syntax of the human-readable form of Open QASM has elements of C and assembly languages. The first (non-comment) line of an Open QASM program must be `OPENQASM M.m;` indicating a major version M and minor version m. Version 2.0 is described in this document. The version keyword cannot occur multiple times in a file. Statements are separated by semicolons. Whitespace is ignored. The language is case sensitive. Comments begin with a pair of forward slashes and end with a new line. The statement `include`

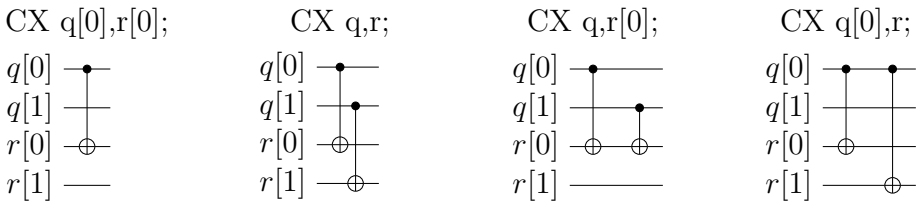


Figure 2: The built-in two-qubit entangling gate is the controlled-NOT gate. If `a` and `b` are qubits, the statement `CX a,b;` applies a controlled-NOT (CNOT) gate that flips the target qubit `b` iff the control qubit `a` is one. If `a` and `b` are quantum registers, the statement applies CNOT gates between corresponding qubits of each register. There is a similar meaning when `a` is a qubit and `b` is a quantum register and vice versa.

Figure 3: The single-qubit unitary gates are built in. These gates are parameterized by three real parameters θ, ϕ, and λ. If the argument `q` is a quantum register, the statement applies `size(q)` gates in parallel to the qubits of the register.

`"filename";` continues parsing `filename` as if the contents of the file were pasted at the location of the `include` statement. The path is specified relative to the current working directory.

The only storage types of Open QASM (version 2.0) are classical and quantum registers, which are one-dimensional arrays of bits and qubits, respectively. The statement `qreg name[size];` declares an array of qubits (quantum register) with the given name and size. Identifiers, such as `name`, must start with a lowercase letter and can contain alpha-numeric characters and underscores. The label `name[j]` refers to a qubit of this register, where $j \in \{0, 1, \ldots, \text{size(name)} - 1\}$. The qubits are initialized to $|0\rangle$. Likewise, `creg name[size];` declares an array of bits (register) with the given name and size. The label `name[j]` refers to a bit of this register, where $j \in \{0, 1, \ldots, \text{size(name)} - 1\}$. The bits are initialized to 0.

The built-in universal gate basis is "CNOT + $U(2)$". There is one built-in two-qubit gate (Fig. 2)

$$\text{CNOT} := \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{1}$$

called the controlled-NOT gate. The statement `CX a,b;` applies a CNOT gate that flips the target qubit `b` if and only if the control qubit `a` is one. The arguments cannot refer to the same qubit. Built-in gates have reserved uppercase keywords. If `a` and `b` are quantum registers *with the same size*, the statement means apply `CX a[j], b[j];` for each index `j`

into register `a`. If instead, `a` is a qubit and `b` is a quantum register, the statement means apply `CX a, b[j];` for each index `j` into register `b`. Finally, if `a` is a quantum register and `b` is a qubit, the statement means apply `CX a[j], b;` for each index `j` into register `a`.

All of the single-qubit unitary gates are also built in (Fig. 3) and parameterized as

$$U(\theta,\phi,\lambda) := R_z(\phi)R_y(\theta)R_z(\lambda) = \begin{pmatrix} e^{-i(\phi+\lambda)/2}\cos(\theta/2) & -e^{-i(\phi-\lambda)/2}\sin(\theta/2) \\ e^{i(\phi-\lambda)/2}\sin(\theta/2) & e^{i(\phi+\lambda)/2}\cos(\theta/2) \end{pmatrix}. \quad (2)$$

Here $R_y(\theta) = \exp(-i\theta Y/2)$ and $R_z(\phi) = \exp(-i\theta Z/2)$. This specifies any element of $SU(2)$. When `a` is a quantum register, the statement `U(theta,phi,lambda) a;` means apply `U(theta,phi,lambda) a[j];` for each index `j` into register `a`. The real parameters $\theta \in [0, 4\pi)$, $\phi \in [0, 4\pi)$, and $\lambda \in [0, 4\pi)$ are given by *parameter expressions* constructed using in-fix notation. These support scientific calculator features with arbitrary precision real numbers[1]. For example, `U(pi/2,0,pi) q[0];` applies a Hadamard gate to qubit `q[0]`. Open QASM (version 2.0) does not provide a mechanism for computing parameters based on measurement outcomes.

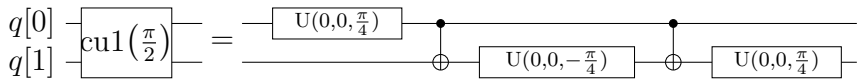
New gates can be defined as unitary subroutines using the built-in gates, as shown in Fig. 4. These can be viewed as macros whose expansion we defer until run-time. Gates are defined by statements of the form

```
// comment
gate name(params) qargs
{
  body
}
```

where the optional parameter list `params` is a comma-separated list of variable parameter names, and the argument list `qargs` is a comma-separated list of qubit arguments. Both the parameter names and qubit arguments are identifiers. If there are no variable parameters, the parentheses are optional. At least one qubit argument is required. The first comment may contain documentation, such as TeX markup, to be associated with the gate. The arguments in `qargs` cannot be indexed within the body of the gate definition.

```
// this is ok:
gate g a
{
  U(0,0,0) a;
}
// this is invalid:
gate g a
{
  U(0,0,0) a[0];
}
```

[1] Features include scientific notation; real arithmetic; logarithmic, trigonometic, and exponential functions; square roots; and the built-in constant π. The Quantum Experience uses a double precision floating point type for real numbers.




```
gate cu1(lambda) a,b
{
  U(0,0,theta/2) a;
  CX a,b;
  U(0,0,-theta/2) b;
  CX a,b;
  U(0,0,theta/2) b;
}
cu1(pi/2) q[0],q[1];
```

Figure 4: New gates are defined as unitary subroutines. The gates are applied using the statement `name(params) qargs;` just like the built-in gates. The parentheses are optional if there are no parameters. The gate cu1(θ) corresponds to the unitary matrix $\mathrm{diag}(1, 1, 1, e^{i\theta})$ up to a global phase.

Only built-in gate statements, calls to previously defined gates, and barrier statements can appear in `body`. The statements in the body can only refer to the symbols given in the parameter or argument list, and these symbols are scoped only to the subroutine body. An empty body corresponds to the identity gate. Subroutines must be declared before use and cannot call themselves. The statement `name(params) qargs;` applies the subroutine, and the variable parameters `params` are given as parameter expressions. The gate can be applied to any combination of qubits and quantum registers *of the same size* as shown in the following example. The quantum circuit given by

```
gate g qb0,qb1,qb2,qb3
{
  // body
}
qreg qr0[1];
qreg qr1[2];
qreg qr2[3];
qreg qr3[2];
g qr0[0],qr1,qr2[0],qr3; // ok
g qr0[0],qr2,qr1[0],qr3; // error!
```

has a second-to-last line that means

for $j \leftarrow 0, 1$ **do**
    `g qr0[0],qr1[j],qr2[0],qr3[j];`

We provide this so that user-defined gates can be applied in parallel like the built-in gates.

To support gates whose physical implementation may be possible, but whose definition is unspecified, we provide an "opaque" gate declaration. This may be used in practice in several instances. For example, the system may evolve under some fixed but uncharacterized

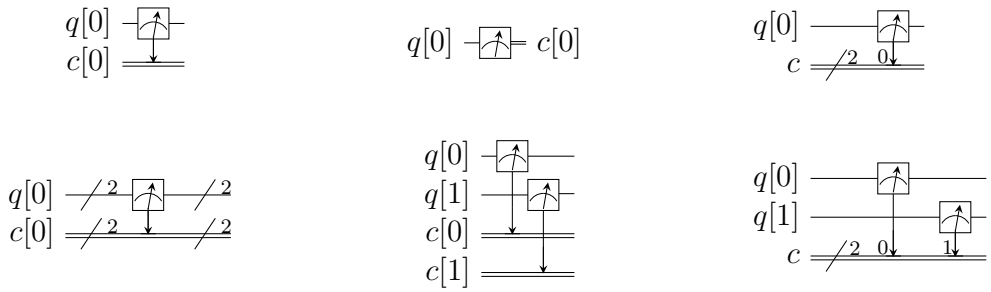


Figure 5: The `measure` statement projectively measures a qubit or each qubit of a quantum register. The measurement projects onto the Z-basis and leaves qubits available for further operations. The top row of circuits depicts single-qubit measurement using the statement `measure q[0] -> c[0];` while the bottom depicts measurement of an entire register using the statement `measure q -> c;`. The center circuit of the top row depicts measurement as the final operation on `q[0]`.

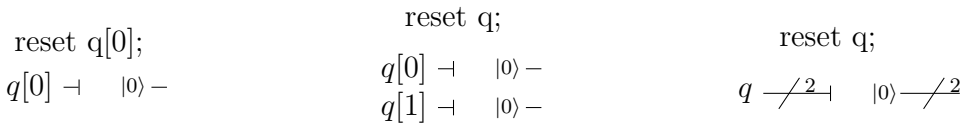


Figure 6: The `reset` statement prepares a qubit or quantum register in the state $|0\rangle$.

drift Hamiltonian for some fixed amount of time. The system might be subject to an n-qubit operator whose parameters are computationally challenging to estimate. The syntax for an opaque gate declaration is the same as a gate declaration but without a body.

Measurement is shown in Fig. 5. The statement `measure qubit|qreg -> bit|creg;` measures the qubit(s) in the Z-basis and records the measurement outcome(s) by overwriting the bit(s). Measurement corresponds to a projection onto one of the eigenstates of Z, and qubit(s) are immediately available for further quantum computation. Both arguments must be register-type, or both must be bit-type. If both arguments are register-type and have the same size, the statement `measure a -> b;` means apply `measure a[j] -> b[j];` for each index `j` into register `a`.

The `reset qubit|qreg;` statement resets a qubit or quantum register to the state $|0\rangle$. This corresponds to a partial trace over those qubits (i.e. discarding them) before replacing them with $|0\rangle\langle 0|$, as shown in Fig. 6.

There is one type of classically-controlled quantum operation: the `if` statement shown in Fig. 7. The `if` statement conditionally executes a quantum operation based on the value of a classical register. This allows measurement outcomes to determine future quantum operations. We choose to have one decision register for simplicity. This register is interpreted as an integer, using the bit at index zero as the low order bit. The quantum operation executes only if the register has the given integer value. Only quantum operations, i.e. built-in gates, gate (and opaque) subroutines, preparation, and measurement, can be prefaced by `if`. A quantum program with a parameter that depends on values that are known only

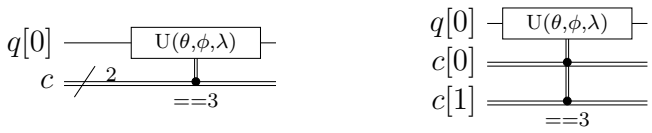


Figure 7: The `if` statement applies a quantum operation only if a classical register has the indicated integer value. These circuits depict the statement `if(c==3) U(theta,phi,lambda) q[0];`.

at run-time can be rewritten using a sequence of `if` statements. Specifically, for a single-parameter gate with n bits of precision, we may choose to write 2^n statements, only one of which is executed, or we can decompose the parameterized gate into a sequence of n conditional gates.

The `barrier` instruction prevents optimizations from reordering gates across its source line. For example,

```
CX r[0],r[1];
h q[0];
h s[0];
barrier r,q[0];
h s[0];
CX r[1],r[0];
CX r[0],r[1];
```

will prevent an attempt to combine the CNOT gates but will allow the pair of `h s[0];` gates to cancel.

Open QASM statements are summarized in Table 1. The grammar is presented in Appendix A.

Table 1: Open QASM language statements (version 2.0)

Statement	Description	Example
`OPENQASM 2.0;`	Denotes a file in Open QASM format[a]	`OPENQASM 2.0;`
`qreg name[size];`	Declare a named register of qubits	`qreg q[5];`
`creg name[size];`	Declare a named register of bits	`creg c[5];`
`include "filename";`	Open and parse another source file	`include "qelib1.inc";`
`gate name(params) qargs { body }`	Declare a unitary gate	(see text)
`opaque name(params) qargs;`	Declare an opaque gate	(see text)
`// comment text`	Comment a line of text	`// oops!`
`U(theta,phi,lambda) qubit|qreg;`	Apply built-in single qubit gate(s)[b]	`U(pi/2,2*pi/3,0) q[0];`
`CX qubit|qreg,qubit|qreg;`	Apply built-in CNOT gate(s)	`CX q[0],q[1];`
`measure qubit|qreg -> bit|creg;`	Make measurement(s) in Z basis	`measure q -> c;`
`reset qubit|qreg;`	Prepare qubit(s) in $\|0\rangle$	`reset q[0];`
`gatename(params) qargs;`	Apply a user-defined unitary gate	`crz(pi/2) q[1],q[0];`
`if(creg==int) qop;`	Conditionally apply quantum operation	`if(c==5) CX q[0],q[1];`
`barrier qargs;`	Prevent transformations across this source line	`barrier q[0],q[1];`

[a] This must appear as the first non-comment line of the file.

[b] The parameters `theta`, `phi`, and `lambda` are given by *parameter expressions*; see text and Appendix A.

3 Examples

This section gives several examples of quantum circuits expressed in Open QASM (version 2.0). The circuits use a gate basis defined for the Quantum Experience.

3.1 Quantum Experience standard header

The Quantum Experience standard header defines the gates that are implemented by the hardware, gates that appear in the Quantum Experience composer, and a hierarchy of additional user-defined gates. Our approach is to define physical gates that the hardware implements in terms of the abstract gates `U` and `CX`. The current physical gates supported by the Quantum Experience are a superset of the abstract gates, but this is not true of all physical gate sets and devices. Choosing to use abstract gates merely to define physical gates gives some flexibility to add or change physical gates at a later time without changing Open QASM. We believe this approach is preferable to invisibly compiling abstract gates to physical gates or to changing the underlying set of abstract gates whenever the hardware changes.

The Quantum Experience currently implements the controlled-NOT gate via the cross-resonance interaction and implements three distinct types of single-qubit gates. The one-parameter gate

$$u_1(\lambda) := \mathrm{diag}(1, e^{i\lambda}) \sim U(0, 0, \lambda) = R_z(\lambda) \tag{3}$$

changes the phase of a carrier without applying any pulses. The symbol "$\sim$" denotes equivalence up to a global phase. The gate

$$u_2(\phi, \lambda) := U(\pi/2, \phi, \lambda) = R_z(\phi + \frac{\pi}{2}) R_x(\pi/2) R_z(\lambda - \frac{\pi}{2}) \tag{4}$$

uses a single $\pi/2$-pulse. The most general single-qubit gate

$$u_3(\theta, \phi, \lambda) := U(\theta, \phi, \lambda) = R_z(\phi + 3\pi) R_x(\pi/2) R_z(\theta + \pi) R_x(\pi/2) R_z(\lambda) \tag{5}$$

uses a pair of $\pi/2$-pulses.

```
// Quantum Experience (QE) Standard Header
// file: qelib1.inc

// --- QE Hardware primitives ---

// 3-parameter 2-pulse single qubit gate
gate u3(theta,phi,lambda) q { U(theta,phi,lambda) q; }
// 2-parameter 1-pulse single qubit gate
gate u2(phi,lambda) q { U(pi/2,phi,lambda) q; }
// 1-parameter 0-pulse single qubit gate
gate u1(lambda) q { U(0,0,lambda) q; }
// controlled-NOT
```

```
gate cx c,t { CX c,t; }
// idle gate (identity)
gate id a { U(0,0,0) a; }

// --- QE Standard Gates ---

// Pauli gate: bit-flip
gate x a { u3(pi,0,pi) a; }
// Pauli gate: bit and phase flip
gate y a { u3(pi,pi/2,pi/2) a; }
// Pauli gate: phase flip
gate z a { u1(pi) a; }
// Clifford gate: Hadamard
gate h a { u2(0,pi) a; }
// Clifford gate: sqrt(Z) phase gate
gate s a { u1(pi/2) a; }
// Clifford gate: conjugate of sqrt(Z)
gate sdg a { u1(-pi/2) a; }
// C3 gate: sqrt(S) phase gate
gate t a { u1(pi/4) a; }
// C3 gate: conjugate of sqrt(S)
gate tdg a { u1(-pi/4) a; }

// --- Standard rotations ---
// Rotation around X-axis
gate rx(theta) a { u3(theta,-pi/2,pi/2) a; }
// rotation around Y-axis
gate ry(theta) a { u3(theta,0,0) a; }
// rotation around Z axis
gate rz(phi) a { u1(phi) a; }

// --- QE Standard User-Defined Gates  ---

// controlled-Phase
gate cz a,b { h b; cx a,b; h b; }
// controlled-Y
gate cy a,b { sdg b; cx a,b; s b; }
// controlled-H
gate ch a,b {
h b; sdg b;
cx a,b;
h b; t b;
cx a,b;
t b; h b; s b; x b; s a;
}
```

```
// C3 gate: Toffoli
gate ccx a,b,c
{
  h c;
  cx b,c; tdg c;
  cx a,c; t c;
  cx b,c; tdg c;
  cx a,c; t b; t c; h c;
  cx a,b; t a; tdg b;
  cx a,b;
}
// controlled rz rotation
gate crz(lambda) a,b
{
  u1(lambda/2) b;
  cx a,b;
  u1(-lambda/2) b;
  cx a,b;
}
// controlled phase rotation
gate cu1(lambda) a,b
{
  u1(lambda/2) a;
  cx a,b;
  u1(-lambda/2) b;
  cx a,b;
  u1(lambda/2) b;
}
// controlled-U
gate cu3(theta,phi,lambda) c, t
{
  // implements controlled-U(theta,phi,lambda) with  target t and control c
  u1((lambda-phi)/2) t;
  cx c,t;
  u3(-theta/2,0,-(phi+lambda)/2) t;
  cx c,t;
  u3(theta/2,phi,0) t;
}
```

3.2 Quantum teleportation

Quantum teleportation (Fig. 8) demonstrates conditional application of future gates based on prior measurement outcomes.

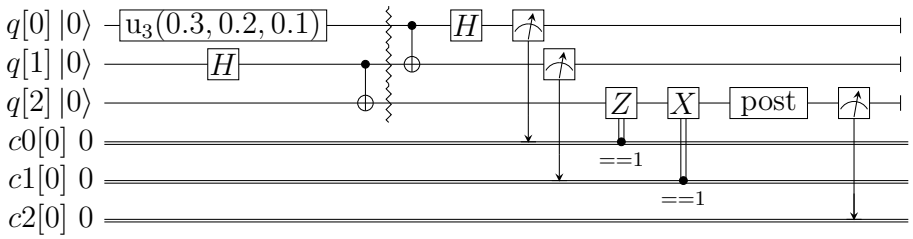


Figure 8: Example of quantum teleportation. Qubit `q[0]` is prepared by `U(0.3,0.2,0.1) q[0];` and teleported to `q[2]`.

```
// quantum teleportation example
OPENQASM 2.0;
include "qelib1.inc";
qreg q[3];
creg c0[1];
creg c1[1];
creg c2[1];
// optional post-rotation for state tomography
gate post q { }
u3(0.3,0.2,0.1) q[0];
h q[1];
cx q[1],q[2];
barrier q;
cx q[0],q[1];
h q[0];
measure q[0] -> c0[0];
measure q[1] -> c1[0];
if(c0==1) z q[2];
if(c1==1) x q[2];
post q[2];
measure q[2] -> c2[0];
```

3.3 Quantum Fourier transform

The quantum Fourier transform (QFT, Fig. 9) demonstrates parameter passing to gate subroutines. This circuit applies the QFT to the state $|q_0q_1q_2q_3\rangle = |1010\rangle$ and measures in the computational basis.

```
// quantum Fourier transform
OPENQASM 2.0;
include "qelib1.inc";
qreg q[4];
creg c[4];
x q[0];
```

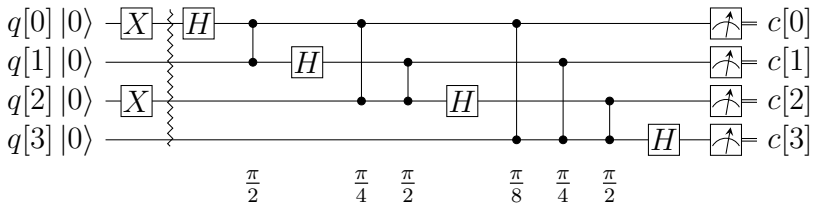


Figure 9: Example of a 4-qubit quantum Fourier transform. The circuit applies the QFT to $|1010\rangle$ and measures in the computational basis. The output is read in reverse order `c[3], c[2], c[1], c[0]`.

```
x q[2];
barrier q;
h q[0];
cu1(pi/2) q[1],q[0];
h q[1];
cu1(pi/4) q[2],q[0];
cu1(pi/2) q[2],q[1];
h q[2];
cu1(pi/8) q[3],q[0];
cu1(pi/4) q[3],q[1];
cu1(pi/2) q[3],q[2];
h q[3];
measure q -> c;
```

3.4 Inverse QFT followed by measurement

If the qubits are all measured after the inverse QFT, the measurement commutes with the controls of the `cu1` gates, and those gates can be replaced by classically-controlled single qubit rotations (see for example Figure 3.3 in [28]). The example demonstrates how to implement this classical control using conditional gates.

```
// QFT and measure, version 1
OPENQASM 2.0;
include "qelib1.inc";
qreg q[4];
creg c[4];
h q;
barrier q;
h q[0];
measure q[0] -> c[0];
if(c==1) u1(pi/2) q[1];
h q[1];
measure q[1] -> c[1];
```

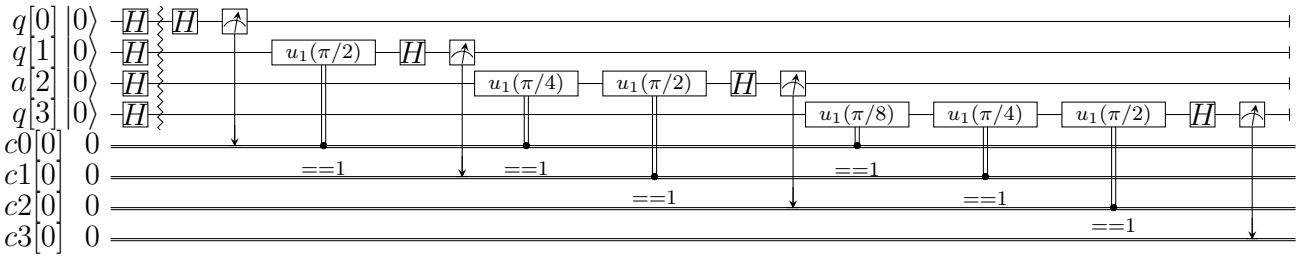


Figure 10: Example of a 4-qubit inverse quantum Fourier transform followed by measurement. In this case, the measurement commutes with the controls of the `cu1` gates and can be rewritten as shown (see Figure 3.3 in [28]). The circuit applies the inverse QFT to the uniform superposition and measures in the computational basis.

```
if(c==1) u1(pi/4) q[2];
if(c==2) u1(pi/2) q[2];
if(c==3) u1(pi/2+pi/4) q[2];
h q[2];
measure q[2] -> c[2];
if(c==1) u1(pi/8) q[3];
if(c==2) u1(pi/4) q[3];
if(c==3) u1(pi/4+pi/8) q[3];
if(c==4) u1(pi/2) q[3];
if(c==5) u1(pi/2+pi/8) q[3];
if(c==6) u1(pi/2+pi/4) q[3];
if(c==7) u1(pi/2+pi/4+pi/8) q[3];
h q[3];
measure q[3] -> c[3];
```

Alternatively, we can decompose the rotations and apply them using fewer statements but more quantum gates. The corresponding circuit for this example is shown in Fig. 10.

```
// QFT and measure, version 2
OPENQASM 2.0;
include "qelib1.inc";
qreg q[4];
creg c0[1];
creg c1[1];
creg c2[1];
creg c3[1];
h q;
barrier q;
h q[0];
measure q[0] -> c0[0];
if(c0==1) u1(pi/2) q[1];
```

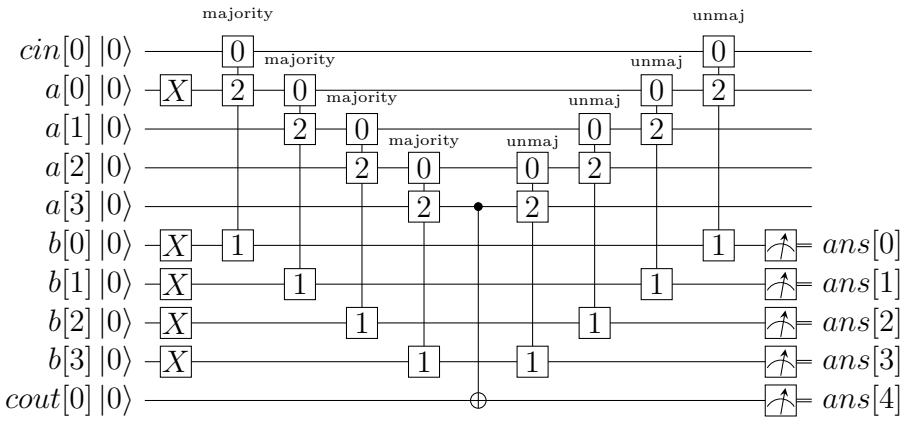


Figure 11: Example of a quantum ripple-carry adder from [29]. This circuit prepares $a = 1$, $b = 15$ and computes the sum into `b` with an output carry `cout[0]`.

```
h q[1];
measure q[1] -> c1[0];
if(c0==1) u1(pi/4) q[2];
if(c1==1) u1(pi/2) q[2];
h q[2];
measure q[2] -> c2[0];
if(c0==1) u1(pi/8) q[3];
if(c1==1) u1(pi/4) q[3];
if(c2==1) u1(pi/2) q[3];
h q[3];
measure q[3] -> c3[0];
```

3.5 Ripple-carry adder

The ripple-carry adder [29] shown in Fig. 11 exhibits hierarchical use of gate subroutines.

```
// quantum ripple-carry adder from Cuccaro et al, quant-ph/0410184
OPENQASM 2.0;
include "qelib1.inc";
gate majority a,b,c
{
  cx c,b;
  cx c,a;
  ccx a,b,c;
}
gate unmaj a,b,c
{
  ccx a,b,c;
  cx c,a;
```

```
  cx a,b;
}
qreg cin[1];
qreg a[4];
qreg b[4];
qreg cout[1];
creg ans[5];
// set input states
x a[0]; // a = 0001
x b;    // b = 1111
// add a to b, storing result in b
majority cin[0],b[0],a[0];
majority a[0],b[1],a[1];
majority a[1],b[2],a[2];
majority a[2],b[3],a[3];
cx a[3],cout[0];
unmaj a[2],b[3],a[3];
unmaj a[1],b[2],a[2];
unmaj a[0],b[1],a[1];
unmaj cin[0],b[0],a[0];
measure b[0] -> ans[0];
measure b[1] -> ans[1];
measure b[2] -> ans[2];
measure b[3] -> ans[3];
measure cout[0] -> ans[4];
```

3.6 Randomized benchmarking

A complete randomized benchmarking experiment could be described by a high level program. After passing through the upper phases of compilation, the program consists of many quantum circuits and associated classical control. Benchmarking is a particularly simple example because there is no data dependence between these quantum circuits.

Each circuit is a sequence of random Clifford gates composed from a set of basic gates (Fig. 12 uses the gate set `h`, `s`, `cz`, and Paulis). If the gate set differs from the built-in gate set, new gates can be defined using the `gate` statement. Each of the randomly-chosen Clifford gates is separated from prior and future gates by barrier instructions to prevent the sequence from simplifying to the identity as a result of subsequent transformations.

```
// One randomized benchmarking sequence
OPENQASM 2.0;
include "qelib1.inc";
qreg q[2];
creg c[2];
h q[0];
barrier q;
```

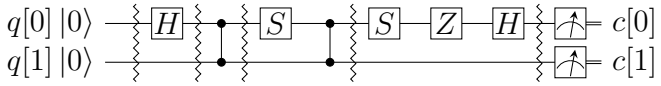


Figure 12: Example of a two-qubit randomized benchmarking (RB) sequence over the basis $\langle H, S, CZ, X, Y, Z\rangle$. Barriers separate the implementations of each Clifford gate. An RB experiment consists of many sequences. Each sequence runs some number of times ("shots").

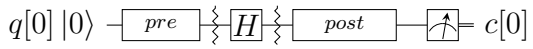


Figure 13: Example of a single-qubit quantum process tomography circuit. The `pre` and `post` gates are described by a higher-level program that generates intermediate code containing several independent circuits. Each circuit is executed some number of times ("shots") to compute statistics from which the `h` gate process is reconstructed. Barriers separate the process under study from the pre- and post- gates.

```
cz q[0],q[1];
barrier q;
s q[0];
cz q[0],q[1];
barrier q;
s q[0];
z q[0];
h q[0];
barrier q;
measure q -> c;
```

3.7 Quantum process tomography

As in randomized benchmarking, a high-level program describes a quantum process tomography (QPT) experiment. Each program compiles to intermediate code with several independent quantum circuits that can each be described using Open QASM (version 2.0). Fig. 13 shows QPT of a Hadamard gate. Each circuit is identical except for the definitions of the `pre` and `post` gates. The empty definitions in the current example are placeholders that define identity gates. For textbook QPT, the `pre` and `post` gates are both taken from the set $\{I, H, SH\}$ to prepare $|0\rangle$, $|+\rangle$, and $|+i\rangle$ and measure in the Z, X, and Y basis.

```
OPENQASM 2.0;
include "qelib1.inc";
gate pre q { }   // pre-rotation
gate post q { }  // post-rotation
qreg q[1];
creg c[1];
```

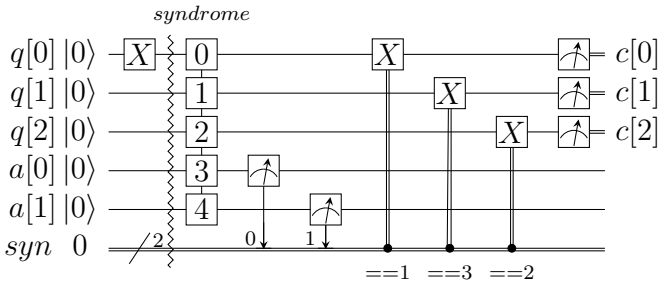


Figure 14: Example of a quantum bit-flip repetition code. The circuit begins with the (classical) encoded state $|000\rangle$, applies an error on `q[0]`, and uses feedback on a syndrome measurement to correct the error.

```
pre q[0];
barrier q;
h q[0];
barrier q;
post q[0];
measure q[0] -> c[0];
```

3.8 Quantum error-correction

This example of the 3-bit quantum repetition code (Fig. 14) demonstrates how Open QASM (version 2.0) can express simple quantum error-correction circuits.

```
// Repetition code syndrome measurement
OPENQASM 2.0;
include "qelib1.inc";
qreg q[3];
qreg a[2];
creg c[3];
creg syn[2];
gate syndrome d1,d2,d3,a1,a2
{
  cx d1,a1; cx d2,a1;
  cx d2,a2; cx d3,a2;
}
x q[0]; // error
barrier q;
syndrome q[0],q[1],q[2],a[0],a[1];
measure a -> syn;
if(syn==1) x q[0];
if(syn==2) x q[2];
if(syn==3) x q[1];
measure q -> c;
```

A Open QASM Grammar

⟨mainprogram⟩ ⊨ `OPENQASM` ⟨real⟩ `;` ⟨program⟩

⟨program⟩ ⊨ ⟨statement⟩ | ⟨program⟩ ⟨statement⟩

⟨statement⟩ ⊨ ⟨decl⟩
| ⟨gatedecl⟩ ⟨goplist⟩ `}`
| ⟨gatedecl⟩ `}`
| `opaque` ⟨id⟩ ⟨idlist⟩ `;`
| `opaque` ⟨id⟩ `(` `)` ⟨idlist⟩ `;` | `opaque` ⟨id⟩ `(` ⟨idlist⟩ `)` ⟨idlist⟩ `;`
| ⟨qop⟩
| `if` `(` ⟨id⟩ `==` ⟨nninteger⟩ `)` ⟨qop⟩
| `barrier` ⟨anylist⟩ `;`

⟨decl⟩ ⊨ `qreg` ⟨id⟩ `[` ⟨nninteger⟩ `]` `;` | `creg` ⟨id⟩ `[` ⟨nninteger⟩ `]` `;`

⟨gatedecl⟩ ⊨ `gate` ⟨id⟩ ⟨idlist⟩ `{`
| `gate` ⟨id⟩ `(` `)` ⟨idlist⟩ `{`
| `gate` ⟨id⟩ `(` ⟨idlist⟩ `)`⟨idlist⟩ `{`

⟨goplist⟩ ⊨ ⟨uop⟩
| `barrier` ⟨idlist⟩ `;`
| ⟨goplist⟩ ⟨uop⟩
| ⟨goplist⟩ `barrier` ⟨idlist⟩ `;`

⟨qop⟩ ⊨ ⟨uop⟩
| `measure` ⟨argument⟩ `-` `>` ⟨argument⟩ `;`
| `reset` ⟨argument⟩ `;`

⟨uop⟩ ⊨ `U` `(` ⟨explist⟩ `)` ⟨argument⟩ `;`
| `CX` ⟨argument⟩ `,` ⟨argument⟩ `;`
| ⟨id⟩ ⟨anylist⟩ `;` | ⟨id⟩ `(` `)` ⟨anylist⟩ `;`
| ⟨id⟩ `(` ⟨explist⟩ `)` ⟨anylist⟩ `;`

⟨anylist⟩ ⊨ ⟨idlist⟩ | ⟨mixedlist⟩

⟨idlist⟩ ⊨ ⟨id⟩ | ⟨idlist⟩ `,` ⟨id⟩

⟨mixedlist⟩ ⊨ ⟨id⟩ `[` ⟨nninteger⟩ `]` | ⟨mixedlist⟩ `,` ⟨id⟩
| ⟨mixedlist⟩ `,` ⟨id⟩ `[` ⟨nninteger⟩ `]`
| ⟨idlist⟩ `,` ⟨id⟩`[` ⟨nninteger⟩ `]`

⟨argument⟩ ⊨ ⟨id⟩ | ⟨id⟩ `[` ⟨nninteger⟩ `]`

⟨explist⟩ ⊨ ⟨exp⟩ | ⟨explist⟩ `,` ⟨exp⟩

⟨exp⟩ ⊨ ⟨real⟩ | ⟨nninteger⟩ | `pi` | ⟨id⟩
| ⟨exp⟩ `+` ⟨exp⟩ | ⟨exp⟩ `-` ⟨exp⟩ | ⟨exp⟩ `*` ⟨exp⟩
| ⟨exp⟩ `/` ⟨exp⟩ | `-` ⟨exp⟩ | ⟨exp⟩ `^` ⟨exp⟩
| `(` ⟨exp⟩ `)` | ⟨unaryop⟩ `(` ⟨exp⟩ `)`

⟨unaryop⟩ ⊨ `sin` | `cos` | `tan` | `exp` | `ln` | `sqrt`

This is a simplified grammar for Open QASM presented in Backus-Naur form. The unlisted productions ⟨id⟩, ⟨real⟩ and ⟨nninteger⟩ are defined by the regular expressions:

```
id         := [a-z][A-Za-z0-9_]*
real       := ([0-9]+\.[0-9]*|[0-9]*\.[0-9]+)([eE][-+]?[0-9]+)?
nninteger := [1-9]+[0-9]*|0
```

Not all programs produced using this grammar are valid Open QASM circuits. As explained in Section 2, there are additional rules concerning valid arguments, parameters, declarations, and identifiers, as well as the standard operator precedence rules in the parameter expressions.

References

[1] P. Selinger. A brief survey of quantum programming languages. *Proc. Seventh Int'l Symp. Functional and Logic Programming*, pages 1–6, 2004.

[2] S. Gay. Quantum programming languages: survey and bibliography. *Math. Structures in Computer Science*, 16:581–600, 2006.

[3] K. Svore, A. Cross, A. Aho, I. Chuang, and I. Markov. A layered software architecture for quantum computing design tools. *IEEE Computer*, (39(1)):74–83, 2006.

[4] T. Häner, D. Steiger, K. Svore, and M. Troyer. A software methodology for compiling quantum programs. *arxiv:1604.01401*, 2016.

[5] D. Wecker and K. Svore. LIQ$Ui|\rangle$: A software design architecture and domain-specific language for quantum computing. *arXiv:1402.4467*, 2014.

[6] LIQ$Ui|\rangle$: The Language Integrated Quantum Operations Simulator. http://stationq.github.io/Liquid/, 2016. accessed November 2016.

[7] A. JavadiAbhari, S. Patil, D. Kudrow, J. Heckey, A. Lvov, F. Chong, and M. Martonosi. ScaffCC: a framework for compilation and analysis of quantum computing programs. *ACM International Conference on Computing Frontiers (CF 2014)*, 2014.

[8] Compilation, analysis and optimization framework for the Scaffold quantum programming language. https://github.com/epiqc/ScaffCC, 2016. accessed November 2016.

[9] B. Valiron, N. Ross, P. Selinger, D. Scott Alexander, and J. Smith. Programming the quantum future. *Communications of the ACM*, 58(8):52–61, 2015.

[10] The Quipper Language. http://www.mathstat.dal.ca/~selinger/quipper/, 2013. accessed November 2016.

[11] A. S. Green, P. LeFanu Lumsdaine, N. J. Ross, P. Selinger, and B. Valiron. Quipper: a scalable quantum programming language. *ACM SIGPLAN Notices*, (48(6)):333–342, 2013.

arXiv:2504.20291v1 [quant-ph] 28 Apr 2025

Quantum Gate Decomposition

A Study of Compilation Time vs. Execution Time Trade-offs

Evandro C. R. Rosa
evandro.crr@posgrad.ufsc.br
0000-0002-8197-9454
Universidade Federal de Santa Catarina FlorianópolisSanta Catarina Brazil

Jerusa Marchi
0000-0002-4864-3764
jerusa.marchi@ufsc.br
Universidade Federal de Santa Catarina FlorianópolisSanta Catarina Brazil

Eduardo I. Duzzioni
0000-0002-8971-2033
eduardo.duzzioni@ufsc.br
Universidade Federal de Santa Catarina FlorianópolisSanta Catarina Brazil

Rafael de Santiago
0000-0001-7033-125X
r.santiago@ufsc.br
Universidade Federal de Santa Catarina FlorianópolisSanta Catarina Brazil

Abstract.

Similar to classical programming, high-level quantum programming languages generate code that cannot be ex-

Ket quantum programming platform and collecting numerical performance data. This is the first study to both implement and analyze the current state-of-the-art decomposition methods within a single platform. Based on our findings, we propose two compilation profiles: one optimized for minimizing compilation time and another for minimizing quantum execution time. Our results provide valuable insights for both quantum compiler developers and quantum programmers, helping them make informed decisions about gate decomposition strategies and their impact on overall performance.

Quantum Computing, Quantum Programming, Quantum Compiler, Gade Decomposition, Ket

†

1. Introduction

For programming languages and platforms that treat the quantum bit (qubit) as a first-class citizen in quantum programming (Svore et al., 2018; Da Rosa and De Santiago, 2022), coding a quantum application involves invoking functions known as quantum gates. Quantum gates have no side effects on the classical state of the program; they only affect the quantum state and can induce superposition and entanglement (Nielsen and Chuang, 2010). Another class of functions, known as measurements, is used to extract information from the quantum state, returning it as classical data while causing the collapse of the quantum state, *i.e.*, destroying the superposition.

Figure 1(a) illustrates a simple quantum program written in Python using the Ket quantum programming platform (Da Rosa and De Santiago, 2022). This code implements the Grover diffusion operator, a key component of Grover's quantum search algorithm (Grover, 1996), utilizing the Hadamard (H), Pauli-X (X), and Pauli-Z (Z) gates. The term *quantum gate*, or more precisely *quantum logical gate*, is inspired by classical circuit and logic gates. Each quantum code has an equivalent quantum circuit; for example, the circuit in Figure 1(b) is equivalent to the code in Figure 1(a) applied for 4-qubits.

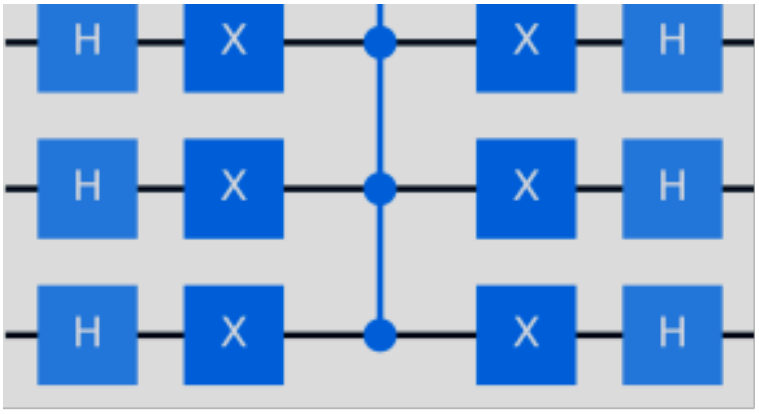
Despite the direct use of quantum bits and quantum gates in quantum programming, which gives the impression of low-level programming, the approach provides feature-rich high-level programming (Rosa et al., 2025), producing operations that cannot be executed directly by a quantum computer and must undergo a compilation process. For example, in Figure 1(b), note that the quantum gate in the middle of the circuit, a multi-controlled Pauli Z gate, encompasses all 4 qubits. This type of multi-qubit gate cannot be executed directly by a quantum computer and therefore must be decomposed into a sequence of one- and two-qubit gates with equivalent effect, as shown in Figure 1(c). This is analogous to a line of high-level classical code being decomposed into several lines of assembly code.

{minted}

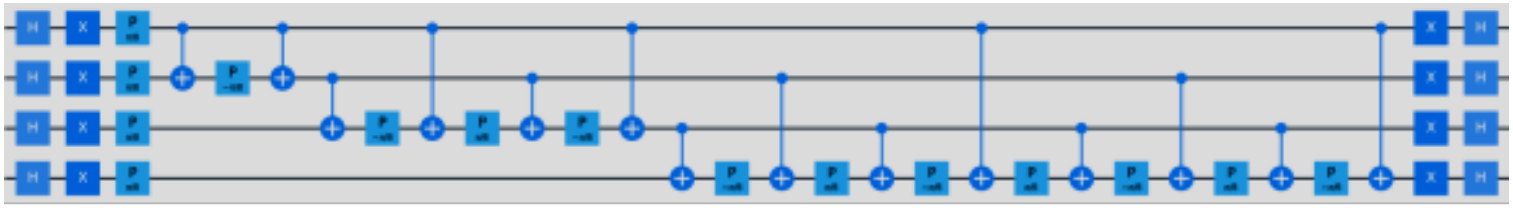
[frame=lines,fontsize=,breaklines]py def diffusion(q : Quant):

with around(cat(H, X), q): with control(q[:-1]): Z(q[-1])

(a) Ket code.



(b) Quantum Circuit.



(c) Decomposed Circuit.

Figure 1. Grover diffusion operation implemented using Ket (LABEL:sub@fig:cirq:diffusion), along with its corresponding high-level (LABEL:sub@fig:cirq:diffusion) and decomposed (LABEL:sub@fig:cirq:diffusion_decompose) quantum circuit for 4 qubits.

\Description

The quantum compilation process can be split into three main parts: quantum gate decomposition (Da Silva and Park, 2022; Iten et al., 2021; Rosa et al., 2025; Vale et al., 2024; Barenco et al., 1995; Iten et al., 2016; Rosa et al., 2024; Zindorf and Bose, 2024), where multi-qubit gates are broken down into a sequence of one- and two-qubit operations; circuit mapping (Chowdhury et al., 2024; Li et al., 2019; Niu et al., 2020; Wille and Burgholzer, 2023; Zhu et al., 2020, 2023), where the logical qubits are mapped to physical qubits that are limited in their connectivity, thereby restricting which qubits can participate in a two-qubit gate; and lastly, a pulse schedule is generated based on the mapped circuit (Huang et al., 2023; Stefanazzi et al., 2022; Alexander et al., 2020). These pulses are then passed to an arbitrary waveform generator (AWG) that physically controls the qubits. Each step of this decomposition process brings the quantum code closer to the hardware execution and becomes more hardware-dependent.

tum programming.

We implemented state-of-the-art quantum gate decomposition algorithms in Ket's quantum compiler, Libket, and evaluated the compilation and execution times of each algorithm. There is no definitive answer as to whether prioritizing execution time over compilation time, or *vice-versa*, is preferable; the best choice depends on the specifications of the classical and quantum hardware. The objective of this paper is to provide a basis for helping quantum compiler developers and programmers make informed decisions.

The main contributions of this paper are:

A comprehensive survey of quantum gate decomposition algorithms, including optimizations beyond those proposed by the original authors.

Implementation of all surveyed algorithms in the Ket quantum compiler–marking the first unified implementation on a single platform and enabling consistent benchmarking.

A classification of the algorithms into two categories, forming the basis for two quantum compilation profiles: one focused on minimizing compilation time, the other on reducing quantum execution time.

The remainder of this paper is organized as follows. Section 2 introduces classical and quantum runtime, highlighting the implications for quantum program compilation. Section 3 examines high-level quantum programming and shows how multi-qubit gates naturally arise in such code. Section 4 then surveys the state-of-the-art decomposition algorithms used to break down these gates into executable instructions. Section 5 follows with a performance evaluation of these algorithms, focusing on CNOT count and circuit depth. Based on the benchmark data, Section 6 analyzes the trade-offs involved and introduces two practical quantum compilation profiles. Finally, Section 7 presents our conclusions and final remarks.

2. Classical and Quantum Runtime

A quantum program is not entirely quantum; rather, it is a classical-quantum program. Classical processing is always required at least to prepare the quantum circuit inputs and process the quantum execution outputs. The quantum computer can be seen as an accelerated processing unit, similar to an FPGA or GPU, where the CPU manages the execution. This implies that a quantum application has two distinct runtimes: a classical runtime, which includes all program execution, and a quantum runtime, which occurs within it, with a program potentially initiating several quantum runtimes.

For instance, in Figure 1(a), the function `diffusion` takes a list of qubits as input. In classical compilation, this would typically generate a loop whose size depends on the input list. However, for quantum compilation, the exact number of qubits must be known in advance to generate the correct quantum circuit. This requirement means the quantum compiler must be invoked dynamically at classical runtime–just before executing the quantum code.

The performance of a quantum application is evaluated by the number of two-qubit gates in the compiled quantum circuit. This two-qubit gate is typically a CNOT gate[1]

[1]All analyses in this paper are also valid if CZ gates are used instead.

. Single-qubit gates are generally not counted, as sequences of such gates can often be merged into a single equivalent operation. The time required to execute quantum code on a quantum computer is directly related to the circuit depth. Since gates acting on independent qubits can be executed in parallel, the execution time may be shorter than the total number of CNOT gates. For this reason, circuit depth serves as a metric for quantum execution time. Conversely, the number of CNOTs impacts compilation time, as each CNOT represents a computational step for the quantum compiler.

Since the compilation of the quantum circuit occurs at classical runtime, the trade-off between compilation time and quantum execution time directly impacts the overall classical-quantum execution time. This trade-off can also be viewed as a balance between classical computation (compilation) and quantum execution time.

3. High-Level Quantum Programming

While quantum programming introduces unique challenges–such as the impossibility of copying quantum data due to the no-cloning theorem (Wootters and Zurek, 1982)–it also offers powerful abstractions that simplify the development of quantum applications. One such feature is the ability to automatically invoke the inverse of a quantum operation. In this paper, we examine quantum programming through the lens of Ket[2]

[2]https://quantumket.org

, an open-source quantum programming platform with a Python API (Da Rosa and De Santiago, 2022).

At its core, Ket provides a minimal yet expressive set of quantum gates: the Pauli gates (X, Y, and Z); rotation gates (RX, RY, and RZ); the Phase gate (P); and the Hadamard gate (H). These gates are sufficient to prepare a

By adding just a controlled NOT[3]

[3]Also known as the controlled Pauli X or simply the CNOT gate.

to the basic set of quantum gates, universal quantum computation becomes achievable. In Ket, the CNOT gate is provided by a function of the same name, but it can also be constructed using the function `ctrl` as

```
lambda c, t: ctrl(c, X)(t),
```

or by using the "`with control`" construction. Any quantum gate or function that contains quantum gates–as in the one shown in Figure 1(a)–can be called with control qubits. This flexibility to append control qubits to any quantum gate greatly enhances high-level quantum programming by enabling the easy construction of complex operations from basic quantum gates.

A controlled quantum gate call is the quantum analog of the classical `if` statement, where a gate is applied to the target qubits only if the control qubits are all in the state $|1\rangle$. The key difference is that a controlled operation acts on the superposition and has the ability to create entanglement.

The code in Figure 2 illustrates how controlled operations can appear in high-level quantum programming. The function `prepare` takes a list of qubits alongside a list of measurement probabilities r_k and a list of phase values θ_k, and prepares the qubits in the state $\sum_k \sqrt{r_k}\, e^{i\theta_k} |k\rangle$. This function calls itself recursively, adding a control qubit at each recursion. Therefore, even though there is no direct call for controlled RY or P gates, the execution of this code creates several controlled operations.

{minted}

[frame=lines,fontsize=,breaklines]py def prepare(qubits: Quant, prob: ParamTree — list[float], amp: list[float] — None = None,): if not isinstance(prob, ParamTree): prob = ParamTree(prob, amp) head, *tail = qubits RY(prob.value, head) if prob.is_leaf(): with around(X, head): P(prob.phase0, head) return P(prob.phase1, head) with around(X, head): ctrl(head, prepare)(tail, prob.left) ctrl(head, prepare)(tail, prob.right)

Figure 2. Arbitrary quantum state preparation algorithm implemented using Ket. For the implementation of the `ParamTree`, see reference Rosa et al. (2025, Figure 10a).

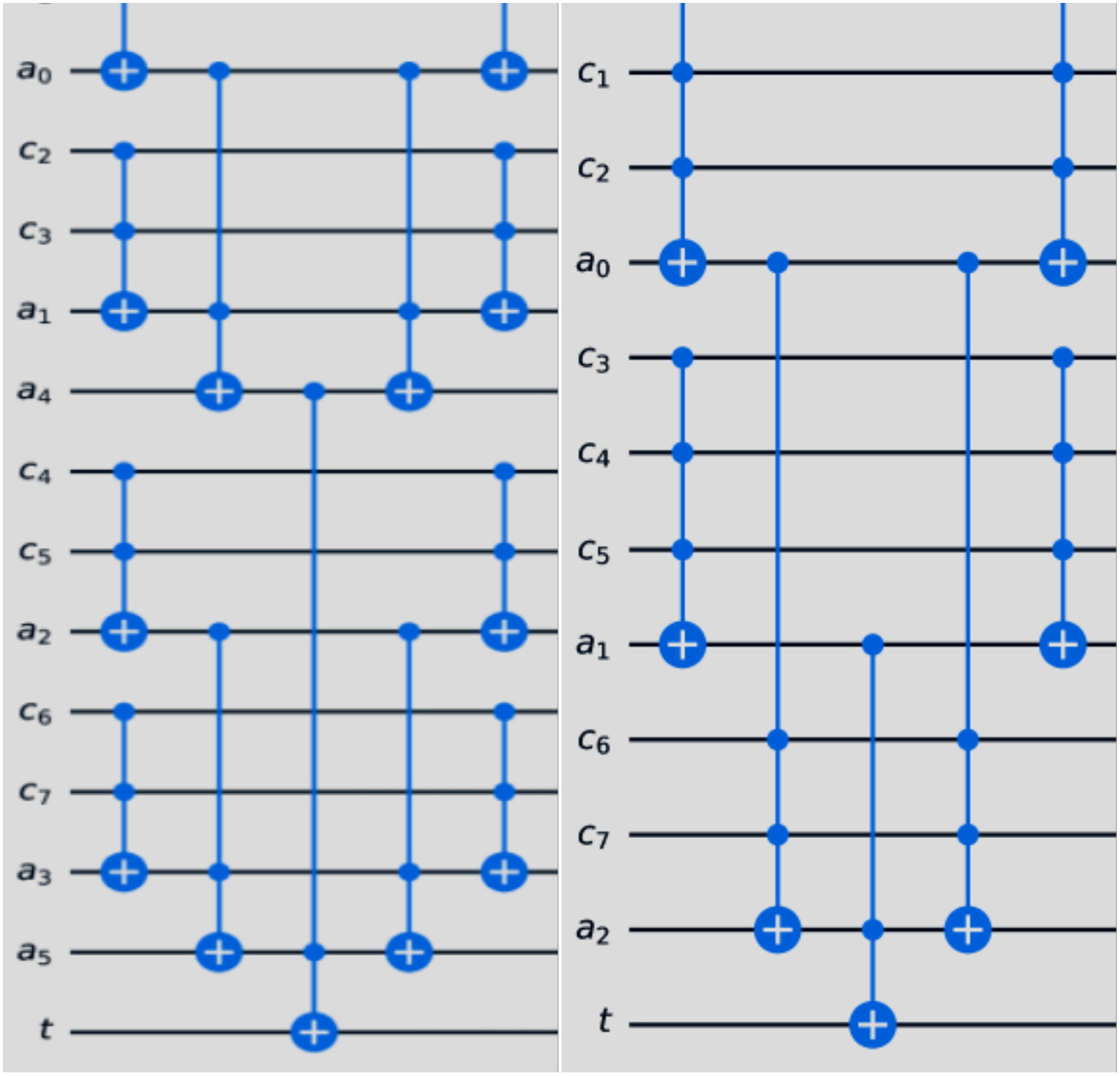
\Description

This construction also allows the compiler to remove some controlled gate calls, for the code from Figure 2, those associated with the X gate (Rosa et al., 2025).

4. Decomposition Algorithms

Table 1. Gate decomposition algorithms evaluated in this paper. *No. Aux* denotes the number of auxiliary qubits required, and *Aux State* specifies their state. *Depth*, *No. CNOTs*, and *No. Aux* are reported for an n-controlled gate decomposition.

Gates	Algorithms	Mode	No. Aux	Aux State	Depth	No. CNOTs
Pauli Gates	V Chain	C2X	$n-2$	Clean	$4n$	$6n$
				Dirty	$8n$	$8n$
		C3X	$\lceil \frac{n-3}{2} \rceil$	Clean	$6n$	$6n$
				Dirty	$12n$	$12n$
	Single Aux	Linear	1	Clean/Dirty	$8n$	$12n$
		Log	1	Clean	$O(\log n)^3$	$O(n \log n)^4$
				Dirty	$O(\log n)^3$	$O(n \log n)^4$
Rotation Gates	SU(2)	Linear	0	N/A	$8n$	$12n$
		Log	0	N/A	$O(\log n)^3$	$O(n \log n)^4$
Phase and Hadamard	SU(2) Rewrite		1	Clean	$8n$	$12n$
	Single Aux U(2)		1	Clean	$O(\log n)^3$	$O(n \log n)^4$
U(2)	Network	C2X	$\approx n$	Clean	$6 \log_2(n)$	$6n$
		C3X	$\approx \frac{n}{2}$	Clean	$8 \log_2(n)$	$6n$
	Liner Depth		0	N/A	$16n$	$n^2/10$



(a) Network C2X.

(b) Network C3X.

Figure 3. Network decomposition for 8-controlled Pauli X.

\Description

these gates can be applied to the others with a simple basis change on the target qubit; (ii) Rotation Gates (RX, RY, and RZ), these belong to the special unitary group SU(2); (iii) Phase and Hadamard Gates (P and H), these belong to the broader unitary group U(2)[4]

[4]All single-qubit gates belong to U(2), but some, like the rotation gates, are also in SU(2), a subgroup of U(2).

.

Table 1 summarizes state-of-the-art quantum gate decomposition algorithms. Some of these algorithms require *auxiliary qubits*, which are additional qubits beyond those directly involved in the controlled gate. Depending on the algorithm, auxiliary qubits must either be initialized in the 0 state (Clean auxiliary) or can be in any state (Dirty auxiliary). Regardless of their initial state, auxiliary qubits must be restored to their original state after decomposition so that they can be reused elsewhere and do not create new entanglements. Typically, decompositions using clean qubits are more efficient. The Ket compiler automatically manages the allocation and usage of auxiliary qubits when available (Rosa et al., 2024).

Table 1 also presents algorithm variations, such as C2X and C3X, which use approximations of 2-controlled and 3-controlled NOT gates (Maslov, 2016), respectively. Additionally, some algorithms are labeled *Linear* or *Log*, which are algorithms that share similar auxiliary requirements.

The quantum circuit depth and the number of CNOTs presented in Table 1 were obtained by fitting the curves of the benchmark data. In cases where a good curve fit was not possible, we present the complexity as stated by the authors in big-O notation.

In the following subsections, we discuss the decomposition algorithms required for each class of quantum gates. In Section 5, we present a performance comparison of the decomposition algorithms.

4.1. Pauli Gates

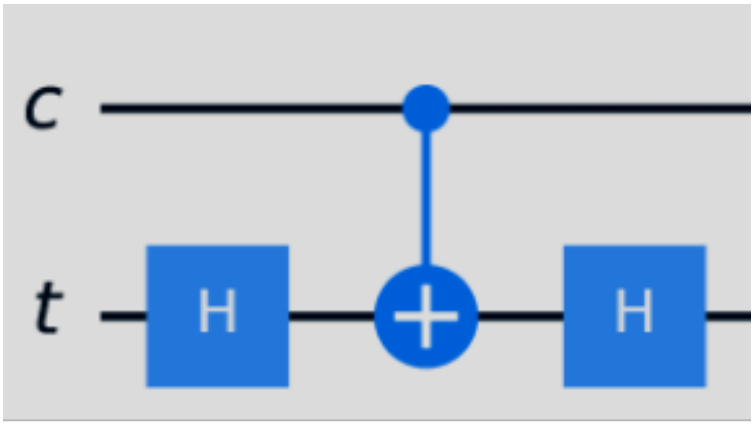
The Pauli gates are among the most commonly used quantum gates in algorithm design. A key insight for gate decomposition is that the decomposition of one Pauli gate can be adapted for the others. For example, in Figure 4(a), the Y and Z gates are defined using the X gate with a basis change. Consequently, the circuits for their controlled versions, shown in Figures 4(b) and 4(c), rely on a controlled X gate. This example explicitly illustrates the behavior of Ket's compiler, which automatically applies this principle: only the decomposition of the Pauli X gate is explicitly defined, while the other Pauli gates are implemented through basis changes. In this paper, we evaluate only the performance of the X gate decomposition, as the decomposition of the other Pauli gates requires only an additional two or four single-qubit gates.

(b) myCY Circuit.

{minted}

[frame=lines,fontsize=,breaklines]py def myY(q): HS = cat(H, S) with around(HS, q): X(q) def myZ(q): with around(H, q): X(q) def myCY(c, t): ctrl(c, myY)(t) def myCZ(c, t): ctrl(c, myZ)(t)

(a) Ket code.



(c) myCZ Circuit.

Figure 4. Defining the gates Y and Z using the X gate and a basis change, allowing the controlled X gate to implement the controlled versions of these gates.

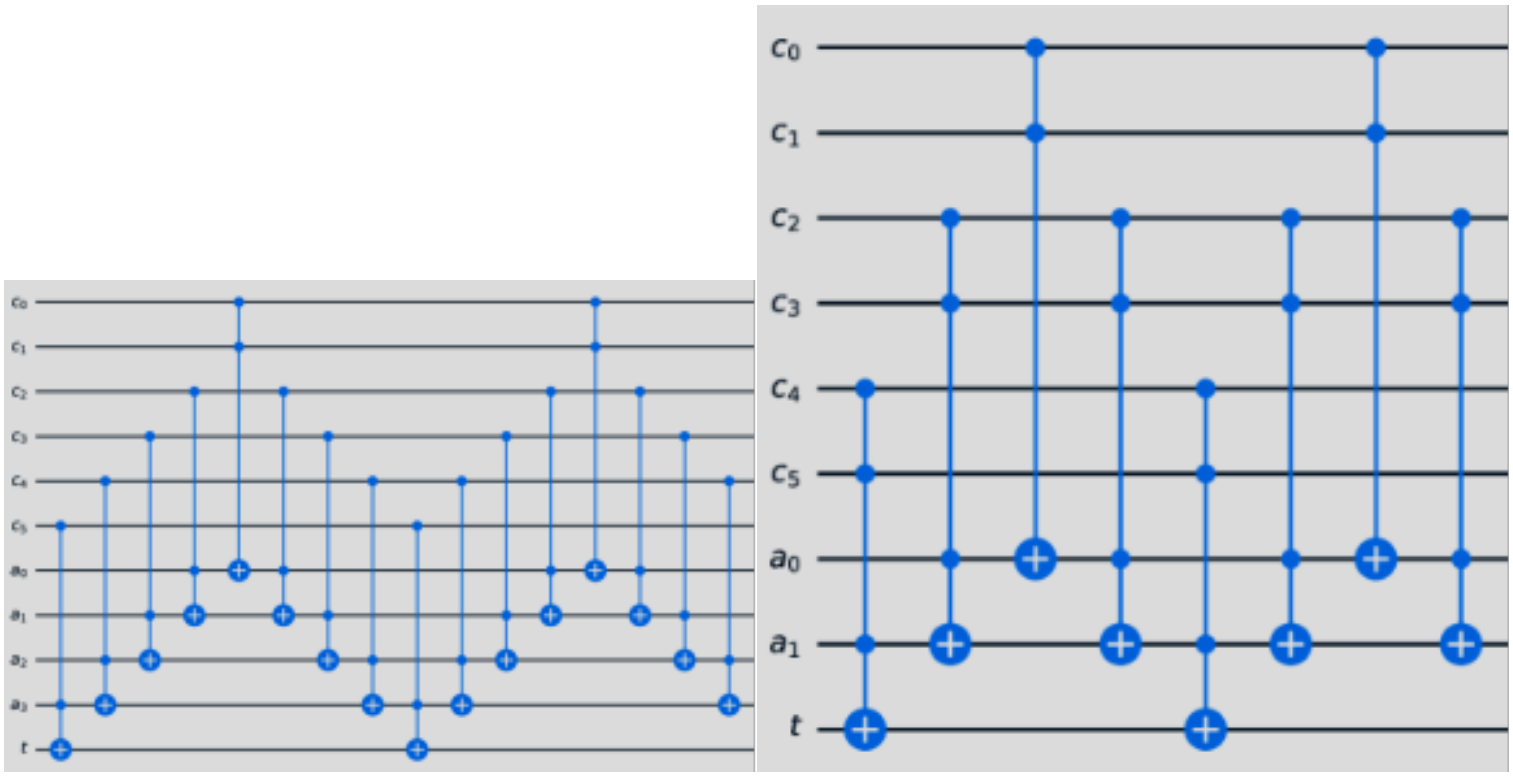
\Description

Network

The Network decomposition (Nielsen and Chuang, 2010, p. 183) is one of the most efficient algorithms and can be applied to any multi-controlled gate. Figure 3 illustrates two variants of the algorithm for a 8-controlled Pauli X gate. Note that, in addition to the target and control qubits, this decomposition requires auxiliary qubits, all initialized to the 0 state. For Pauli gate decomposition, we can reduces the number of auxiliary qubits by one or two compared to other gates.

This decomposition results in a circuit depth $O \log n$ for an n-controlled gate when organizing the gates as presented by Maslov (2016, Corollary 5). Additionally, for the decomposition of 2- and 3-controlled X gates, where the original target qubit is not directly affected, approximate versions of this decomposition are applied (Maslov, 2016).

decomposition utilizes approximate 2- or 3-controlled X. Although this algorithm results in a number of CNOT gates similar to the Network decomposition, especially when using clean auxiliary qubits, it produces a circuit with linear depth rather than logarithmic depth. Figure 5 illustrates a V Chain decomposition using dirty auxiliary qubits for a 6-controlled X gate.



(a) V Chain C2X, Dirty.

(b) V Chain C3X, Dirty.

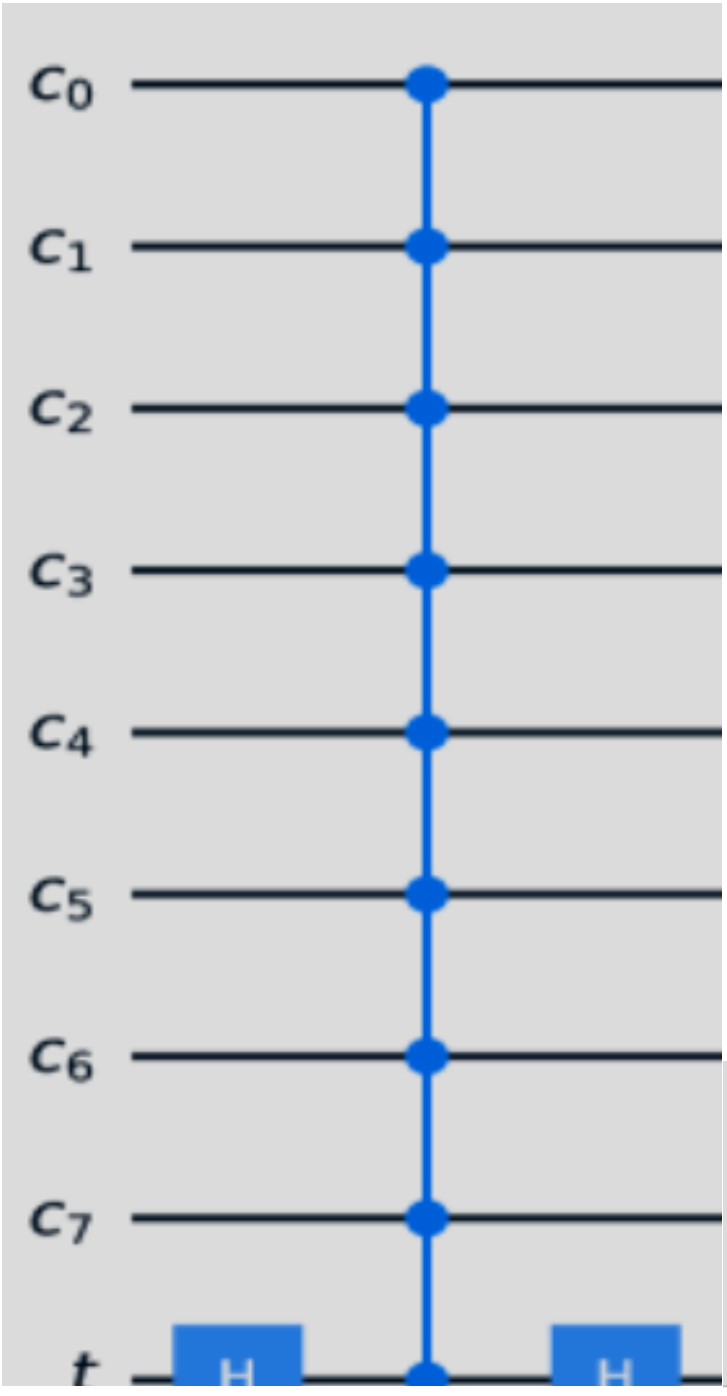
Figure 5. V Chain decomposition for 6-controlled Pauli X.

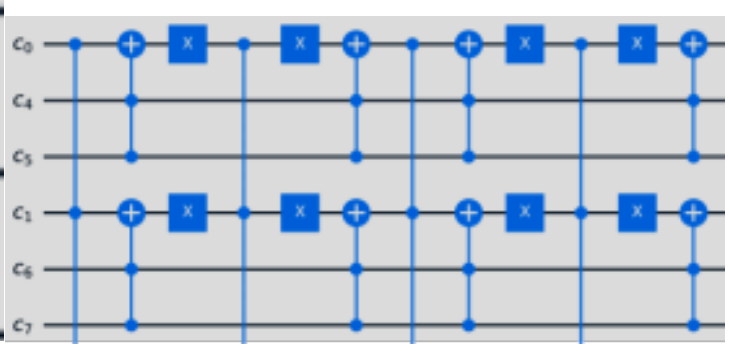
\Description

Single Aux

The Single Aux decomposition algorithms require only a single auxiliary qubit and include two methods: one that results in a quantum circuit with linear depth, proposed by Zindorf and Bose (2024), and another that achieves logarithmic depth, proposed by Claudon et al. (2024).

The Single Aux Log decomposition (Claudon et al., 2024) recursively breaks an n-controlled X gate into 2 n-controlled X gates that can execute in parallel. The base case of the recursion is a 4-controlled X gate, which can be decomposed without auxiliary qubits. Unlike the Linear algorithm, the number of CNOTs and the circuit depth can be reduced when using a clean auxiliary qubit. Figure 6(b) illustrates the Single Aux Log decomposition of an 8-controlled X gate.





(a) Linear, Dirty. (b) Log, Dirty.

Figure 6. Single Aux decomposition for 8-controlled Pauli X.

\Description

Linear Depth

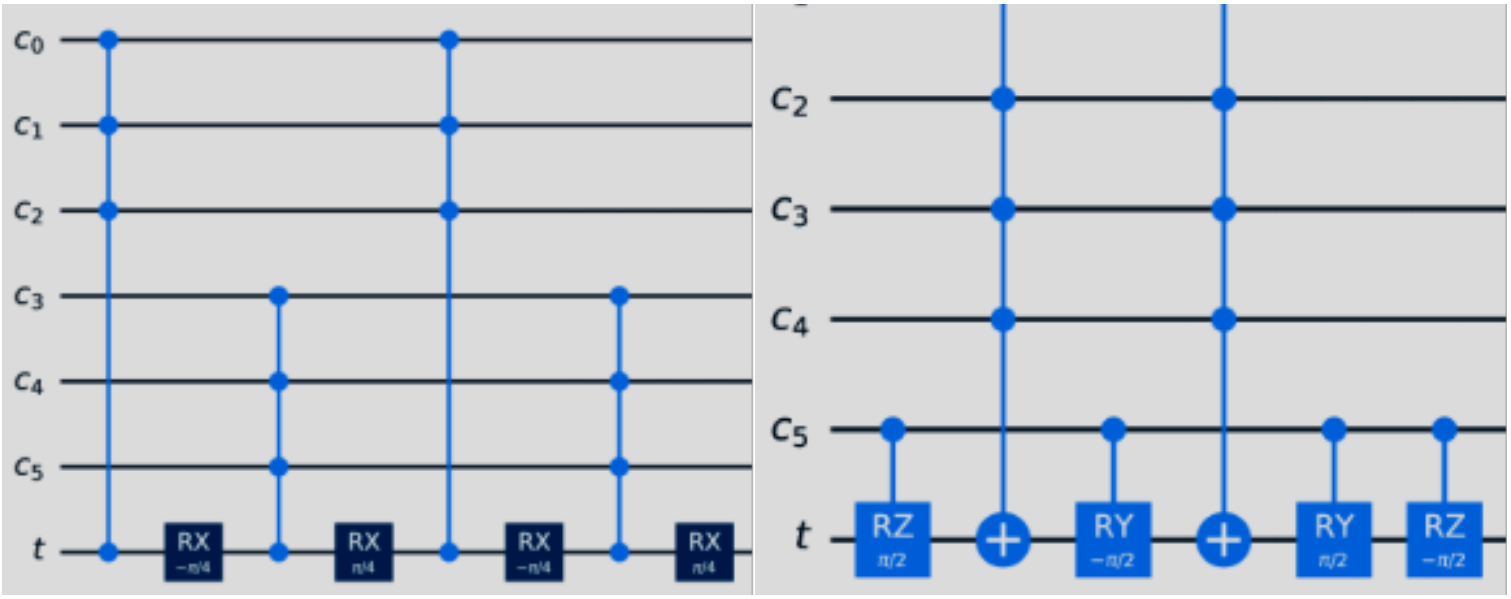
The Linear Depth decomposition proposed by Da Silva and Park (2022), as the name suggests, results in a circuit with linear depth. This is an efficient algorithm that requires no auxiliary qubits and can be used for any quantum gate. We consider this algorithm as the fallback decomposition since it has no requirements to be applied to any U(2) gate. However, this algorithm results in a circuit with a quadratic number of CNOTs.

4.2. Rotation Gates

Alongside the Network decomposition, rotation gates allow for the use of the SU(2) decomposition algorithms, which requires no auxiliary qubits. We categorize these algorithms as SU(2) Linear, proposed by Zindorf and Bose (2024), which is also used for the Single Aux Linear and SU(2) Rewrite decompositions (Zindorf and Bose, 2024; Rosa et al., 2025); and SU(2) Log, which relies on the Single Aux Log decomposition algorithm for Pauli gates (Claudon et al., 2024).

The SU(2) Linear decomposition algorithm uses an approximate version of a multi-controlled Z gate, as illustrated in Figure 7(a). Each multi-controlled Z gate is decomposed into an approximate version where free qubits serve as auxiliaries. The approximate decomposition of the Z gate relies on phase differences canceling out in pairs.

Figure 7(b) illustrates the decomposition using the SU(2) Log algorithm, where each multi-controlled X is decomposed using the Single Aux Log algorithm with a dirty auxiliary, and the single-controlled gates can be decomposed without an auxiliary qubit using just two CNOTs. Figure 7 illustrates the decomposition of a multi-controlled RX(π) gate. For other rotation gates and angles, the multi-controlled gates are arranged in the same structure.



(a) SU(2) Linear.

(b) SU(2) Log.

Figure 7. SU(2) decomposition for 6-controlled RX(π).

\Description

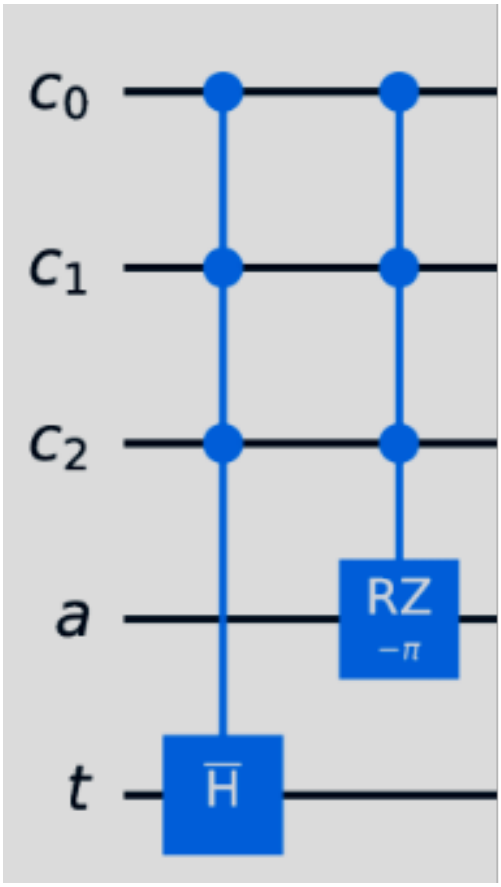
4.3. Phase and Hadamard Gates

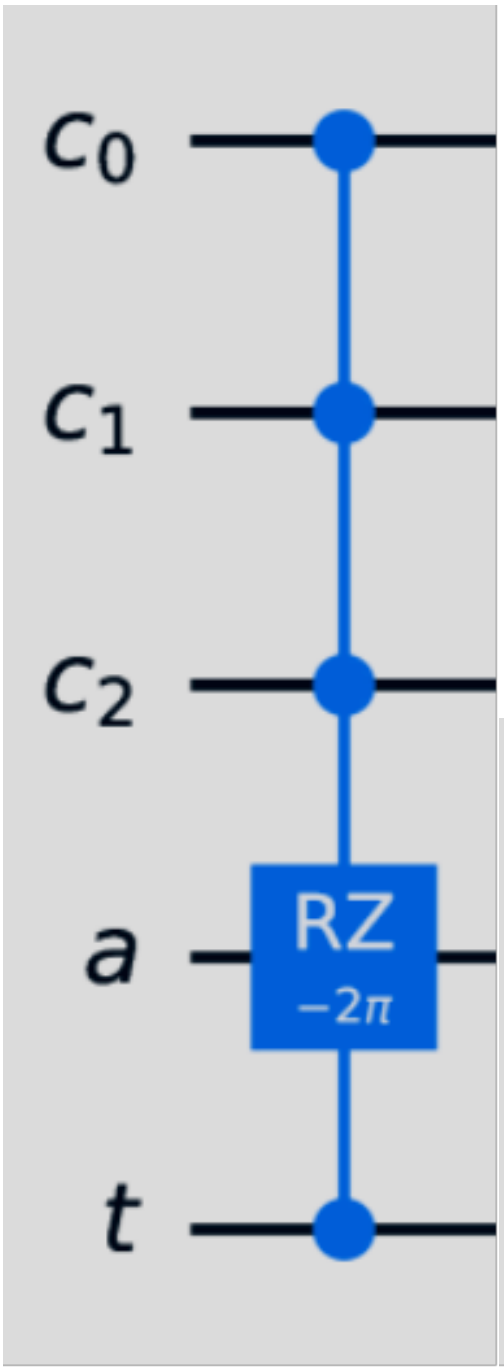
When multiple auxiliary qubits, or none, are available, the Network and Linear Depth decompositions can be used, respectively, as for most gates. However, when only a single auxiliary qubit is available, there are two decomposition algorithms available: SU(2) Rewrite (Rosa et al., 2025), which leverages the SU(2) Linear decomposition (Zindorf and Bose, 2024), and the Single Aux U(2), which uses the Single Aux Log decomposition (Claudon et al., 2024).

The SU(2) Rewrite decomposition relies on the fact that any U(2) gate can be represented by a SU(2) gate and a global phase, and that this global phase can be "corrected" by a multi-controlled RZ gate (Rosa et al., 2025). Figure 8(a) illustrates the decomposition of a multi-controlled Hadamard gate. For this decomposition, the two multi-controlled gates can be fused, resulting in the same complexity as decomposing a single multi-controlled SU(2) gate (Zindorf and Bose, 2024). For the multi-controlled Phase gate, illustrated in Figure 8(b), the decomposition can be simplified, but this results only in a constant reduction in the number of CNOTs (Zindorf and Bose, 2024).

other gate, but in many cases, a more efficient decomposition is available for Pauli and rotation gates.

The decompositions listed for the U(2) gate in Table 1 are used for any gate as presented in the previous sections, with the exception of the Linear Depth decomposition for rotation gates. In the next section, we present a benchmark of the decomposition algorithms discussed in this section.





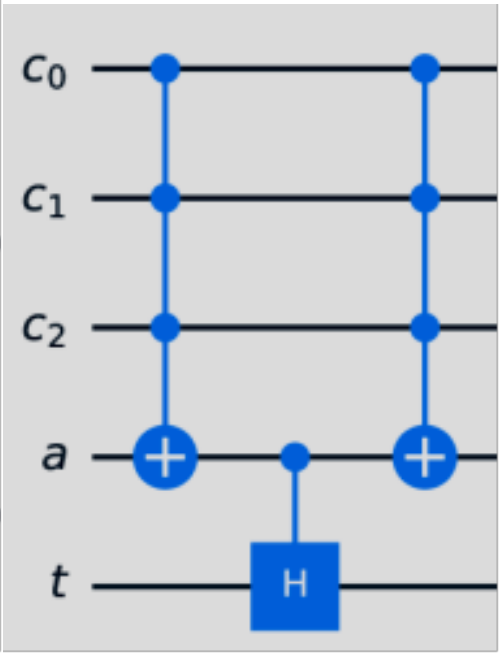


Figure 8. Decomposition for a 3-controlled Hadamard (LABEL:sub@fig:su2r:linear and LABEL:sub@fig:su2r:log) and a 3-controlled Phase(π) (LABEL:sub@fig:su2r:linear_p). In LABEL:sub@fig:su2r:linear, H is a SU(2) gate equivalent to H (Rosa et al., 2025).

\Description

5. Algorithms Benchmark

We implemented all the decomposition algorithms presented in Table 1 in the Ket quantum programming platform. This is the first implementation that consolidates the current state-of-the-art decomposition algorithms into a single platform, allowing us to collect numerical data from their execution. Our objective is to classify the performance of these algorithms in terms of compilation time and quantum execution time, creating compilation profiles where those key metrics will be taken into consideration. We classify the algorithms into two profiles: *Compilation Time* and *Execution Time*.

The *Compilation Time* profile, as the name suggests, is optimized to reduce the classical execution time of the decomposition algorithms, which may also result in improved execution time for simulated quantum executions. This profile is also suitable for NISQ computers (Preskill, 2018) with quantum programs with up to 400 qubits. The *Execution Time* profile targets large-scale Fault-Tolerant quantum computers (Devitt et al., 2013; Google Quantum AI and Collaborators et al., 2025) with the potential to compute with millions of qubits.

As Ket's compiler automatically manages auxiliary qubits whenever they are available on the quantum computer (Rosa et al., 2024), the data was gathered using high-level programming instructions such as "`ctrl(c, gate)(t)`", where `c` is a list of control qubits, `gate` is a single-qubit gate, and `t` is the target qubit. These qubits belong to a quantum processor with enough additional qubits available for use as auxiliary in the decomposition. No actual quantum execution was performed during the tests; only the decomposition was evaluated.

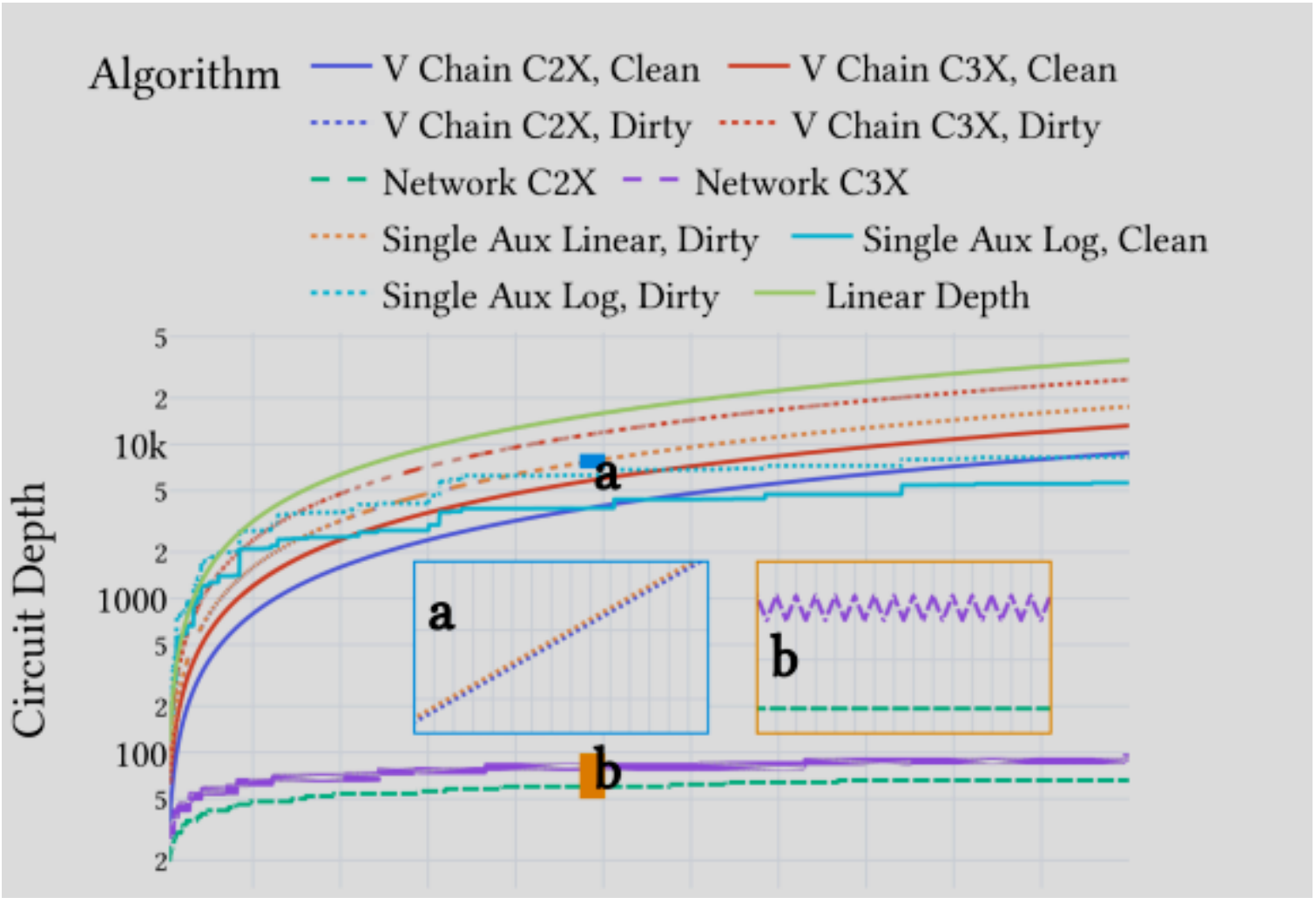
The primary metrics we evaluated in our benchmarks are the compilation and quantum execution times required to decompose an n-controlled quantum gate. We measure the compilation time based on the number of CNOTs and the quantum execution time based on the quantum circuit depth. The decomposition of multi-qubit gates, which is the focus of this study, is just one step in compiling a quantum program. Therefore, measuring the time in seconds to compile the code or decompose the gates may not be a suitable metric for compilation time. Instead, the number of CNOTs allows us to infer the impact of the decomposition on subsequent compilation steps, which have time complexity directly related to the number of CNOTs.

gle operation, and in some cases, they can be virtually implemented (McKay et al., 2017) with no impact on execution time. Also, the actual quantum execution time, in seconds, depends on hardware constraints such as qubit connectivity, which also affects the compilation step related to circuit mapping, as well as the availability of hardware resources to perform operations in parallel.

We organize our results using the same gate categories introduced in the previous section: Pauli gates, Rotation gates, and Phase and Hadamard gates. Next, we present the benchmark results for each group, followed by our analysis in Section 6.

Pauli Gates Benchmark

Figure 9 presents the number of CNOTs and circuit depth for multi-controlled Pauli gates. The data was obtained with the instruction "`ctrl(c, X)(t)`", but for the other Pauli gates, there is only a constant number of single-qubit gates' difference, which does not affect the presented data.



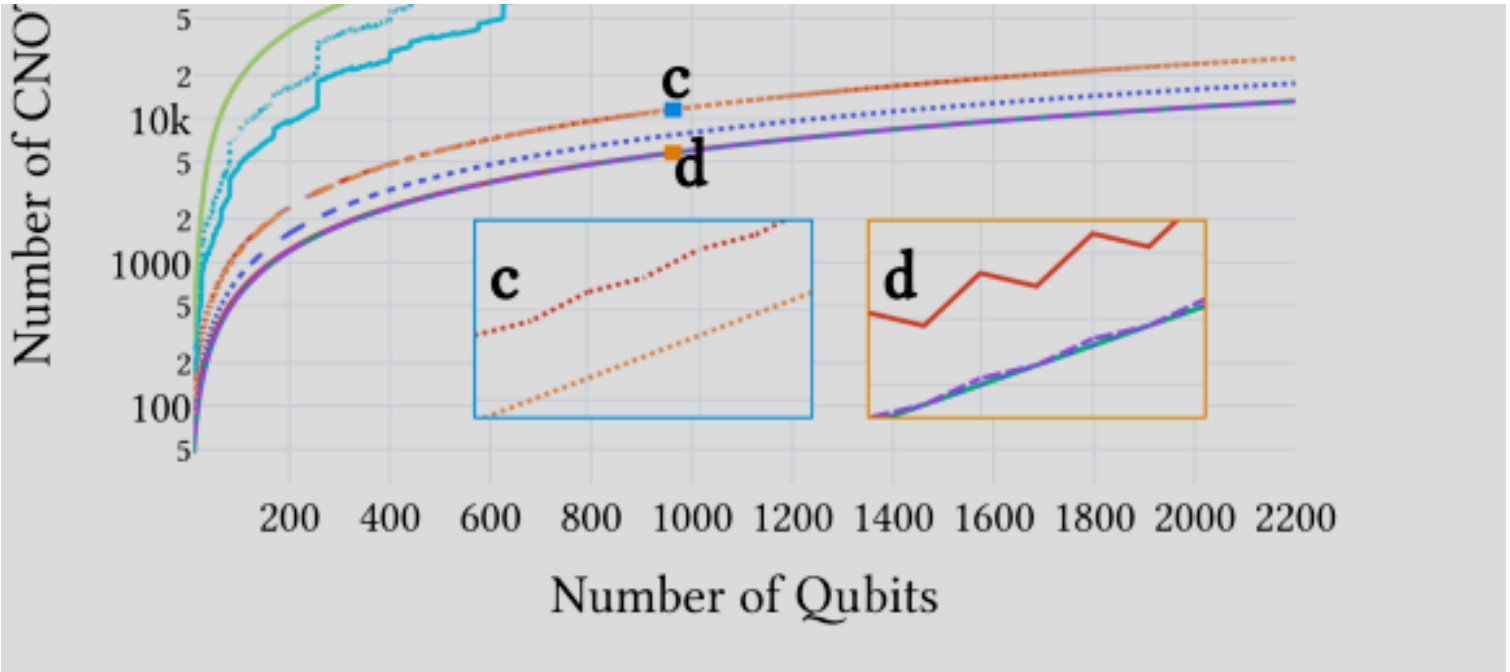


Figure 9. Comparison of various decomposition algorithms for *Multi-Controlled Pauli Gates* across two metrics: Circuit Depth and Number of CNOTs. Insets display zoomed-in views for specific regions with matching colors.

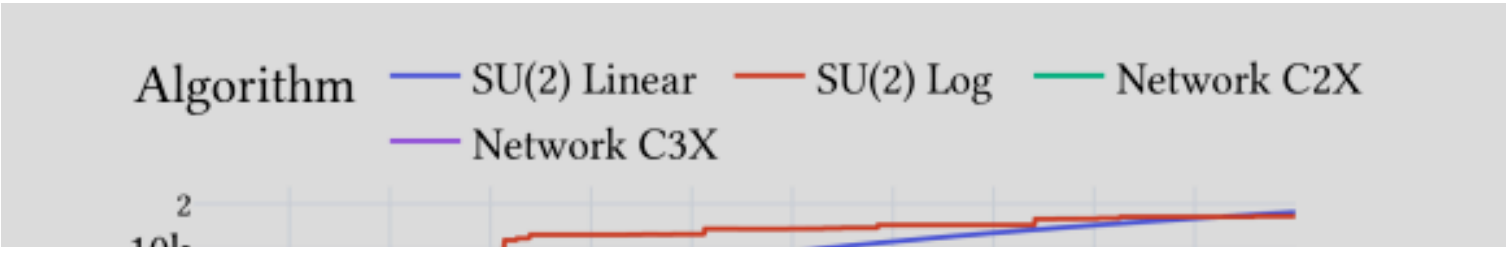
\Description

Rotation Gates Benchmark

Figure 10 presents the number of CNOTs and circuit depth for multi-controlled Rotation gates. The data was obtained using the instruction "`ctrl(c, RX(pi))(t)`"[4]

[4]We set `pi = 1.570796327`.

, but for the other Rotation gates, there is only a constant number of single-qubit gates' difference, which does not affect the presented data.



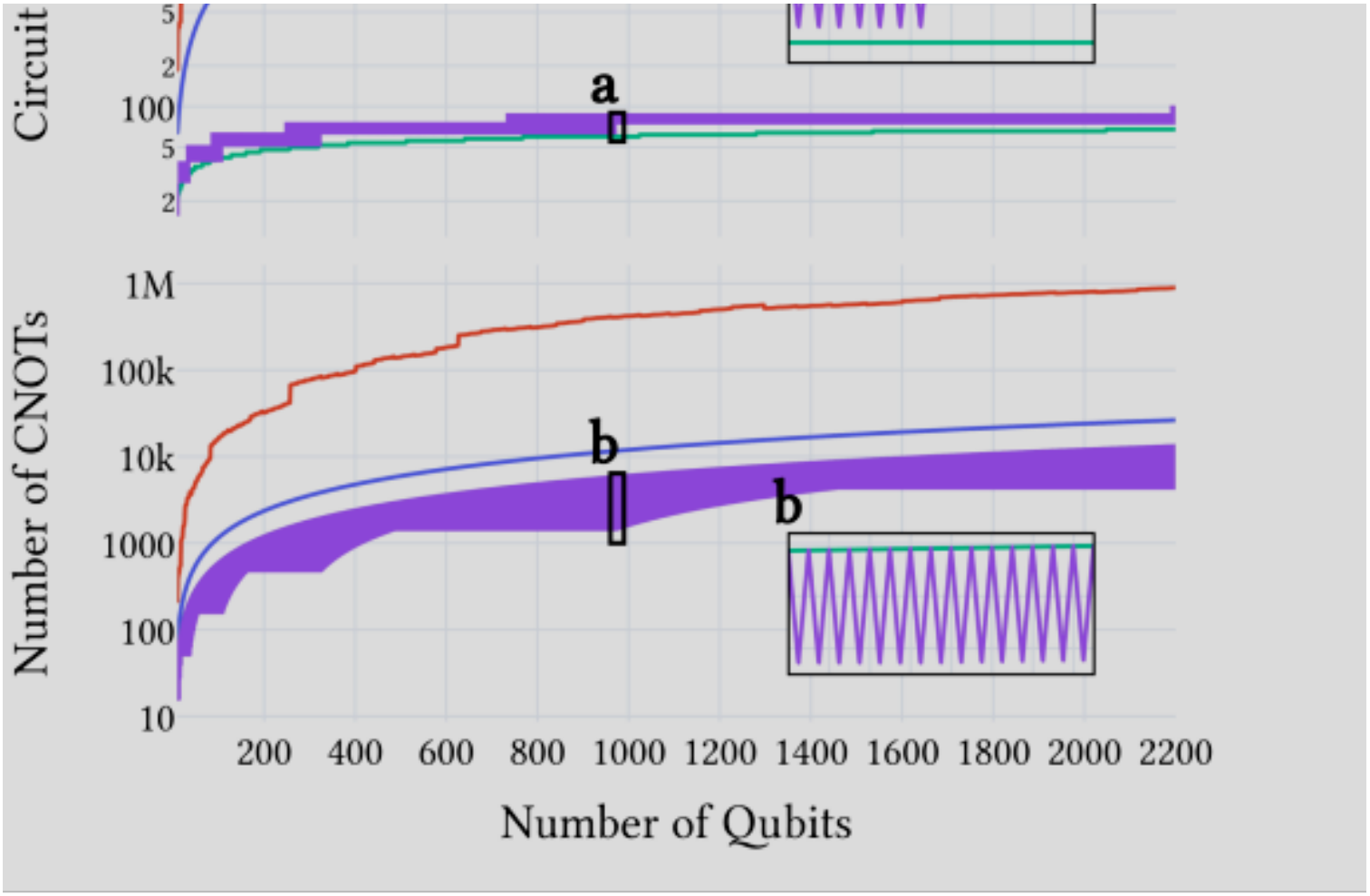


Figure 10. Comparison of various decomposition algorithms for *Multi-controlled Rotation Gates* across two metrics: Circuit Depth and Number of CNOTs. Insets display zoomed-in views for specific regions.

\Description

Phase and Hadamard Gates Benchmark

Figure 11 presents the number of CNOTs and circuit depth for multi-controlled Phase and Hadamard gates. The data was obtained with the instruction "`ctrl(c, H)(t)`", but for the Phase gates, there is only a constant number of CNOT gates' difference, which does not significantly affect the data.

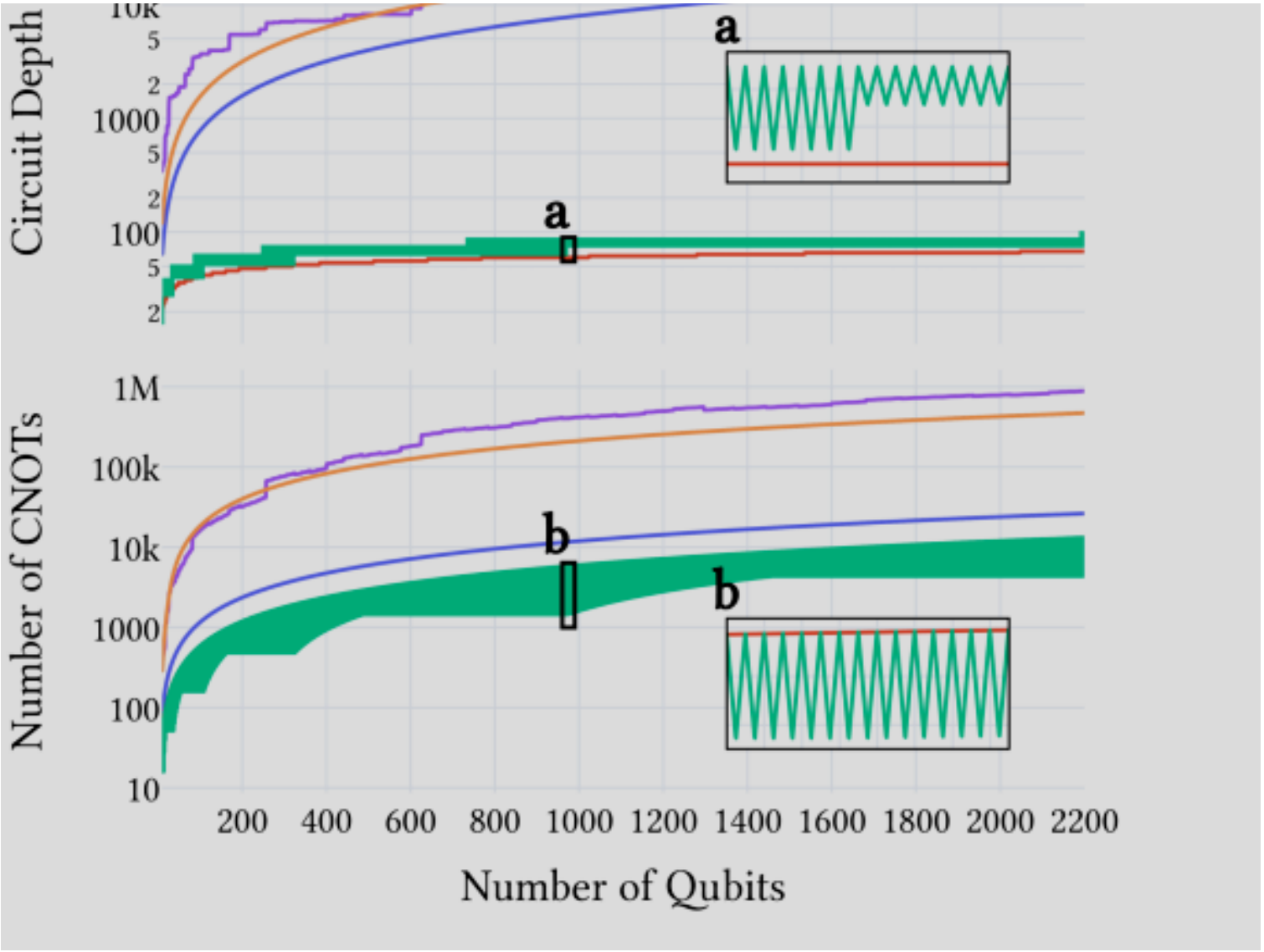


Figure 11. Comparison across different decomposition algorithms for *Multi-controlled Phase and Hadamard Gates* across two metrics: Circuit Depth and Number of CNOTs. Insets display zoomed-in views for specific regions.

\Description

6. Results Analysis

Based on the data presented in the previous section, Table 2 presents the quantum gate decomposition algorithms from Table 1, classified into the two compilation profiles. Note that not all algorithms are included in Table 2, as some decompositions with fewer auxiliary qubit requirements may be more efficient.

Gate	Compilation Time	Execution Time
Pauli Gates	1^{th} Network C2X	1^{th} Network C2X
	2^{nd} Network C3X	2^{nd} Network C3X
	3^{rd} V Chain C3X, Clean	3^{rd} Single Aux Log, Clean
	4^{th} V Chain C2X, Dirty	4^{th} Single Aux Log, Dirty
	5^{th} Single Aux Linear	5^{th} Linear Depth
	6^{th} Linear Depth	
Rotation Gates	1^{th} Network C3X	1^{th} Network C2X
	2^{nd} SU(2) Linear	2^{nd} Network C3X
		3^{rd} SU(2) Log
Phase and Hadamard	1^{th} Network C3X	1^{th} Network C2X
	2^{nd} SU(2) Rewrite	2^{nd} Network C3X
	3^{rd} Linear Depth	3^{rd} Single Aux U(2)
		4^{th} Linear Depth

The benchmark results reveal that the Network decomposition stands out as the most efficient algorithm across all gate types. However, this performance comes at the cost of requiring the largest number of clean auxiliary qubits. Interestingly, the initial assumption that increasing the number of auxiliary qubits would always lead to more efficient decompositions does not consistently hold. In some cases, additional auxiliaries provide no performance improvements.

Algorithms that produce logarithmic circuit depth demonstrate their advantages only in scenarios involving more than 1000 qubits. Despite their depth efficiency, these algorithms tend to perform poorly in terms of CNOT gate count, often ranking alongside the Linear Depth decomposition as the least efficient in this regard.

With the exception of rotation gates–which can be decomposed without auxiliary qubits and still maintain a linear number of CNOTs–most other gate types fall back on the Linear Depth decomposition when no auxiliary qubits are available. This makes Linear Depth a crucial strategy for current quantum compilers. However, as future quantum devices provide access to more qubits, the relevance of this method may decline.

A particularly noteworthy result from the benchmarks is the significant performance gap between using clean versus dirty auxiliary qubits. The data suggest that having access to a quantum processor with twice the number of qubits as needed by the program can dramatically reduce both compilation time and quantum execution

7. Final Remarks

In this paper, we addressed the trade-off between compilation time and quantum execution time in quantum programs. We argue that this trade-off is more important for quantum applications than for classical ones, as the compilation of a quantum program cannot be performed *a priori*.

Our analysis focuses on quantum gate decomposition, which is the first step in quantum code compilation. We have gathered data from the state-of-the-art quantum gate decomposition algorithms, which we implemented on the Ket quantum programming platform. To the best of our knowledge, this is the first implementation of all these algorithms within a single platform.

From the data presented in Section 5, we created two compilation profiles: one focused on reducing compilation time and the other on reducing quantum execution time. These results are presented in Table 2. The quantum execution time profile considers large-scale quantum computers that go beyond current capabilities but will be necessary to run algorithms such as Shor's algorithm (Shor, 1997), which could break RSA encryption (Gidney and Ekerå, 2021). Additionally, the presented data can be used to construct a lookup table that selects the best decomposition algorithm based on the number of qubits involved in a multi-qubit gate, as the performance may not be easily predictable for operations with fewer than 2200 qubits.

The results of this paper can help quantum compiler developers select the best decomposition algorithms for their compilers, depending on the type of gate and the number of control qubits. Note that the selection of the decomposition algorithm can be automatically performed by the compiler based on the available auxiliary qubits (Rosa et al., 2024). However, the findings from this paper can also aid quantum developers in identifying the performance associated with each multi-controlled gate, enabling them to make more informed decisions when selecting instructions to use.

In our study, we only evaluated the decomposition step of quantum compilation. However, circuit mapping may have a significant impact on the final performance of the quantum program. Circuit mapping adjusts the quantum program to the connectivity constraints of the quantum computer. This means that, regardless of the circuit mapping algorithm used, different quantum computer architectures (or qubit connectivity structures) may be better suited to different decomposition algorithms. Future work could analyze these decomposition algorithms across various qubit connectivity structures, such as line, grid, and torus configurations.

Another direction for future work is to analyze the performance of the decomposition algorithms in the context of non-Clifford gates (Aaronson and Gottesman, 2004), as these gates have a significant impact on quantum execution time when the program is encoded in a Quantum Error Correction code (Piveteau et al., 2021).

The Simplified Toffoli Gate Implementation by Margolus is Optimal

Guang Song* and Andreas Klappenecker
Department of Computer Science, Texas A&M University,
College Station, TX 77843-3112, USA
{gsong,klappi}@cs.tamu.edu

Abstract

Unitary operations are expressed in the quantum circuit model as a finite sequence of elementary gates, such as controlled-not gates and single qubit gates. We prove that the simplified Toffoli gate by Margolus, which coincides with the Toffoli gate up to a single change of sign, cannot be realized with less than three controlled-not gates. If the circuit is implemented with three controlled-not gates, then at least four additional single qubit gates are necessary. This proves that the implementation suggested by Margolus is optimal.

1 Introduction

The simplified Toffoli gate realizes the unitary map $M\colon \mathbf{C}^8 \to \mathbf{C}^8$ given by

$$\begin{aligned} |00\rangle \otimes |\phi\rangle &\mapsto |00\rangle \otimes |\phi\rangle, \\ |01\rangle \otimes |\phi\rangle &\mapsto |01\rangle \otimes |\phi\rangle, \\ |10\rangle \otimes |\phi\rangle &\mapsto |10\rangle \otimes Z|\phi\rangle, \\ |11\rangle \otimes |\phi\rangle &\mapsto |11\rangle \otimes X|\phi\rangle, \end{aligned}$$

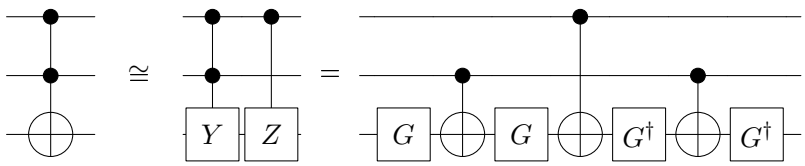
where $|\phi\rangle$ is an arbitrary state in $\mathbf{C}^2$, X denotes the not gate, and Z a phase gate,

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{1}$$

The unitary map M coincides with the Toffoli gate [1] on all vectors of the standard basis, except that it maps the state $|101\rangle$ to $-|101\rangle$ instead of $|101\rangle$; this strong resemblance explains the name.

*Guang Song's current address is: Baker Center for Bioinformatics and Biological Statistics, Iowa State University, Ames, IA 50011-3020, USA. Email: gsong@iastate.edu.

The simplified Toffoli gate has an elegant implementation [1], which is due to Margolus [4,2], see also [1,3]. It merely requires three controlled-not gates and four single qubit gates:



(2)

where

$$Y = ZX = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad G = \begin{pmatrix} \cos(\frac{1}{8}\pi) & -\sin(\frac{1}{8}\pi) \\ \sin(\frac{1}{8}\pi) & \cos(\frac{1}{8}\pi) \end{pmatrix}. \tag{3}$$

The congruence sign indicates equivalence up to a multiplication with a diagonal matrix of phase factors. There are some alternatives to M that differ in some other basis state by a sign, but that is not an essential change.

The simplified Toffoli gate cannot substitute for the Toffoli gate in general, because phase factors are important in true quantum algorithms. However, if the Toffoli gates appear in pairs, then it is possible to adapt the circuit structure to take advantage of the simplified Toffoli gate [1]. The saving are quite substantial in this case, because the best implementations of the Toffoli gate known to date need *fourteen* controlled-not and single qubit gates.

Our main result expresses our appreciation of the beautiful structure of the quantum circuit (2). We prove that the circuit is optimal in the following sense:

Theorem M *Suppose that the simplified Toffoli gate M is realized by a sequence of controlled-not and single qubit gates. Any such sequence contains at least three controlled-not gates. If it contains three controlled-not gates, then at least four single qubit gates are needed.*

Our proof reveals that the elegant structure of the circuit (2) is not an arbitrary artifact. Indeed, any optimal quantum circuit realizing M is essentially of this form, except that the single qubit gates are possibly different.

We followed the seminal paper [1] in our choice of the universal set of gates, because this has been adopted in several textbooks as well. Choosing controlled-not gates and single qubit gates is somewhat arbitrary, but similar arguments can be carried out for other universal sets of quantum gates. The main reason for our choice is that the number of controlled-not and single

qubit gates constitute the prevailing measure of complexity currently used in Computer Science. This paper is part of a larger program, where we try to gain a better understanding of basic quantum circuit structures.

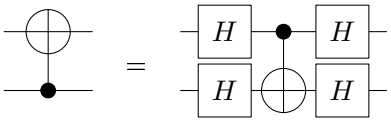
Notations. In addition to the gates introduced in (1) and (3), we use

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

to denote the Hadamard gate. The state $|0_x\rangle = H|0\rangle$ denotes the eigenstate of X with eigenvalue $+1$. The most significant qubit is represented by the topmost wire in the quantum circuit notation, and the least significant qubit by the lowest wire. We denote by $\mathbf{C}$ the field of complex numbers, and by $\mathbf{R}$ the field of real numbers.

2 Review of Previous Work

We review in this section some useful lemmas, all of which are proved in [5]. We will use these results in the proof of Theorem M. Recall that it is possible to switch the control and the target qubit of a controlled-not gate by conjugation with Hadamard matrices H:



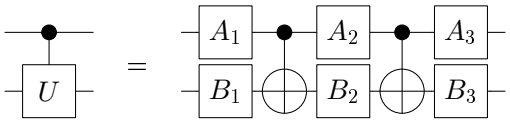
(4)

Due to this important fact, it suffices to consider controlled-not gates where the control is on a higher significant qubit that the target qubit when we write down the general form of a circuit.

Lemma 1 *Let $|\psi\rangle$, $|\phi\rangle$ be nonzero elements of $\mathbf{C}^2$. The input $|\psi\rangle \otimes |\phi\rangle$ to a controlled-U gate will produce an entangled output state if and only if $|\phi\rangle$ is not an eigenvector of U and $|\psi\rangle = a|0\rangle + b|1\rangle$ with $a, b \neq 0$.*

Lemma 2 *Assume that $|\phi\rangle$ is an eigenvector of a unitary 2×2 matrix U with eigenvalue λ_ϕ. Let $|\psi\rangle$ denote a state in $\mathbf{C}^2$. If we input $|\psi\rangle \otimes |\phi\rangle$ to the controlled-U gate, then the output is of the form $\mathrm{diag}(1, \lambda_\phi)|\psi\rangle \otimes |\phi\rangle$. In particular, the output is not entangled.*

A controlled-U gate can be realized with two controlled-not gates and several single qubit gates, as follows:



(5)

The following two lemmas describe some constraints on the gates A_1, A_2, and A_3. We call a single qubit gate sparse if it is realized by a diagonal or antidiagonal 2×2 matrix.

Lemma 3 *If the matrix A_1 is sparse, then A_2, A_3 are sparse as well.*

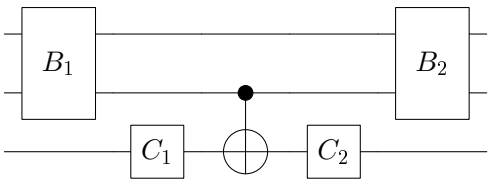
Lemma 4 *Suppose that U is not a multiple of the identity matrix. If A_1 in the circuit (5) is not sparse, then A_2, A_3 are not sparse either.*

3 Proof of Theorem M

We proceed to show that three controlled-not gates are necessary and sufficient in any realization of unitary map M by a sequence of controlled-not and single qubit gates. We first prove that at least two controlled-not operations act on the last qubit:

Lemma 5 *Suppose there are some interactions between the top two qubits, but only one controlled-not interaction between the two control qubits and the target bit. The circuit cannot realize the Margolus map M.*

Proof. Any such circuit can be represented in the form



(6)

Let $|0_x\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, and denote by $|\varphi\rangle$ an arbitrary state in $\mathbf{C}^4$. If we input $|\varphi\rangle \otimes C_1^\dagger |0_x\rangle$ to the above quantum circuit, then the least significant qubit of the output state is not entangled with the remaining two qubits, regardless of the nature of input state $|\varphi\rangle$.

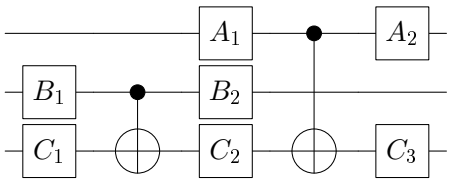
However, if we choose the input $|00\rangle \otimes |\phi\rangle + |10\rangle \otimes |\phi\rangle + |11\rangle \otimes |\phi\rangle$, then the output of M is $|00\rangle \otimes |\phi\rangle + |10\rangle \otimes Z|\phi\rangle + |11\rangle \otimes X|\phi\rangle$. Note that the

target qubit is entangled with the other two qubits, since $|\phi\rangle$ cannot be an eigenvector of X and Z at the same time. Contradiction. □

Corollary 6 *The target qubit is affected by at least two controlled-not operations.*

Two Controlled-Not Gates. Assume there are only two controlled-not gates in the circuit. Taking Corollary 6 and the identity (4) into account, we may assume that both controlled-not gates operate on the target bit. The control qubits of the two gates have to be different, for otherwise it would not be possible to entangle all input qubits with the outout qubit.

Since $M = M^\dagger$, we do not need to concern ourselves with the order the two controlled-not gate in such a circuit. Therefore, we may assume that the circuit is of the form



(7)

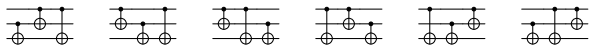
Lemma 7 *The circuit (7) cannot implement the simplified Toffoli gate.*

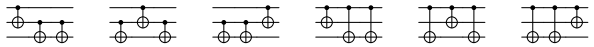
Proof. When the top qubit is $|0\rangle$, then the circuit (7) can still entangle the least significant two qubits, contradicting the behavior of M. □.

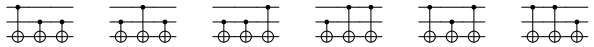
Corollary 8 *At least three controlled-not gates are necessary in an implementation of the simplified Toffoli gate M by a sequence of controlled-not and single qubit gates.*

Three Controlled-Not Gates. The remaining argument proceeds by considering all possible configurations of the three controlled-not gates. Initially, we allow an arbitrary number of single qubit operations. Thus, we may assume that the target qubit has lesser significance than the control qubit by applying (4), so that we have to consider $\binom{3}{2}^3 = 27$ configurations of controlled-not gates. We use the pictogram

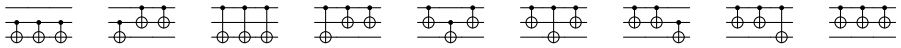
as a shorthand for a general quantum circuit of the form (8) that contains in addition to the specified controlled-not configuration all potential single qubit gates. We distinguish three different cases that we record here for the orientation of the reader:

Case 1 

Case 2 

Case 3 

The remaining configurations



are excluded because they are ruled out by Corollary 6 or lack the capability to entangle all three qubits, hence cannot implement M.

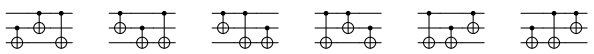
It turns out that only a single configuration allows to realize the simplified Toffoli gate M. In most cases, we are able to exclude circuit structures because they exhibit entanglement properties that are inconsistent with the behavior of the simplified Toffoli gate M. We record the following trivial observation:

Lemma 9 *Suppose that two systems A and B of qubits are entangled. If the remaining gate operations affect the systems A and B separately, then A and B remain entangled.*

Proof. Seeking a contradiction, we assume that the resulting output state is not entangled, i.e., is of the form $|\phi_A\rangle \otimes |\phi_B\rangle$. The gate operations leading to this output state can be written in the form $U_A \otimes U_B$, since the operations affect the systems separately. This would imply that the input state $U_A^\dagger|\phi_A\rangle \otimes U_B^\dagger|\phi_B\rangle$ was separable as well, in contradiction to the assumption. □

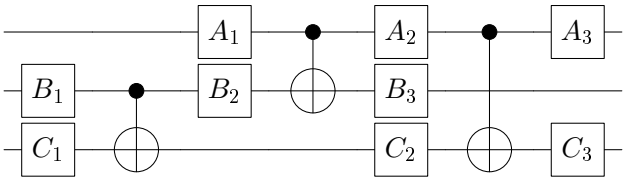
Case 1. The circuits configuration are in this case characterized by the fact that exactly two controlled-not gates act on the target qubit of M, and they are controlled from different qubits. The third controlled-not gate is between the two most significant qubits. It turns out that none of these circuits can implement M, even if we allow single qubit gates on all possible positions.

Lemma 10 *A circuit with one of the configurations*



cannot implement the simplified Toffoli gate M.

Proof. The most general circuit corresponding to the first pictogram is given by

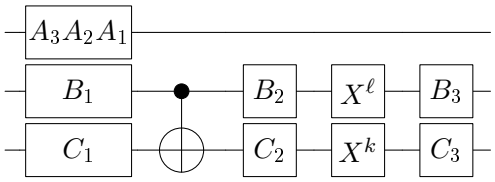


(8)

Seeking a contradiction, we assume that A_1 is not sparse. If we provide an input $|0\rangle \otimes |\phi\rangle \otimes C_1^\dagger |0_x\rangle$, then the top two quantum bits could possibly get entangled by Lemma 1, and if so, these two qubits would remain entangled by Lemma 9, contradicting the behavior of M. Hence, A_1 must be sparse.

By the same token, A_2 has to be sparse, because otherwise we can find an input state of the form $|0\rangle \otimes |\varphi\rangle$, such that the most and least significant qubit get entangled, contradicting the behavior of M. As a consequence, A_3 is sparse as well.

Therefore, the behavior of the circuit (8) on input of $|0\rangle \otimes |\varphi\rangle$, $|\varphi\rangle \in \mathbf{C}^4$, can be simulated by a circuit of the form



(9)

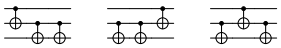
where the values of $k, \ell \in \{0, 1\}$ depend on whether A_1 and A_2 are diagonal or antidiagonal. It is obvious from this circuit that we can choose a separable state $|\varphi\rangle$ such that the two least significant qubits get entangled, even though the topmost qubit is in the state $|0\rangle$. This contradicts the behavior of M, thus a circuit of the form (8) cannot realize M.

In the same way, it is straightforward to see that neither the second nor the third pictogram can realize M.

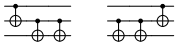
Finally, the last three pictograms represent the inverse circuits of the first three, so none of them can realize M since M is self-inverse. □

Case 2. The circuit configurations of the second case are characterized by the fact that exactly two controlled-not gates act on the target qubit, and both are controlled from the same qubit. We distinguish in our discussion whether they are controlled by the middle qubit (*Case 2.1*), or by the most significant qubit (*Case 2.2*).

Case 2.1. This case treats the configurations of controlled-not gates that have the pictorial representation

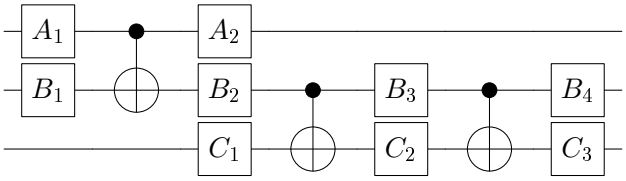


Lemma 11 *A circuit with one of the controlled-not configurations*



cannot implement the simplified Toffoli gate M.

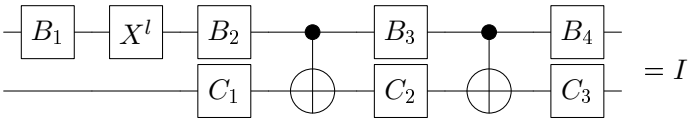
Proof. The most general circuit corresponding to the first pictogram is given by



(10)

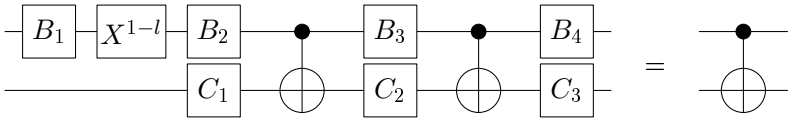
The second pictogram is covered by taking the inverse of the above circuit, hence does not need to be treated separately. If we input $|0\rangle \otimes B_1^\dagger|0\rangle \otimes |0\rangle$, then the circuit will produce an entangled output state by Lemmas 1 and 9 if A_1 is not sparse. Hence, A_1 has to be sparse, and consequently A_2 as well.

If we take the input state $|0\rangle \otimes |\varphi\rangle$, with $|\varphi\rangle \in \mathbf{C}^4$, then the circuit (10) has to act identically on $|\varphi\rangle$. Consequently, we obtain the circuit identity



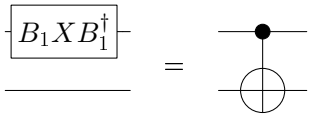
(11)

On the other hand, we can derive from the input $|1\rangle \otimes |\varphi\rangle$ the circuit identity



(12)

Combining the previous two circuit identities, we obtain

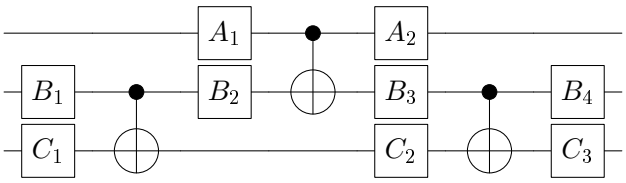


which is absurd. Therefore, a circuit of the form (10) cannot implement M. □

Lemma 12 *A circuit with controlled-not configuration*

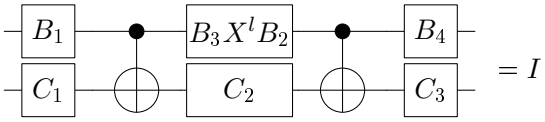
cannot implement the simplified Toffoli gate M.

Proof. The most general circuit corresponding to the configuration depicted in the last pictogram is given by



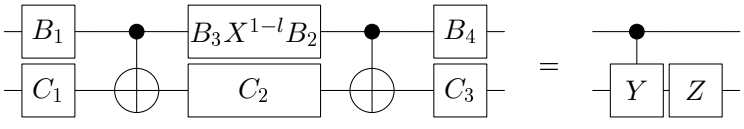
(13)

Once again, we want to show that this circuit structure cannot implement M. We note that A_1 and A_2 have to be sparse (for otherwise it would be possible to find a state $|\varphi\rangle \in \mathbf{C}^4$ such that $|0\rangle \otimes |\varphi\rangle$ leads to an entangled output state, arguing as in the previous case). The circuit (13) has to act as the identity an input state $|0\rangle \otimes |\varphi\rangle$. Thus, we obtain the circuit identity



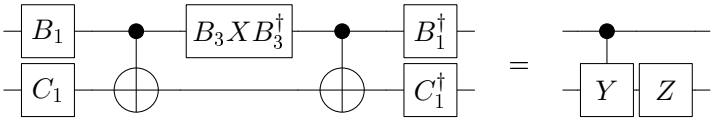
(14)

Similarly, the input $|1\rangle \otimes |\phi\rangle \otimes |\psi\rangle$ leads to the circuit identity



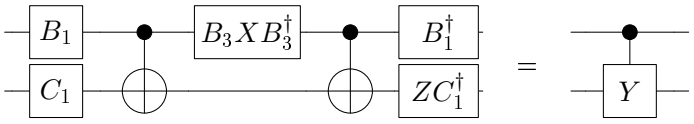
(15)

From circuits (14) and (15), we have,



(16)

Moving Z to the left-hand side of the equation, we have



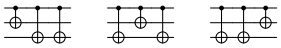
(17)

Now for any input state $|\phi\rangle \otimes (C_1)^\dagger|0_x\rangle$, such that $|\phi\rangle = a|0\rangle + b|1\rangle$ is in superposition, $a, b \neq 0$, the circuit on the left hand side does not entangle the two qubits. Therefore, by Lemma 11, we know that $(C_1)^\dagger|0_x\rangle$ has to be an eigenvector of Y. Furthermore, by Lemma 2, we have

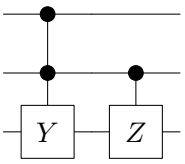
$$B_1^\dagger B_3^\dagger X B_3 B_1 = \mathrm{diag}(1, y_0), \tag{18}$$

where y_0 is one the eigenvalues of Y, i.e., $y_0 = (-i)$ or i. Taking the trace on both sides, we get a contradiction. □

Case 2.2. We now treat the configurations that have the pictorial representation

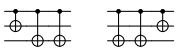


Changing the role of the two most significant qubits, we arrive at simplified Toffoli gate M' that is implemented by



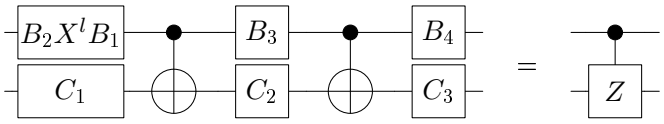
Thus, it is equivalent to use the circuits (10) and (13) of the previous case to implement M'.

Lemma 13 *Circuits with configuration*



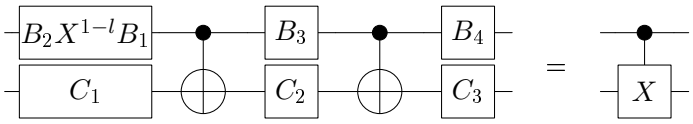
cannot realize the Toffoli gate M.

Proof. We use the above trick and show that circuit (10) cannot implement M'. We derive from the input state $|0\rangle \otimes |\phi\rangle \otimes |\psi\rangle$ the circuit identity



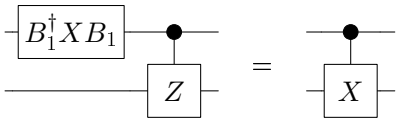
(19)

Similarly, the input state $|1\rangle \otimes |\phi\rangle \otimes |\psi\rangle$ yields



(20)

We can deduce from circuits (19) and (20) the relation



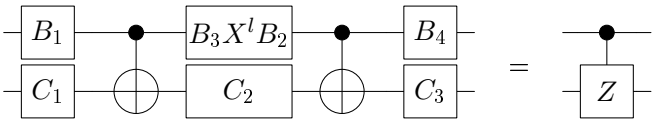
(21)

which clearly leads to a contradiction. □

Lemma 14 *Circuits with configuration*

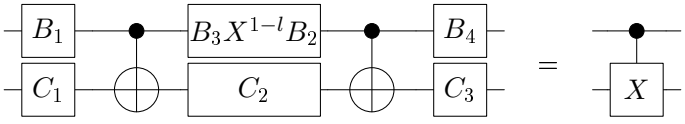
cannot realize the Toffoli gate M.

Proof. It suffices to show that circuit (13) cannot implement M'. Considering the input state $|0\rangle \otimes |\phi\rangle \otimes |\psi\rangle$, we obtain the circuit identity



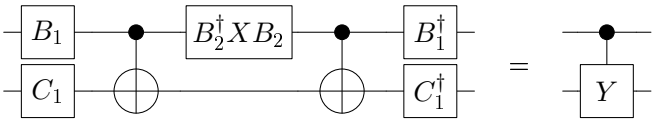
(22)

And with input $|1\rangle \otimes |\phi\rangle \otimes |\psi\rangle$, we get



(23)

Combining circuits (22) and (23), we have,

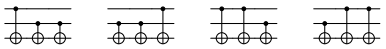


(24)

Similar to the proof for circuit (17), we again arrive at a contradiction after considering the trace. □

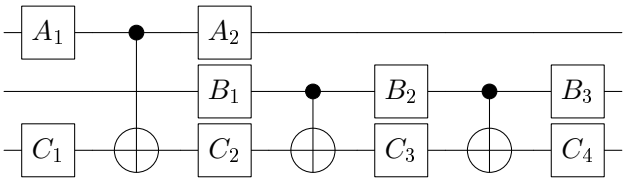
Case 3. We now examine five of the remaining six configurations.

Lemma 15 *Circuit configurations of the form*



cannot implement the simplified Toffoli gate M.

Proof. The most general circuit corresponding to the first configuration in the statement of the lemma is of the form



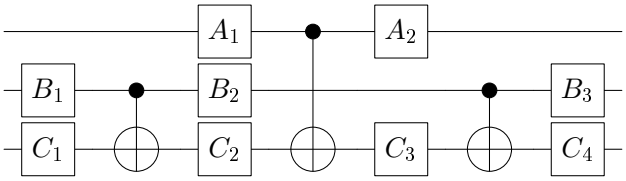
(25)

Following the same steps as in the proofs of Lemmas 11 and 13, it is straightforward to see that such a circuit cannot realize M nor M'. Therefore, the first and the third configuration can be ruled out. The other two configurations can be ruled out by realizing that M is self-inverse. □

Lemma 16 *The circuit configuration*

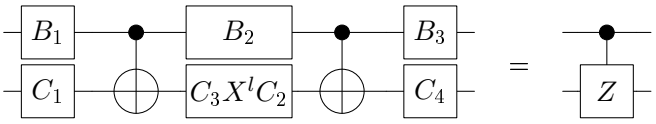
cannot implement the simplified Toffoli gate M.

Proof. It suffices to prove that the circuit



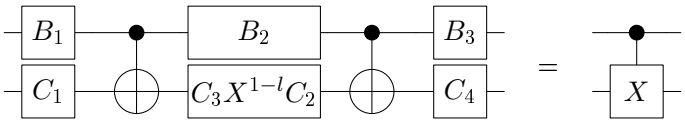
(26)

cannot realize M'. As in Lemma 11, we note that both A_1 and A_2 have to be sparse. Considering input states of the form $|0\rangle \otimes |\phi\rangle \otimes |\psi\rangle$, it follows from circuit (26) that the circuit identity



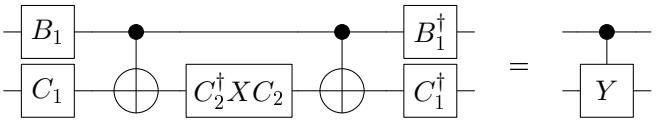
(27)

must hold. We derive from the action on the input $|1\rangle \otimes |\phi\rangle \otimes |\psi\rangle$ the circuit identity



(28)

Combining circuits (27) and (28), we have,



(29)

The right hand side of (29) cannot produce entangled output states when provided with the input states $B_1^\dagger|0\rangle \otimes |\phi\rangle$ and $B_1^\dagger|1\rangle \otimes |\phi\rangle$. It follows that either

$$C_1^\dagger C_2^\dagger X C_2 C_1 = I \tag{30}$$

or

$$C_1^\dagger X C_2^\dagger X C_2 X C_1 = I \tag{31}$$

holds. Taking the trace on both side, we arrive at a contradiction in either case. □

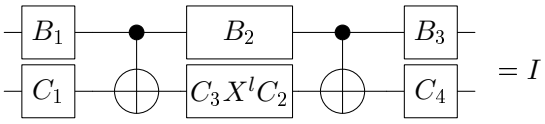
Final Step. We have now excluded 26 of the 27 possible control-not configurations. We know that the only viable configuration

actually allows to realize the simplified Toffoli gate M. The general circuit associated with the circuit configuration is of the form (26).

Lemma 17 *At least four single qubit gates are necessary in any realization of the simplified Toffoli gate M with three controlled-not gates.*

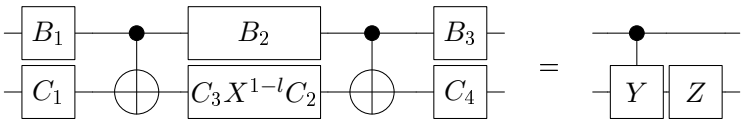
Proof. The only viable general circuit structure with three controlled-not gate is given by the circuit (26). We will show that at least four single qubit gates in this circuit differ from multiples of the identity, and that the gate count cannot be reduced by flipping the gates using (4).

A circuit (26) realizing M is supposed to leave an input state of the form $|0\rangle \otimes |\varphi\rangle$ invariant. This implies the circuit identity



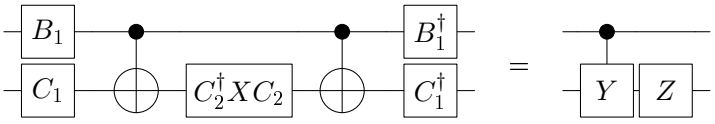
(32)

And we obtain from the action on $|1\rangle \otimes |\varphi\rangle$ the identity



(33)

Combining circuits (32) and (33), we obtain the equality



(34)

It follows from Lemma 4 that B_1 has to be sparse. By Lemma 3, this implies that the gates B_2 and B_3 in circuit (33) have to be sparse as well. Since we know that A_1 and A_2 are sparse, flipping any number of controlled-not gates in circuit (26) using equality (4) cannot decrease the count of the single-qubit gates. It remains to show that none of the four gates $C_1, \ldots, C_4$ in circuit (26) can be a multiple of the identity.

Since B_1 is sparse, let $B_1 = \mathrm{diag}(e^{i\phi_0}, e^{i\phi_1})X^k$. Considering the input state $|0\rangle \otimes |\phi\rangle$ and $|1\rangle \otimes |\phi\rangle$, we can deduce from circuit (34) the equalities

$$C_1^\dagger X^k C_2^\dagger X C_2 X^k C_1 = Z, \tag{35}$$

$$C_1^\dagger X^{1-k} C_2^\dagger X C_2 X^{1-k} C_1 = X. \tag{36}$$

If follows from equations (35) and (36) that neither C_1 nor C_2 can be a multiple of the identity. Since $M = M^\dagger$, we can apply the same argument to the inverse of the circuit (26), hence neither C_3 nor C_4 is a multiple of the identity either. Therefore, at least four single qubit gates are non-trivial, as claimed. □

4 Conclusions

We have demonstrated that the simplified Toffoli gate by Margolus cannot be realized with fewer than three controlled-not gates. Four additional single qubit gates are required when M is realized with minimal number of controlled-not gates. Our proof of this lower bound revealed the interesting fact that the solution by Margolus is essentially uniquely determined by these constraints. The tedium of cases in lower bound proofs can be daunting, but one usually gains valuable structural insights, particularly if

the bounds are tight. It would be interesting to know tight lower bounds for other fundamental constructions of quantum circuits, such as the Toffoli gate. It is conjectured that the Toffoli gate cannot be implemented with less than six controlled-not and eight single qubit gates.

Acknowledgments. The research by A.K. was supported by NSF grant EIA 0218582 and a Texas A&M TITF initiative.

References

[1] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. Elementary gates for quantum computation. *Phys. Rev. A*, 52:3457–3467, 1995.

[2] D.P. DiVincenzo. Quantum gates and circuits. *Proc. R. Soc. Lond. A*, 454:261–276, 1998.

[3] N. Margolus. Universal cellular automata based on the collisions of soft spheres. In A. Adamatzky, editor, *Collision Based Computation*, page 107. Springer-Verlag, 2002.

[4] N. Margolus. Simple quantum gates. Unpublished manuscript, about 1994.

[5] G. Song and A. Klappenecker. Optimal realizations of controlled unitary gates. *Journal of Quantum Information and Computation*, 3(2):139–156, 2003.